

文件編號：ICST-C-026

103年資安服務暨專案管理辦公室

安全軟體測試參考指引

(V1.0)

執行單位：財團法人資訊工業策進會

中華民國103年12月

報告摘要

報告名稱	安全軟體測試參考指引
資訊等級	☐機密　☐密　☐內部使用　☒普通
相關撰稿人	陳彥銘、盧紹榮、林志堯
閱讀對象	☒一般主管　☒資安人員　☒資訊人員　☐一般使用者

內容摘要：

一、目的

本指引針對系統安全測試流程作法進行探討，歸納出進行系統安全測試之實行重點，並整理與測試相關之國際標準，包含如 ISO/IEC 27001:2013、IEEE 829:2008、ISO/IEC/IEEE 29119:2013，以及 OWASP Testing Guide 等。並針對安全需求進行探討，歸納出驗證安全需求實務活動之重點，最後介紹常用的安全性測試技術，包含源碼靜態分析以及滲透測試，以協助相關人員了解進行軟體安全測試時應該注意的事項及進行方式。

本指引主要為提供機關於進行安全功能驗證之基礎，可作為自行開發或委外管理系統的參考依據，期能落實安全軟體生命週期之實踐，透過測試以驗證整體資訊系統之安全性，強化政府整體安全軟體發展能量。

二、適用對象

本參考指引適用於軟體發展領域之所有對象，包含一般主管、資安人員及資訊人員，並針對不同對象建議說明其使用效益。

- 一般主管，例如資訊部門主管或軟體專案管理者，為確保軟體開發專案或組織使用之現有系統的安全性，可以使用這份指引來計畫安全測試活動，並可供擬訂軟體需求書或合約時參考使用，適用於自行開發或委外開發之軟體專案或系統。

- 資安人員，例如安全測試人員，為實施組織資訊安全部門所屬人員，必須熟悉資訊安全理論與規範，接受資訊開發人員之諮詢，並於開發完成後負責驗證軟體的安全需求是否已被滿足，可以使用這份指引了解安全驗證的施行項目以及學習安全測試的技術來增強安全性檢測能力。
- 資訊人員，例如軟體開發者，為確保進行資訊系統程式於開發與管理時符合資訊安全相關規範的需求與水準，可以使用這份指引來了解軟體可能潛藏的資安弱點與可能面臨的安全威脅，以及驗證所遞交的源碼之安全性。

三、章節架構

第 1 章說明本指引之目的、適用對象及章節架構介紹。

第 2 章介紹安全軟體測試的要項，以及相關國際標準與常見的 Web 應用程式弱點。

第 2 章第 1 節說明安全軟體測試所應注意的事項。

第 2 章第 2 節介紹「安全軟體測試相關國際標準」，如 ISO/IEC 27001:2013、IEEE 829: 2008、ISO/IEC/IEEE 29119:2013，以及 OWASP 測試指引等，作為本指引參考的文獻。

第 2 章第 3 節介紹「常見的 Web 應用程式弱點」，針對國際資安組織所分類的弱點及威脅進行介紹，如 OWASP Top 10、WASC 威脅分類、CWE/SANS Top 25。

第 2 章第 4 節「結語」，總結第 2 章內容。

第 3 章「安全需求驗證」介紹如何針對安全需求進行驗證。

第 3 章第 1 節介紹安全需求的原則、類型、正面需求與負面需求，最後介紹安全需求的萃取方式。

第 3 章第 2 節說明安全需求驗證實務活動的要項，包含 OWASP 應用程式

安全驗證標準以及安全軟體發展生命週期內的安全需求驗證。 第 3 章第 3 節介紹網頁自動化測試可以利用的工具，以避免使用人工進行重複性的測試，提高測試效率。 第 3 章第 4 節「結語」，總結第 3 章內容。 第 4 章針對安全性測試技術進行介紹，包含靜態源碼分析、滲透測試，以及灰箱安全性測試。 第 4 章第 1 節「源碼靜態分析」，說明源碼安全審查及自動化工具檢測的進行要點。 第 4 章第 2 節「滲透測試」，說明進行滲透測試的流程以及可利用的工具。 第 4 章第 3 節「灰箱安全性測試」介紹灰箱測試的概念。 第 4 章第 4 節「結語」，總結第 4 章內容。 第 5 章「案例分析」，以實際系統為範例進行安全測試，測試活動包含安全需求驗證、源碼安全性檢測，以及弱點掃描。 第 6 章「結論」，總結各章節內容。 第 7 章「參考文獻」，詳列本指引參考的文件資料。	
關鍵詞	安全軟體發展生命週期、SSDLC、安全軟體測試、安全需求驗證

目　次

1. 前言 1
1.1. 目的 1
1.2. 適用對象 1
1.3. 章節架構 1
2. 安全軟體測試介紹 4
2.1. 安全軟體測試 4
2.2. 安全軟體測試相關標準介紹 7
2.3. 常見的 Web 應用程式弱點 33
2.4. 結語 40
3. 安全需求驗證 41
3.1. 安全軟體需求 41
3.2. 安全需求驗證實務活動 53
3.3. 網頁自動化測試 60
3.4. 結語 66
4. 安全性測試技術 67
4.1. 源碼靜態分析 67
4.2. 滲透測試 87
4.3. 灰箱安全性測試 107
4.4. 結語 107
5. 案例分析 109
5.1. 案例簡介 109
5.2. 安全需求 109
5.3. 安全測試計畫 109
5.4. 結語 115

6. 結論 116
7. 參考文獻 117

圖 目 次

圖 1 缺陷修復成本 5
圖 2 IEEE 829 測試文件架構圖 11
圖 3 ISO/IEC/IEEE 29119 測試過程 13
圖 4 測試計畫的過程 14
圖 5 測試監測和控制的過程 17
圖 6 測試完成的過程 18
圖 7 測試設計與實現過程 20
圖 8 測試環境建立與維護過程 21
圖 9 測試執行過程 22
圖 10 測試事故報告過程 23
圖 11 測試文件架構圖 26
圖 12 ISO/IEC/IEEE 29119 測試技術 28
圖 13 OWASP 測試框架工作流程 30
圖 14 軟體安全需求類型 43
圖 15 誤用案例範例 50
圖 16 Selenium 範例 63
圖 17 Geb 程式範例 64
圖 18 Sikuli 腳本範例 66
圖 19 Netbeans Inspect 81
圖 20 FindBugs 檢測畫面 82
圖 21 IBM Security AppScan Workflow 87
圖 22 滲透測試的基本流程 89
圖 23 HP Fortify SCA 3.8 對管理維護網站掃描果 113
圖 24 WVS 對管理維護網站掃描結果 114

表 目 次

表 1 OWASP Top 10:2013 34
表 2 WASC Threat Classification v2.0: Attack list 36
表 3 WASC Threat Classification v2.0: Weaknesses list 37
表 4 CWE / SANS Top 25 38
表 5 單元測試工具 58
表 6 findstr 指令說明 70
表 7 grep 指令說明 71
表 8 常用的檢測弱點 76
表 9 FindBugs 檢測範例 83
表 10 測試項目範例 90
表 11 Metasploit 版本 103
表 12 測試項目範例 111

1. 前言

1.1.目的

本指引針對安全軟體發展生命週期中之安全軟體測試進行深入介紹。文中針對 ISO 及 IEEE 等國際標準、資安組織與單位制定之測試方法論與流程進行介紹與分析，並說明安全需求驗證的實務活動，以及安全測試技術包含源碼靜態分析、滲透測試，以及灰箱安全性測試等，作為機關與單位進行安全軟體發展生命週期之推動參考。

1.2.適用對象

本參考指引適用於軟體發展領域之所有對象，包含一般主管、資安人員及資訊人員，並針對不同對象建議說明其使用效益。

- 一般主管，例如資訊部門主管或軟體專案管理者，為確保軟體開發專案或組織使用之現有系統的安全性，可以使用這份指引來計畫安全測試活動，並可供擬訂軟體需求書或合約時參考使用，適用於自行開發或委外開發之軟體專案或系統。
- 資安人員，例如安全測試人員，為實施組織資訊安全部門所屬人員，必須熟悉資訊安全理論與規範，接受資訊開發人員之諮詢，並於開發完成後負責驗證軟體的安全需求是否已被滿足，可以使用這份指引了解安全驗證的施行項目以及學習安全測試的技術來增強安全性檢測能力。
- 資訊人員，例如軟體開發者，為確保進行資訊系統程式於開發與管理時符合資訊安全相關規範的需求與水準，可以使用這份指引來了解軟體可能潛藏的資安弱點與可能面臨的安全威脅，以及驗證所遞交的源碼之安全性。

1.3.章節架構

第 1 章說明本指引之目的、適用對象及章節架構介紹。

本文件之智慧財產權屬行政院資通安全辦公室所有。

第 2 章介紹安全軟體測試的要項，以及相關國際標準與常見的 Web 應用程式弱點。

第 2 章第 1 節說明安全軟體測試所應注意的事項。

第 2 章第 2 節介紹「安全軟體測試相關國際標準」，如 ISO/IEC 27001:2013、IEEE 829: 2008、ISO/IEC/IEEE 29119:2013，以及 OWASP 測試指引等，作為本指引參考的文獻。

第 2 章第 3 節介紹「常見的 Web 應用程式弱點」，針對國際資安組織所分類的弱點及威脅進行介紹，如 OWASP Top 10、WASC 威脅分類、CWE/SANS Top 25。

第 2 章第 4 節「結語」，總結第 2 章內容。

第 3 章「安全需求驗證」介紹如何針對安全需求進行驗證。

第 3 章第 1 節介紹安全需求的原則、類型、正面需求與負面需求，最後介紹安全需求的萃取方式。

第 3 章第 2 節說明安全需求驗證實務活動的要項，包含 OWASP 應用程式安全驗證標準以及安全軟體發展生命週期內的安全需求驗證。

第 3 章第 3 節介紹網頁自動化測試可以利用的工具，以避免使用人工進行重複性的測試，提高測試效率。

第 3 章第 4 節「結語」，總結第 3 章內容。

第 4 章針對安全性測試技術進行介紹，包含靜態源碼分析、滲透測試，以及灰箱安全性測試。

第 4 章第 1 節「源碼靜態分析」，說明源碼安全審查及自動化工具檢測的進行要點。

本文件之智慧財產權屬行政院資通安全辦公室所有。

第 4 章第 2 節「滲透測試」，說明進行滲透測試的流程以及可利用的工具。

第 4 章第 3 節「灰箱安全性測試」介紹灰箱測試的概念。

第 4 章第 4 節「結語」，總結第 4 章內容。

第 5 章「案例分析」，以實際系統為範例進行安全測試，測試活動包含安全需求驗證、源碼安全性檢測，以及弱點掃描。

第 6 章「結論」，總結各章節內容。

第 7 章「參考文獻」，詳列本指引參考的文件資料。

本文件之智慧財產權屬行政院資通安全辦公室所有。

2. 安全軟體測試介紹

軟體測試之目標即為提升軟體品質，因此常參考到軟體品質確保(Quality Assurance, QA)的觀念。軟體測試包含功能性、使用者介面、效能、安全、設定與安裝等，其中安全測試著重於找出軟體中安全性相關問題。

安全軟體測試階段是整體安全軟體發展流程中，不可或缺的階段。本章節針對軟體之安全測試進行介紹，包含測試注意事項以及相關之測試標準。

2.1.安全軟體測試

軟體安全性一般都被列為事後考慮的情況，通常當軟體發生安全事故，或者當功能需求在專案資源範圍內被滿足時，軟體安全性才會被列入考量，使得軟體安全品質的問題愈加嚴重。軟體安全性測試是確保軟體安全品質的重要手段，其目的在於識別應用程式潛在的安全性問題，以及驗證安全控制手段如預期發揮作用，並期望能在發布前找到安全問題，予以修補以降低成本。安全測試包含兩個面向，一個是「安全需求驗證」，另一個是針對「安全性測試」，因此需區分兩者的不同：

「安全需求驗證」是測試軟體中安全相關功能(例如，身分驗證機制、存取控制機制)是否符合需求並運作正確。

「安全性測試」則是以攻擊者的角度去確認軟體承受攻擊的能力，著重在找出軟體的安全錯誤(Bug)與缺陷(Flaw)。

本小節介紹進行軟體安全測試時應該加以注意的事項，並於第 3 章介紹安全需求驗證所進行的活動，安全性測試相關技術則於第 4 章進行介紹。

2.1.1. 安全軟體開發生命週期

傳統軟體發展流程為功能性導向，期望在最短的時間，完成系統的開發與上線，缺乏安全性考量的設計，依賴外部的安全機制，例如防火牆、入侵

本文件之智慧財產權屬行政院資通安全辦公室所有。

偵測系統等，然而這些機制並不能解決所有資安問題，尤其是系統設計邏輯層次的安全瑕疵及缺點。安全軟體發展流程(Secure Software Development Life Cycle, SSDLC)則考量系統功能性的同時，將安全的概念導入到軟體開發生命週期中的各個階段(例如：研究規劃、需求分析、系統設計、系統開發、系統測試、系統部署、系統維護)，進行各項必要的安全活動，可以對應用安全採用綜合方法以確保安全計畫全面有效。雖拉長了各階段的時程，卻降低了系統後續維護的成本，以及遭受到攻擊損失的可能性。

2.1.2. 修補成本

根據 National Institute of Standards and Technology(NIST)提出的研究報告[1]顯示，在軟體發展的過程中，若能愈早發現軟體的安全弱點及瑕疵，就愈能減少因修補所帶來的負面影響與付出的額外成本。若於產品釋出之後才發現錯誤，為進行修補所付出的人力及成本與在設計階段相比可相差達 30 倍之多，研究統計詳見圖 1。

Table 5-2. Preliminary Estimates of Relative Cost Factors of Correcting Errors as a Function of Where Errors Are Introduced and Found (Example Only)

	Where Errors are Found				
Where Errors are Introduced	**Requirements Gathering and Analysis/ Architectural Design**	**Coding/ Unit Test**	**Integration and Component/ RAISE System Test**	**Early Customer Feedback/Beta Test Programs**	**Post-product Release**
Requirements Gathering and Analysis/ Architectural Design	1.0	5.0	10.0	15.0	30.0
Coding/Unit Test		1.0	10.0	20.0	30.0
Integration and Component/ RAISE System Test			1.0	10.0	20.0

資料來源：[1]

圖1　缺陷修復成本

本文件之智慧財產權屬行政院資通安全辦公室所有。

及早發現安全弱點雖可達到減少修補成本之功效，但綜觀不同的安全檢測技術，仍有其成本考量與應用之適當時機，根本解決之道則是在系統發展生命週期的每一階段中都加進安全的概念，推動安全軟體發展生命週期才是根本的解決之道。

由於新的資安威脅頻繁出現，開發者必須體認新的威脅，對所開發的軟體帶來的影響。實現這個目標關鍵的一步即在於對開發部門教育常見的安全問題、安全的程式設計方法，以及如何測試與防範的方法，經由教育訓練可幫助開發者重視安全問題，並從攻擊者的滲透行為中獲取適當的程式設計思路，提升軟體的安全性。

2.1.3. 安全測試限制

安全測試在於依據威脅分析做出假設，驗證針對假設所採取的安全反制措施是否有效，通過安全測試並不代表應用程式全無安全問題。例如進行「安全性測試」時，一般常見的方式為使用安全檢測工具，這些工具擅長於在不同的標的中發現普通的已知弱點，其往往只能依據目前已知規則(pattern)進行比對，但未知的安全問題就未被檢測到，故即使檢測結果顯示無任何安全弱點亦不代表就真的安全無虞。一些研究[2]顯示，工具可能只發現整體漏洞中的 45%。僅依靠從自動化工具中獲得的測試結果，是有其風險的，工具檢測的結果有時不如一位經驗老道的安全測試者。一般來說，由更有經驗的安全測試者並搭配適當的測試工具，所獲得的安全分析和測試的結果可以更準確。

2.1.4. 安全測試執行

於進行安全測試前取得相關利害關係者的授權是相當重要的，必要時須取得書面授權，例如進行滲透測試時，由於實際對目標進行安全探測，與入侵行為相似，因此可能會面臨法律問題。為避免觸犯刑法第三十六章妨害

本文件之智慧財產權屬行政院資通安全辦公室所有。

電腦使用罪，此活動需簽署滲透測試同意書(Permission Memo)，取得相關人士的授權，並明確定義測試範圍後才可進行。

2.1.5. 制定測試計畫

測試計畫為測試活動進行之依據，進行安全測試亦應擬定測試計畫以說明測試活動之目的、範圍、時程、人員，以及測試進行方式。進行測試所使用的測試案例(Test Case)則可描述於測試設計規格書內，但亦可考量實務上的需求，只產生一份測試計畫而涵蓋所有測試設計階段之內容。

ISO/IEC/IEEE 29119 第三部分測試文件提供了相關文件範本，彙整出測試計畫與測試設計規格書的章節架構，其可作為計畫安全測試專案或任何類型的測試專案所使用，詳見 2.2.3.3 小節。

2.1.6. 文件化測試報告

將測試結果使用文件化方式記錄是非常重要的，於軟體測試階段結束後應產生一份測試報告，其中彙整與統計所有之測試結果以確認所有軟體測試項目均已完成測試，報告內容應包含人員、時間、測試方法，以及詳細的測試結果等。該報告應能清楚地指出所發現的安全弱點及其嚴重性，以供開發人員進行修補。而由於此份報告內容具有敏感的資訊，故應注意報告的機密層級，避免非授權的人士取得利用。

2.2.安全軟體測試相關標準介紹

本節介紹與安全軟體測試相關的標準與指引。

ISO/IEC 27001:2013 是針對資訊安全管理的國際性規範，為大多數政府單位及企業所採用。IEEE 829:2008 是一套針對軟體所需測試文件的國際標準，不限於安全性測試，而是針對所有類型的軟體測試過程所產出的文件提供格式範本與參考的範例。ISO/IEC/IEEE 29119:2013 則是新的國際性標準，涵蓋了廣泛的軟體測試範疇，可適用於任何組織執行任何類型的軟體測試，

本文件之智慧財產權屬行政院資通安全辦公室所有。

內容涵蓋測試概念與定義、測試過程、測試文件、測試技術以及使用關鍵字驅動(Keyword Driven)的測試。OWASP Testing Guide[11]為 OWASP (The Open Web Application Security Project)組織提出的軟體安全測試指引，於 2014 年 9 月推出第 4 版本，其提供了一個測試框架(Framework)以幫助進行安全性測試，並介紹針對 Web 應用程式進行安全性測試可用的技術與方法。

2.2.1. ISO/IEC 27001:2013

資訊安全管理系統之定義為，全面管理系統的一部分，依據企業風險導向，建立、實作、運作、監視、審查、維持及改進資訊安全。ISO/IEC 27001:2013 是目前國際上最廣泛採用之資訊安全管理規範[3]。國際標準組織(ISO)於 2013 年公布最新版 ISO/IEC 27001:2013，增加了 A.14 有關系統發展的相關控制措施，名稱為「系統獲取、開發及維護」。與測試相關之章節包含 A.14.2「開發與支援過程的安全」及 A.14.3「測試資料」，以下針對此其內容進行介紹：

●A.14.2 「開發與支援過程的安全」

其目的在確保資訊安全是經過設計的並實施於資訊系統的開發生命週期內。其中與系統測試相關的部分包含以下兩點：

– A.14.2.8「系統安全測試」：於開發過程中應實施安全功能性之測試。

– A.14.2.9「系統驗收測試」：應建立新資訊系統、系統升級及新版本之驗收測試計畫及準則。

●A.14.3「測試資料」

其目的在確保測試用資料之保護。

– A.14.3.1「測試資料之保護」：應小心選擇、保護及控制測試資料。

本文件之智慧財產權屬行政院資通安全辦公室所有。

2.2.2. IEEE 829:2008

IEEE 829:2008 是一套軟體測試文件的標準[4]，發表在 2008 年 7 月 18 日並已經被核准取代 1998 版本，一般可適用於軟體開發的任何測試階段。此標準將文件化過程定義為 8 個階段，每個階段可能產生它自己單獨的文件。這個標準定義了文件的格式但是沒有規定它們是否必須全部被應用，也不包括這些檔中任何相關的其他標準的內容。

軟體測試文件化過程產生的文件如下所列：

- 主要測試計畫(Master Test Plan, MTP)

  此文件的目的在於為多層級的測試(可能在同一個測試專案內或是跨多個專案)提供總體的測試計畫及測試管理文件。

- 層級測試計畫(Level Test Plan, LTP)

  此文件對特定層級的測試活動描述其範圍、方法、資源和時程表。必須識別受測項目、欲測試的功能與欲執行的測試任務，負責每項任務人員以及相關的風險。

- 層級測試設計(Level Test Design, LTD)

  此文件詳細描述測試條件、期望的結果以及測試通過的判定標準。

- 層級測試案例(Level Test Case, LTC)

  此文件用來指定為執行測試案例所需的相關測試資料。

- 層級測試程式(Level Test Procedure, LTPr)

  此文件詳細描述測試人員實際將如何進行每項測試，包含任何設置的先前條件以及必須遵循的步驟。

- 層級測試記錄(Level Test Log, LTL)

本文件之智慧財產權屬行政院資通安全辦公室所有。

此文件記錄測試的運行細節，包含運行了哪幾個測試案例、運行的人員與運行的順序，以及每個測試項目是否通過。

- 異常報告(Anomaly Report, AR)

  此文件記錄測試過程中所發生的任何值得調查的事件。這可能被稱為問題、測試事件、缺陷、麻煩、問題、異常或錯誤報告。這份文件之所以被命名為異常報告而不是錯誤報告，其原因是預期結果和實際結果之間可能由於一些原因存在差異，而這並不能認為是系統存在錯誤。這包括預期結果有誤、測試被錯誤地執行，或者對需求的理解存在差異。這個報告由以下所有附加的細節組成，例如實際結果和預期結果、何時失敗，以及其他有助於解決問題的證據。這個報告還可能包括此附加項對測試所造成的影響的評估。

- 層級中途測試狀態報告(Level Interim Test Status Report, LITSR)

  此文件摘要說明指定的測試活動的中途結果及選擇性提供基於特定測試層級的評估和建議。

- 層級測試報告(Level Test Report, LTR)

  此文件摘要說明指定的測試活動的結果及根據為特定層級之測試執行完畢後的結果提供的評估和建議。

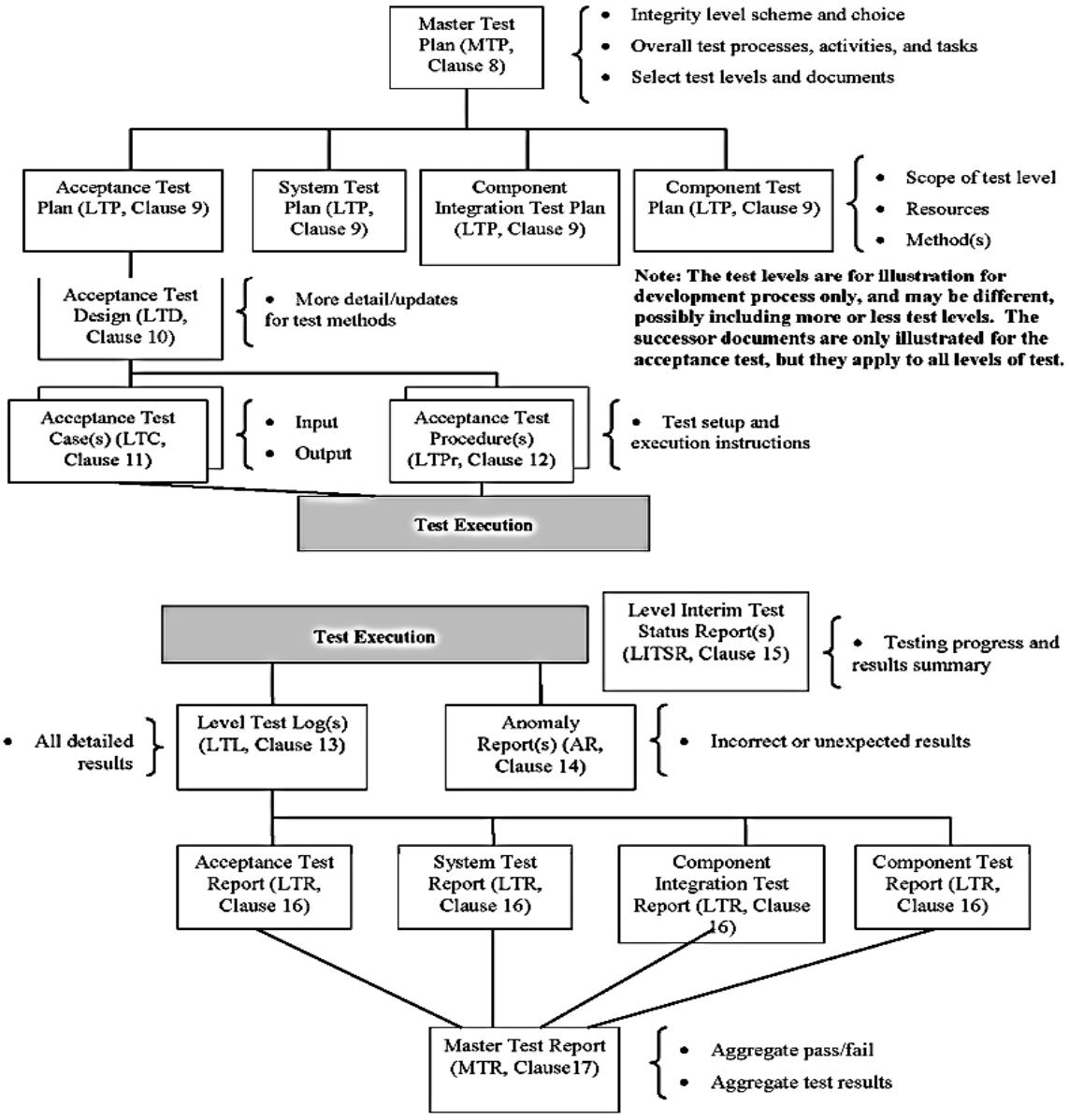
- 主要測試報告(Master Test Report, MTR)

  此文件摘要說明指定的測試活動的結果，並基於這些結果提供評價。這份報告可能被有使用中途測試計畫的任何組織所使用，內容主要為揭露測試完成的狀態並評估測試工作及受測軟體的品質，以及從異常報告得出的統計資料。該報告還記錄已經完成的測試和所花費的時間，以改進未來的測試計畫。這份最終的報告用於指出被測的軟體系統是否與專案管理者所提

本文件之智慧財產權屬行政院資通安全辦公室所有。

出的可接受標準符合。

IEEE 829:2008 所提出的測試文件架構圖，詳見圖 2。



資料來源：[4]

圖2　IEEE 829 測試文件架構圖

2.2.3. ISO/IEC/IEEE 29119:2013

本文件之智慧財產權屬行政院資通安全辦公室所有。

現行國際上較為普遍之測試管理標準及最佳實務為 ISO/IEC/IEEE 29119 軟體測試標準的系列[5]，可適用於任何組織執行任何類型的軟體測試。已於 2013 年 9 月正式發行 3 個子部分，分別為概念與定義、測試過程，以及測試文件；第四部分測試技術目前尚為國際標準草案版(Draft International Standard)，第五部分也仍處於撰寫草稿的階段(Committee Draft)，以下章節分別針對各個部分進行介紹。

2.2.3.1. 第一部分：概念與定義

ISO/IEC/IEEE 29119 第一部分主要是介紹在 ISO/IEC/IEEE 29119 整個系列標準內所引入的概念和詞彙，以及提供其在實踐上的應用實例，此部分為整個系列的標準提供了一個起始點，可作為其他部分的指引。

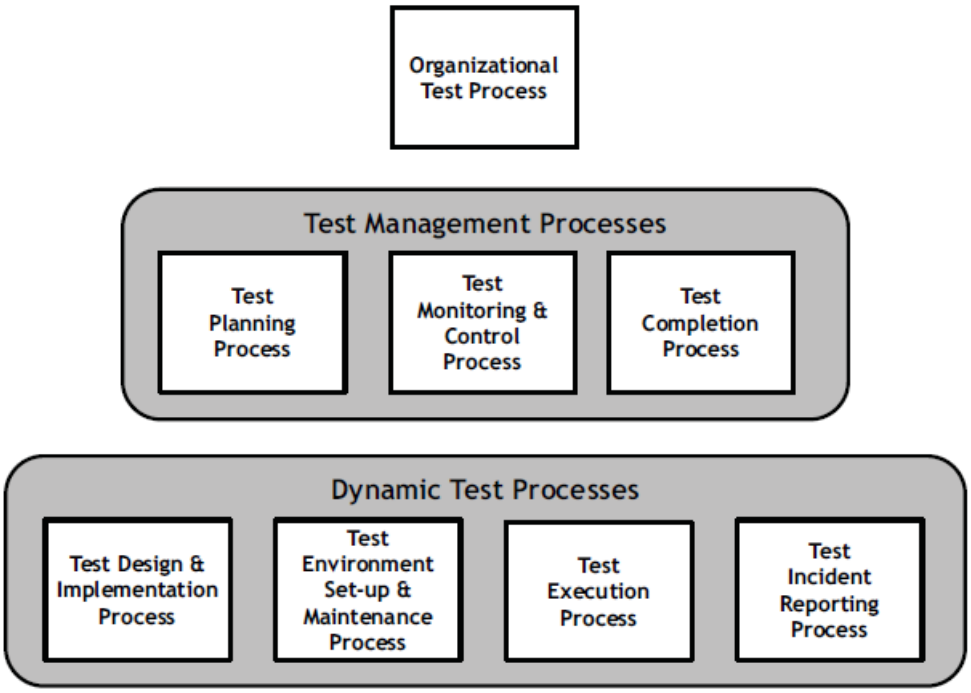
此部分一開始描述通用的軟體測試概念以及軟體測試在組織、專案以及軟體生命週期所扮演的角色，並說明軟體測試如何適用於不同的軟體生命週期模型。並展示了測試計畫不同的實踐做法，以及如何利用自動化來支援測試工作的進行。

2.2.3.2. 第二部分：測試過程

測試是軟體發展中緩和風險的關鍵方法，ISO/IEC/IEEE 29119-2:2013 遵循以風險為基礎的測試方法，此方法的精神為依據風險等級指定測試的優先權，並且將著重於較重要的功能及特性。

ISO/IEC/IEEE 29119 第二部分提出一個測試模型，支持動態測試、功能性及非功能性測試、手動及自動測試，以及腳本化和非腳本化測試，並可適用於任何軟體發展生命週期模型。此測試模型將測試過程定義為三種層級的過程[6]，分別為組織性測試過程、測試管理過程，以及動態測試過程，詳見圖 3。

本文件之智慧財產權屬行政院資通安全辦公室所有。



資料來源：[6]

圖3　ISO/IEC/IEEE 29119 測試過程

以下小節針對組織性測試過程、測試管理過程，以及動態測試過程分別進行說明。

2.2.3.2.1. 組織性測試過程

此過程的目的係為建立和維護組織化測試規範而定義的過程，組織化測試規範可能包含組織測試政策、組織測試策略、測試過程、測試程序和其他測試資產。組織測試政策係描述組織的宗旨、目標、原則和測試範圍的文件，組織測試策略則說明組織中所有測試專案執行的一般要求，提供測試如何進行細節的文件，其精神應要與組織測試政策一致。經由發展最上層的測試組織測試政策及策略，才能決定測試的目標，了解組織對於測試的態度，進而訂出相關測試需求。例如以安全測試的角度為例，若組織的資訊安全政策是為遵循 ISO/IEC 27001:2013 的管理要求，則進行軟體開發測

本文件之智慧財產權屬行政院資通安全辦公室所有。

試時所訂出的安全需求就必須符合其標準的相關規定。

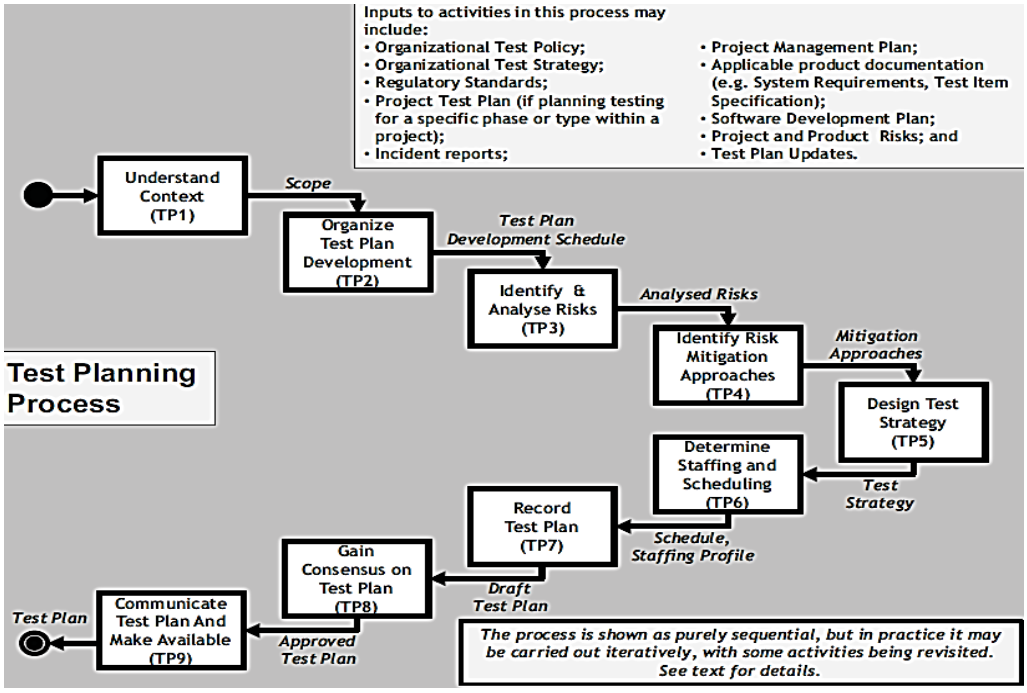
2.2.3.2.2. 測試管理過程(Test Management Process)

為測試專案中(例如專案測試管理、系統測試管理、效能測試管理)，對整個測試專案或任何測試階段(例如系統測試)和測試類型(例如效能測試、安全測試)的測試管理定義其過程。通過實行此過程可以產出測試計畫。

測試管理過程包含 3 個子過程，分別為測試計畫過程(Test Planning Process)、測試監測和控制過程(Test Monitoring and Control Process)，以及測試完成過程(Test Completion Process)，以下分別介紹此 3 個子過程：

●測試計畫過程(Test Planning Process)

此過程的目的在發展、同意、記錄及向利害關係者溝通測試範圍及方法，以早期識別測試所需的資源、環境及其他要求。測試計畫的過程詳見圖 4。



資料來源：[6]

圖4　測試計畫的過程

本文件之智慧財產權屬行政院資通安全辦公室所有。

此過程包含 9 個實行的活動，有些活動可能會被反覆實行。

– TP1：了解內容

了解軟體測試要求及內容以準備測試計畫，識別利害關係者並與其互動，以及發起溝通計畫及記錄溝通管道。

– TP2：組織測試計畫發展

識別及安排所有為了完成測試計畫所需實行的活動及相關的利害關係者，並取得其核可、安排及參與。

– TP3：識別及分析風險

識別與軟體測試相關或可經由軟體測試處理的風險，及審查已識別過的風險，並使用適當的方案進行風險分類以區別專案及產品的風險，藉由考量其影響及可能性指定風險暴露層級，最後記錄風險評鑑的結果。

– TP4：識別風險轉移方法

依據風險類型及暴露層級識別風險處理的方法，並記錄風險緩和的對策

– TP5：設計測試策略

初步評估所需的資源(如工作量及耗費時間)並考量風險及限制，以設計測試策略，其中包含識別測試監控的度量、測試資料、測試環境及工具的要求。

– TP6：決定人員及工作排程

識別人員角色及技能，並根據評估結果、相依性及人員可用性來安排測試活動。

– TP7：記錄測試計畫

本文件之智慧財產權屬行政院資通安全辦公室所有。

整合測試策略、人員、工作排程及最終評估，撰寫測試計畫。

– TP8：取得測試計畫的共識

蒐集利害關係者的觀點，並解決其與測試計畫間的衝突，然後更新測試計畫。

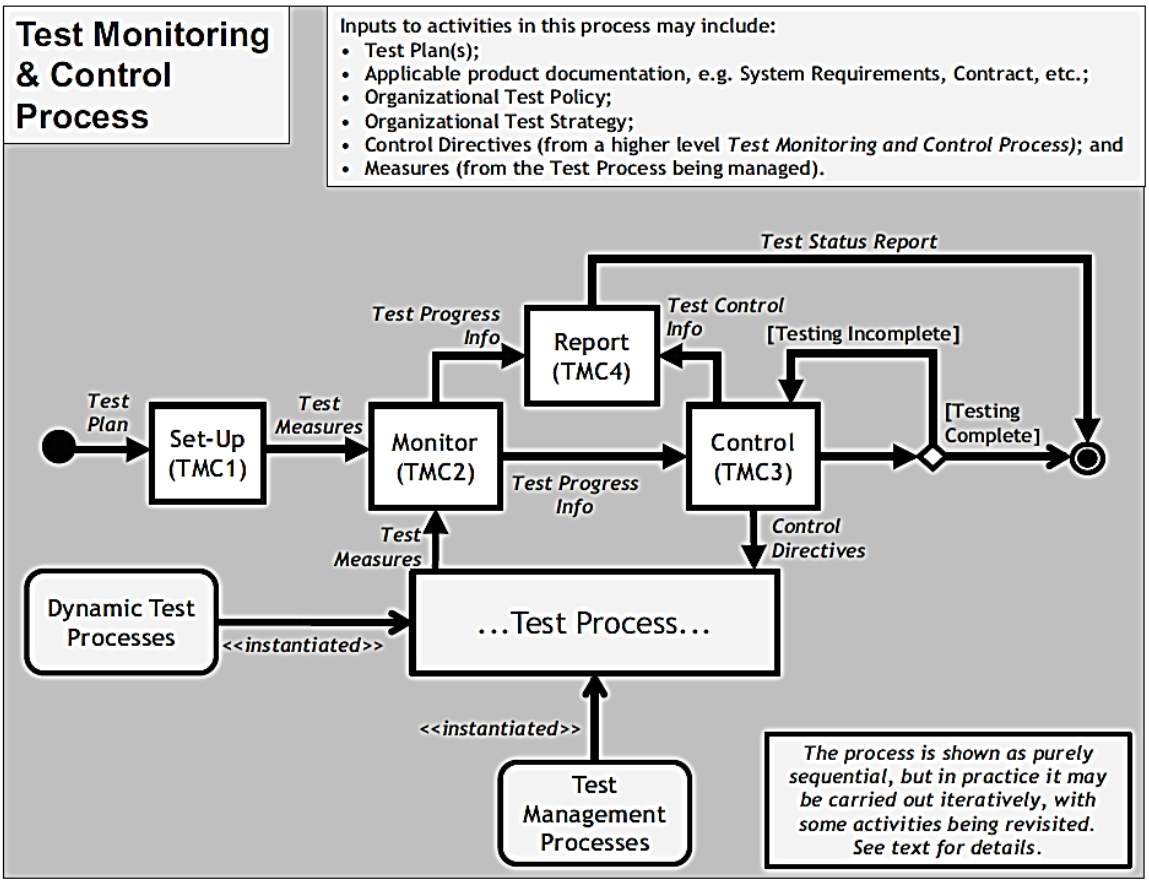
– TP9：溝通測試計畫及讓其可取用

讓測試計畫可被相關利害關係者所取用。

●測試監測和控制過程(Test Monitoring and Control Process)

此過程的目的在判斷測試過程是否符合測試計畫、測試政策與測試策略，並起始必要的控制行動以及識別必要的測試計畫更新，以進行測試管理。透過此過程可以得到測試狀態的資訊，並可利用此資訊更新測試計畫或取得進一步的控制指示(Control Directives)，例如測試環境或測試人員的變更。測試監測和控制的過程詳見圖 5。

本文件之智慧財產權屬行政院資通安全辦公室所有。



資料來源：[6]

圖5　測試監測和控制的過程

此過程包含 4 個實行的活動：

– TMC1：建立(Set-Up)

識別合適的量測方法來監視測試計畫進度。

– TMC2：監視(Monitor)

蒐集及記錄測試量測。

– TMC3：控制(Control)

實行測試計畫或由高階管理過程而來的控制指示所需之活動，以及當測試完成，標準達到時進行審核。

本文件之智慧財產權屬行政院資通安全辦公室所有。

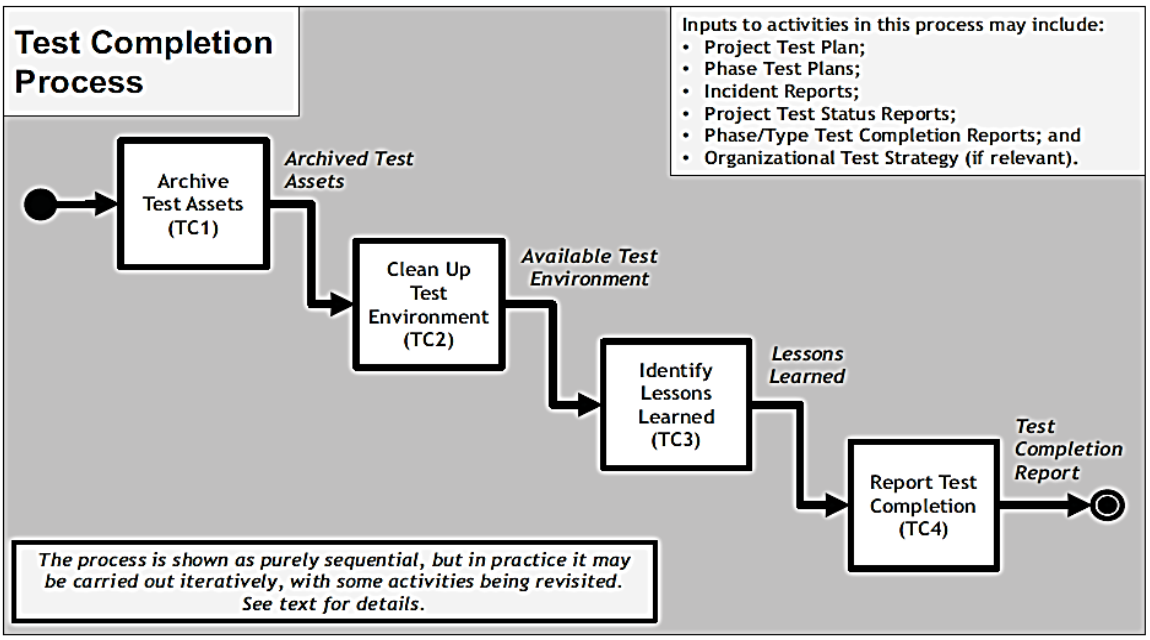
– TMC4：報告(Report)

於特定期間報告測試進度。

●測試完成過程(Test Completion Process)

此過程的目的在為測試任務進行收尾的工作，包含讓測試資產可被將來所使用、讓測試環境置於妥善的狀態，以及記錄測試結果並呈現給利害關係者。透過實行此過程可以產生測試完成報告。

測試完成的過程詳見圖 6。



資料來源：[6]

圖6　測試完成的過程

此過程包含 4 個實行的活動：

– TC1：將測試資產歸檔

識別及歸檔可能被此專案或其他專案所利用的測試資產。測試資產包含測試計畫、測試案例說明書、測試腳本、測試工具、測試資料，以及測

本文件之智慧財產權屬行政院資通安全辦公室所有。

試環境基礎建設等。

– TC2：重置測試環境

於所有測試活動結束後，復原測試環境至預先定義的狀態，例如重置資料庫、復原測試工具設定等。

– TC3：識別學習之經驗

將此次測試專案所學習到的經驗及知識記錄下來。

– TC4：報告測試完成

從測試計畫、測試結果、測試狀態報告、測試完成報告、事故報告等來源蒐集相關資訊以撰寫測試報告，並經過利害關係者的核可後進行分發。

2.2.3.2.3. 動態測試過程

定義用於執行動態測試的通用過程。動態測試可以在測試的特定階段進行(例如單元，整合，系統和驗收)，或是測試項目中的特定類型的測試(例如效能測試，安全測試和功能測試)。

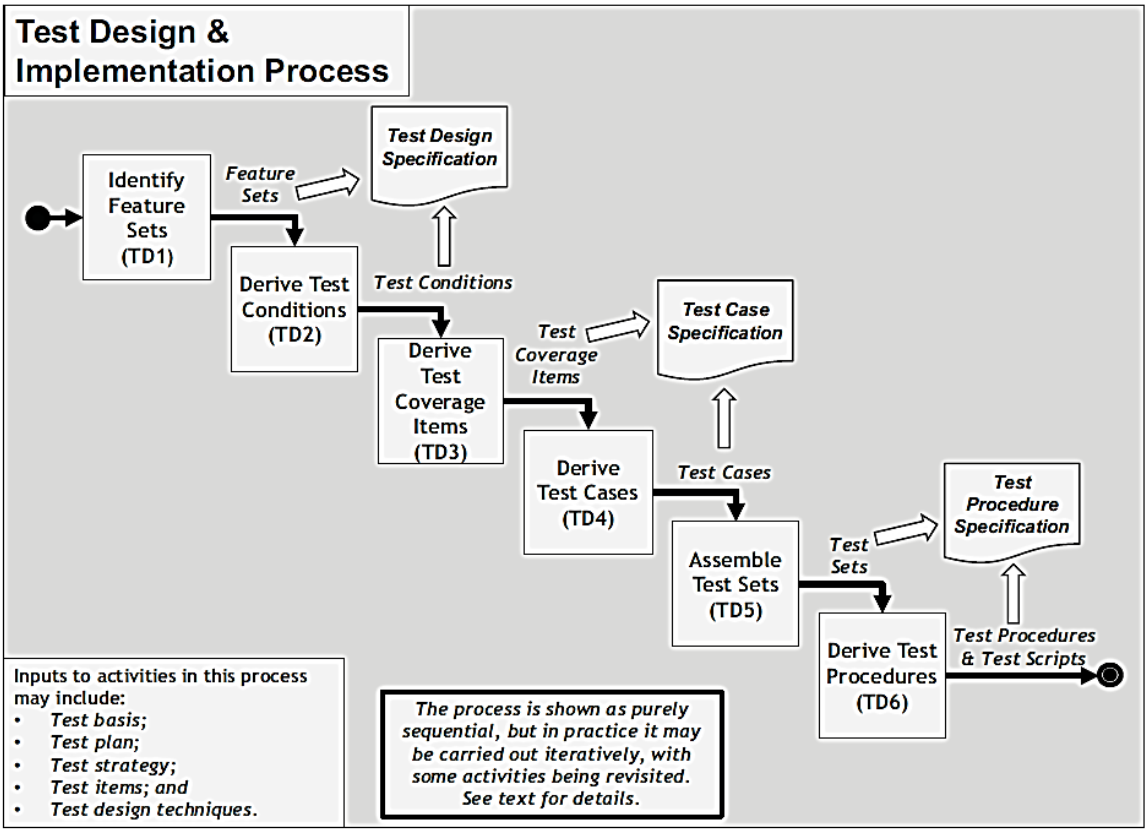
動態測試過程包含以下 4 個子過程：

●測試設計與實現過程(Test Design and Implementation Process)

此過程之目的在於導出於測試執行過程中將會執行的測試程序。通過實行此過程可以產出測試計畫(Test Plan)及測試設計規格書(Test Design Specification)。

動態測試過程詳見圖 7。

本文件之智慧財產權屬行政院資通安全辦公室所有。



資料來源：[6]

圖7　測試設計與實現過程

此過程包含 6 個實行的活動：

– TD1：識別特性集合(Feature Sets)

分析測試基礎(Test Basis)以了解測試項目的要求，欲測試的特性可分類成數個特性集合，並依據風險等級配置其測試優先權。

– TD2：導出測試條件(Test Condition)

決定每個特性的測試條件並配置其優先權。

– TD3：導出測試涵蓋項目

利用測試設計的技術來導出測試項目，以達到測試計畫中所指定的測試完成涵蓋條件。

本文件之智慧財產權屬行政院資通安全辦公室所有。

– TD4：導出測試案例

藉由決定預先條件、選擇輸入值、決定預期結果等方法設計出測試案例。

– TD5：組合測試集

將測試案例依特性進行分類，組合成多個測試集。

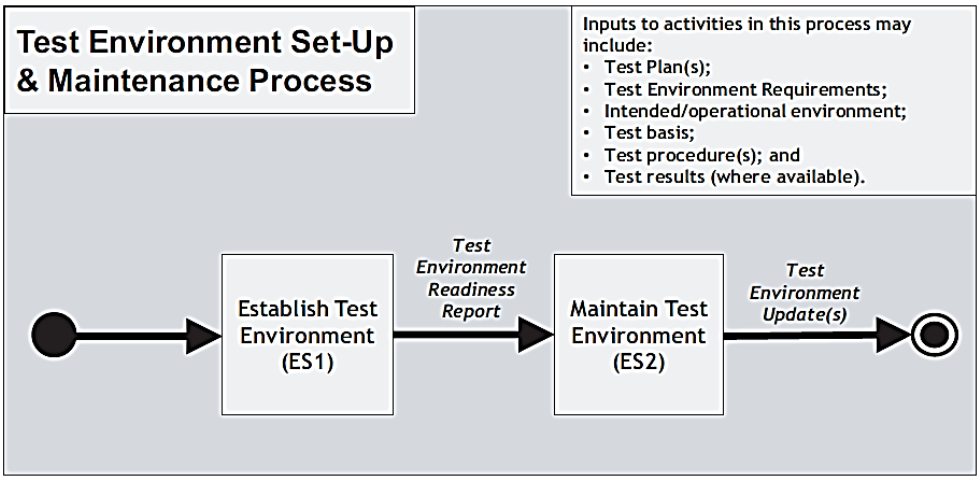
– TD6：導出測試程序

訂定測試集中測試案例的順序，並導出測試程序。

●測試環境設置和維護過程(Test Environment Set-Up and Maintenance Process)

此過程之目的在於建立及維護測試環境，並向利害關係者溝通其狀態。

動態測試過程詳見圖 8。



資料來源：[6]

圖8　測試環境建立與維護過程

此過程包含 2 個實行的活動：

– ES1：建立測試環境

本文件之智慧財產權屬行政院資通安全辦公室所有。

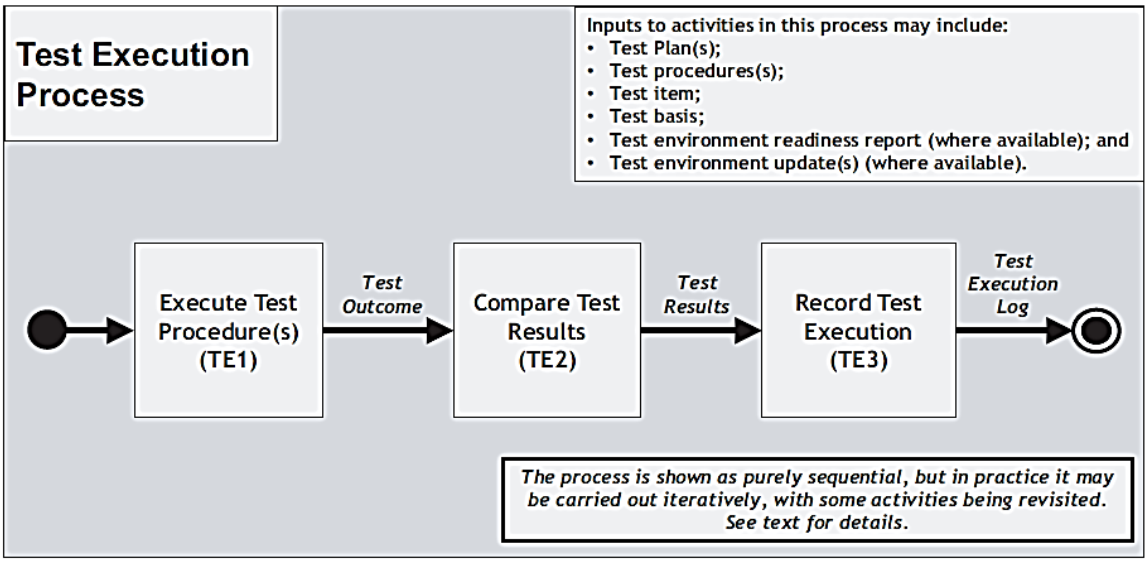
依據測試計畫、細節需求或測試工具需求等建立測試所需之環境，包含安裝測試工具、建立測試資料、決定組態管理程度等，並記錄測試環境及測試資料的準備狀況。

– ES2：維護測試環境

於測試過程中維護測試環境，並回報測試環境的變更。

●測試執行過程(Test Execution Process)

此過程之目的是於測試環境執行測試程序並記錄結果。通過實行此過程可以產出最終的測試結果。測試執行過程詳見圖 9。



資料來源：[6]

圖9　測試執行過程

此過程包含 3 個實行的活動：

– TE1：執行測試程序

於準備的測試環境執行測試程序，觀察及記錄每個測試案例的實際結果。

本文件之智慧財產權屬行政院資通安全辦公室所有。

– TE2：比對測試結果

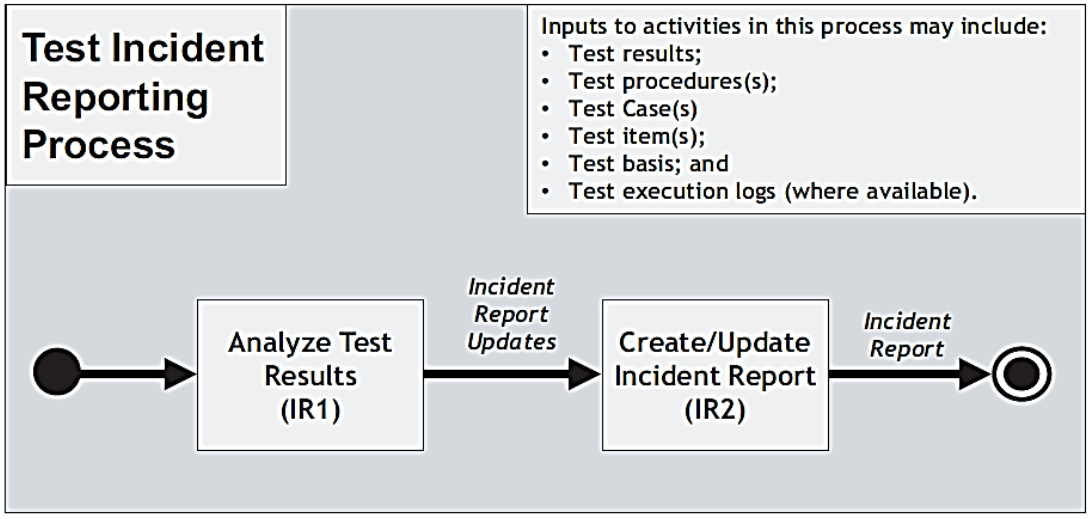
比對每個測試案例的預期結果(Expected Result)及實際結果(Actual Result)以決定測試結果(Test Result)。

– TE3：記錄測試執行

記錄測試執行的過程，例如測試日誌(Test Log)。

●測試事故報告過程(Test Incident Reporting Process)

此過程之目的是向利害關係者報告在測試進行的過程中所發生的任何重要事故，通過實行此過程可以產出事故報告。測試事故報告過程詳見圖10。



資料來源：[6]

圖10 測試事故報告過程

此過程包含2個實行的活動：

– IR1：分析測試結果

測試結果與先前事故相關，則分析測試結果且更新事故細節。而若測試結果導出新的議題，則分析且決定其是否為需要報告之事故、或是一個

本文件之智慧財產權屬行政院資通安全辦公室所有。

將被解決的活動項目，而不必報告，或是不須任何進一步行動。

– IR2：創建及更新事故報告

識別、報告及更新所有必須被記錄的事故資訊。

2.2.3.3. 第三部分：測試文件

ISO/IEC/IEEE 29119 第三部分[7]包括測試文件的範本和範例。範本可對應至 ISO/IEC/IEEE 29119 第二部分所描述的測試過程，明確地指明在哪一個子過程會產出哪一份文件。

產出的文件可分為 16 份，包含：

●組織測試政策(Test Ploicy)。

●組織測試策略(Organizational Test Strategy)。

●測試計畫(Test Plan)。

●測試狀態報告(Test Status Report)。

●測試完成報告(Test Completion Report)。

●測試設計規格(Test Design Specification)。

●測試案例規格(Test Case Specification)。

●測試程序規格(Test Procedure Specification)。

●測試資料需求(Test Data Requirements)。

●測試環境需求(Test Environment Requirements)。

●測試資料準備度報告(Test Data Readiness Report)。

●測試環境準備度報告(Test Environment Readiness Report)。

本文件之智慧財產權屬行政院資通安全辦公室所有。

●實際結果(Actual Result)。

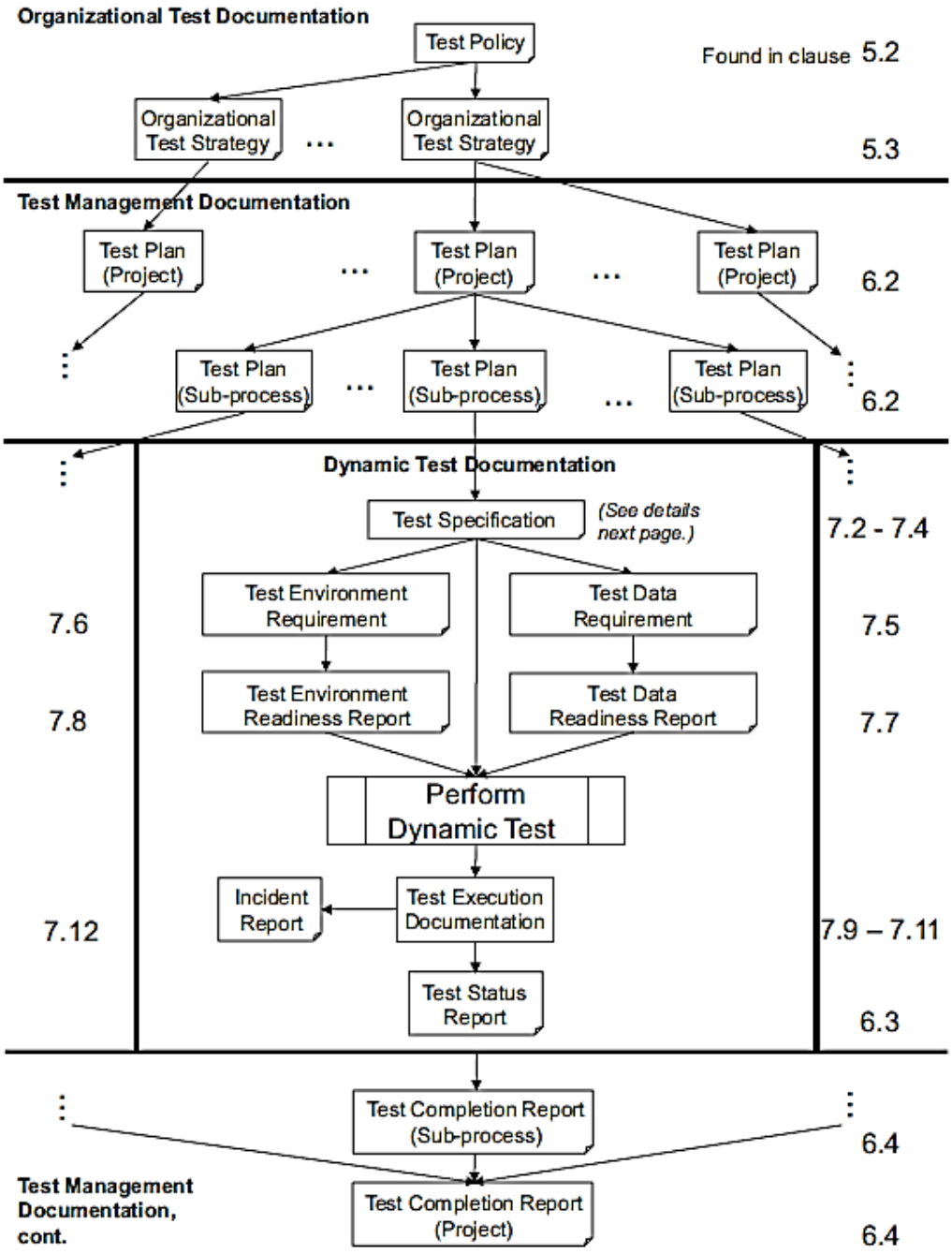
●測試結果(Test Result)。

●測試執行日誌(Test Execution Log)。

●測試事故報告(Test Incident Reporting)。

ISO/IEC/IEEE 29119 提供之測試文件架構圖，詳見圖 11。

本文件之智慧財產權屬行政院資通安全辦公室所有。



資料來源：[7]

圖11 測試文件架構圖

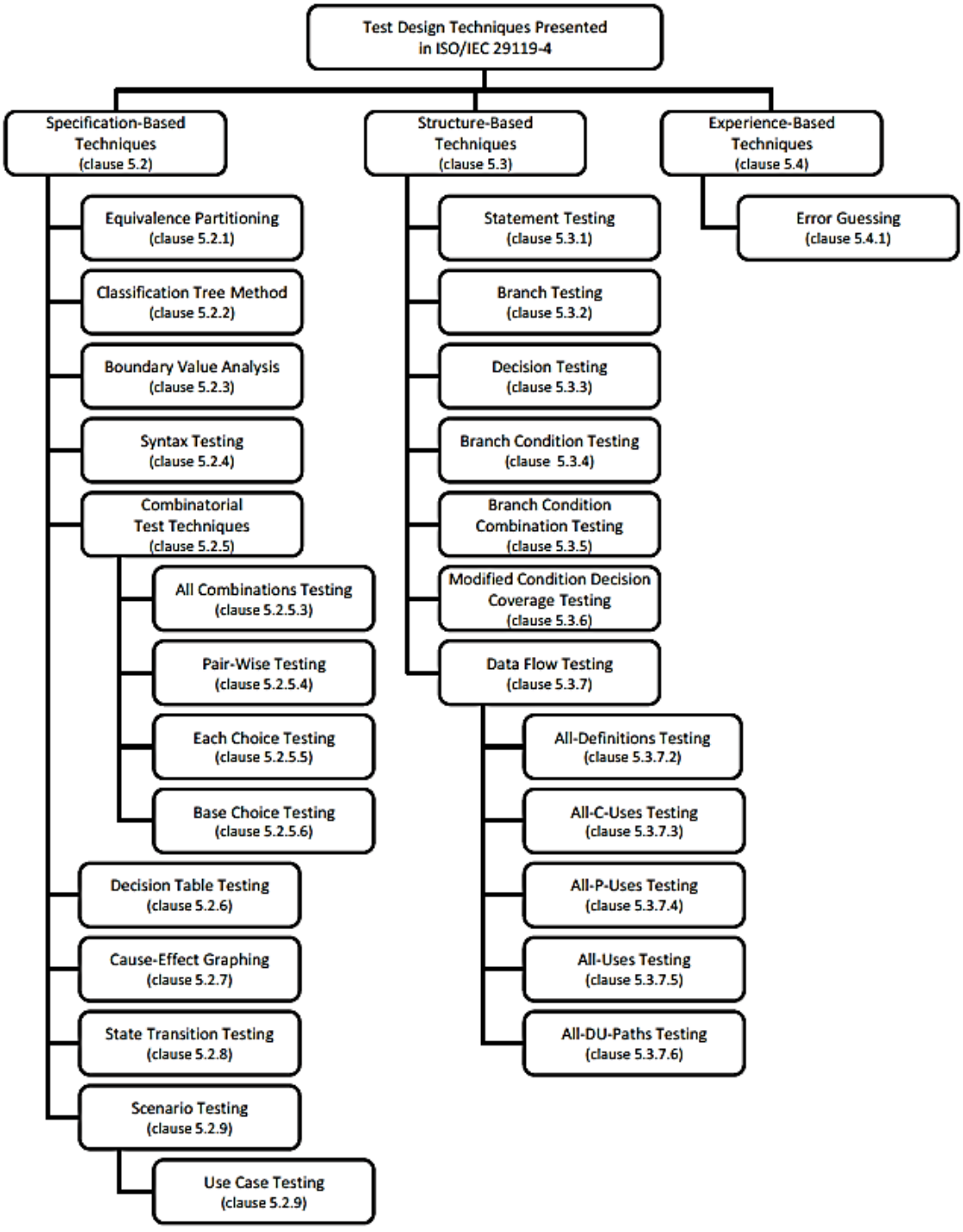
本文件之智慧財產權屬行政院資通安全辦公室所有。

2.2.3.4. 第四部分：測試技術

ISO/IEC/IEEE 29119 第四部分[8]描述了多種測試技術(也稱為測試案例設計技術或試驗方法),以及其測試涵蓋率的度量方法,可適用於在ISO/IEC/IEEE 29119 第二部分所描述的測試過程中。此部分的目的是描述一系列已在軟體測試行業中被廣泛接受的技術，這些技術可用來推導出測試案例以決定測試項目是否以滿足所提出的相關需求。

此部分涵蓋的測試技術主要分為 3 類，分別為以規格為基礎的技術(Specification-Based Techniques)、以結構為基礎的技術(Structure-Based Techniques)，以及以經驗為基礎的技術(Experience-Based Techniques)，詳見圖 12。

本文件之智慧財產權屬行政院資通安全辦公室所有。



資料來源：[8]

圖12 ISO/IEC/IEEE 29119 測試技術

2.2.3.5. 第五部分：關鍵字驅動的測試

本文件之智慧財產權屬行政院資通安全辦公室所有。

ISO/IEC/IEEE 29119 第五部分[9]旨在定義支援關鍵字驅動測試的國際標準。關鍵字驅動測試是通過使用一組預先定義的關鍵字來描述測試案例，這些關鍵字是與一組測試案例中的操作相關聯的名稱，通過使用關鍵字來描述測試步驟而不是使用自然語言，測試案例可以更容易被理解、維護以及自動化。特別注意的是，ISO/IEC/IEEE 29119 第五部分仍處於草案階段，內容仍可能變動。

2.2.4. OWASP 測試指引

OWASP(The Open Web Application Security Project)是一個開放社群、非營利性組織[10]，長期致力於協助政府或企業了解並改善網頁應用程式與網頁服務的安全性。OWASP 測試指引[11]針對日益受到重視的 Web 安全性提供了一個測試框架，適用於軟體發展生命週期(SDLC)的各個階段。並且針對 Web 應用程式之弱點以及其測試的方法進行詳細的介紹。

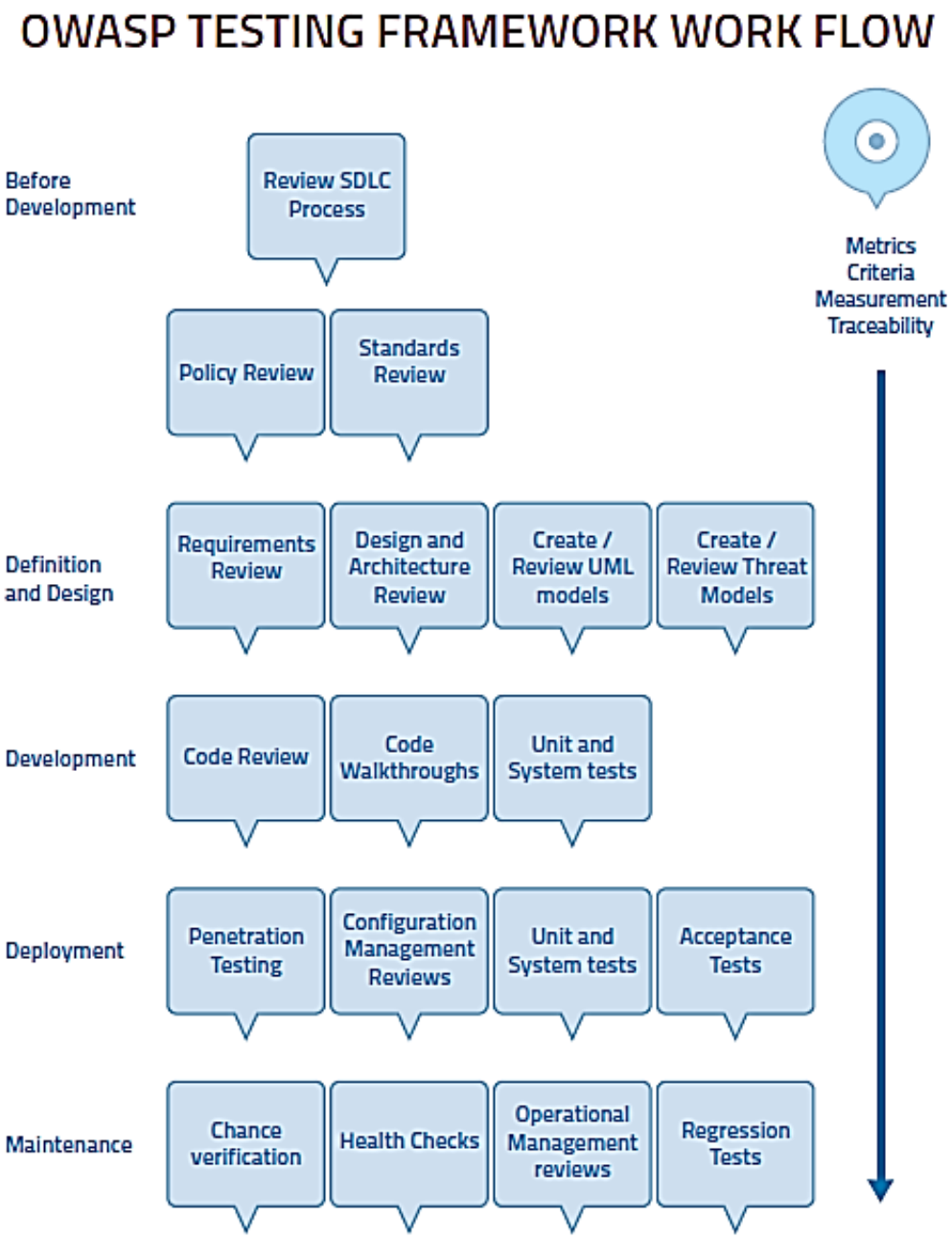
OWASP 測試指引提供了一個包含測試技術和活動的參考框架，可用於組織或企業進行應用測試，適用於軟體發展生命週期(SDLC)的各個階段，可以彈性化的延長和變形，以適應一個組織的開發過程和文化。

該測試框架包含以下內容：

- 於開發開始前進行測試。
- 於定義和設計過程中進行測試。
- 於開發過程中進行測試。
- 於部署過程中進行測試。
- 維護和運行。

OWASP 測試框架的工作流程詳見圖 13。

本文件之智慧財產權屬行政院資通安全辦公室所有。



資料來源：[11]

圖13 OWASP 測試框架工作流程

2.2.4.1. 於開發開始前進行測試

此框架建議應該在軟體開發之前即開始測試，可分為以下 3 個階段進行：

●階段 1：定義安全軟體開發生命週期

本文件之智慧財產權屬行政院資通安全辦公室所有。

於軟體開發之前必須先定義好一個適合的安全軟體開發生命週期，以確保安全考量有納入週期內的每一個階段。

●階段 2：審查策略及標準

確保有適當的政策,標準和文件,以作為開發團隊遵循的指導方針和政策。例如若是以 Java 開發應用程式，則應該準備可以參考的 Java 源碼安全撰寫原則，而若使用了加密技術，則應該準備相關的加密標準文件。

●階段 3：開發衡量標準及測試指標

在軟體發展開始之前為制定量測計畫，定義需要進行量測的準則，以顯示在開發過程和產品中的缺陷。

2.2.4.2. 於定義和設計過程中進行測試

此部分包含對安全需求、設計和架構、UML 模型，以及威脅模型的審查。以下分別進行說明。

●審查安全需求

安全需求是從安全角度來定義應用程式該如何運作，而安全需求應經過測試，這裡的測試對象指的是安全需求所定義的假設，以確認是否在需求定義上存在認知落差。例如若有一個安全需求指出：「必須是合法的使用者才能存取網站白皮書」，所謂合法的使用者指的是無須登入帳號的一般使用者，還是必須向系統註冊並通過身分驗證的使用者?應盡可能確保安全需求明瞭清晰。

●審查設計和架構

應用程式應該有與設計和架構相關的文件化資料。測試這些文件目的在於確保在設計及架構有符合安全需求。

本文件之智慧財產權屬行政院資通安全辦公室所有。

●創建並審查 UML 模型

設計架構一旦完成，建立使用統一模組化語言(Unified Modeling Language，UML)[12]模型以準確地描述應用程式如何工作。

●創建並審查威脅模型

有了設計架構審查，以及 UML 模型解釋究竟系統如何工作，還需要進行威脅建模工作。制定切合實際的威脅模型。分析設計架構，以確保這些威脅已經減少至所能接受的程度。

2.2.4.3. 於開發過程中進行測試

設計文件無法列出每一項細節，故開發者往往需要自行做許多設計面的決策。為避免開發者的決策錯誤例如與軟體設計初衷不一致或違反組織安全原則，故可以透過源碼瀏覽及源碼審查的機制，對源碼進行稽核。

●源碼瀏覽

源碼瀏覽過程中，開發者能清楚解釋被實施源碼的架構流程與邏輯，並解釋採用某種特定開發方式的原因。這個目的並不是執行源碼審查，而是在了解程式流程、布局以及結構。

●源碼審查

充分理解源碼的結構以及採取特定方式進行編碼的原因，測試者能檢驗實際源碼可能存在的安全瑕疵。

進行靜態源碼審核，需確認源碼遵循一系列清單，包括：

– 對有效性，保密性和完整性的業務要求。

– 與使用語言或框架相關的具體問題，例如 PHP 紅皮書或微軟安全編制的紅皮書或微軟 ASP.NET 安全編碼的檢查清單。

本文件之智慧財產權屬行政院資通安全辦公室所有。

– 任何專門性的行業要求，例如 ISO 17799[13]、Sarbanes-Oxley 404[14]、Visa Merchant 指引[15]，或者其他管理辦法。

2.2.4.4. 於部署過程中進行測試

●應用程式滲透測試

在應用部署完成後使用滲透測試以確保沒有遺留的安全漏洞。

●配置管理測試

滲透測試應包括對基礎設施(infrastructure)的安全檢查，因為即使應用程式是安全的，有些細項的組態可能還處在預設安裝階段而可能同樣存在漏洞。

2.2.4.5. 於維護和運行

●進行操作管理審查

需要一個詳細介紹應用程式和架構操作端管理的流程。

●進行定期的健康狀態檢查

每月或每季對應用程式和基礎設施進行例行性檢查，以確保沒有任何新的安全風險產生，該級別的安全性仍然不變。

●確保變更驗證

在每項變更被核准以及在 QA 環境進行測試並部署至正式環境後，須確保此變更不會影響到系統的安全層級，這應該被整合於變更管理流程內。

2.3. 常見的 Web 應用程式弱點

由於安全威脅種類繁多，故應優先針對風險性較高的進行檢測才能提高效率，且許多關於網路安全的研究報告(包括 Gartner、Forrester 及 IDC)指出

本文件之智慧財產權屬行政院資通安全辦公室所有。

Web 應用程式已成為主流網路攻擊目標，主要威脅之增加歸咎於 Web 應用程式的弱點，因此針對網頁應用程式的弱點檢測特別重要，下面列出三個網站或組織對於網頁應用程式弱點的最新消息發布及更新，可作為檢測項目的參考：

2.3.1. OWASP Top 10

此 OWASP 組織所執行的一項計畫，每年會發布該年對於網頁應用程式最具威脅性或最常見的十大弱點，例如 OWASP Top 10:2013 年所提出的清單[20]詳見表 1。

表1　OWASP Top 10:2013

編號	威脅項目	說明
1	注入 (Injection)	攻擊者將攻擊字串注入正常的語法中，使得網站伺服器出現非預期的結果，並執行攻擊者輸入的指令
2	身分驗證相關功能缺陷 (Broken Authentication and Session Management)	網站應用程式中使用的身分驗證相關功能(例如帳號密碼)有缺陷
3	跨站腳本攻擊 (Cross-Site Scripting, XSS)	攻擊者將可透過攻擊語法插入動態網頁程式之中，讓使用者在瀏覽網頁的過程中執行惡意攻擊語法
4	不安全的直接物件參考 (Insecure Direct Object References)	攻擊者利用 Web 系統本身的檔案讀取功能，任意存取系統的某一個檔案或資料
5	不當的安全組態設定 (Security Misconfiguration)	管理者對網站的安全組態設定不夠安全

本文件之智慧財產權屬行政院資通安全辦公室所有。

編號	威脅項目	說明
6	敏感資料暴露 (Sensitive Data Exposure)	資料傳輸或儲存時未使用妥善的保密機制造成資料暴露
7	缺少功能級別的存取控制 (Missing Function Level Access Control)	攻擊者利用存取控制的漏洞嘗試存取網站上其他網頁的網址
8	跨站冒名請求 (Cross-Site Request Forgery, CSRF)	已登入 Web 應用程式的合法使用者執行到惡意的 HTTP 指令，但 Web 應用程式卻當成合法需求處理，而使得惡意指令被正常執行
9	使用具有已知弱點的元件 (Using Components with Known Vulnerabilities)	網站使用第三方的元件，但對於安全性卻沒有妥善驗證，而使用到具有弱點的元件
10	未經驗證的重新導向與轉送 (Unvalidated Redirects and Forwards)	網頁應用程式將使用者導向至其他頁面或網站，卻沒有驗證的機制

資料來源：[20]

2.3.2. WASC 威脅分類

WASC (Web Application Security Consortium)是由許多資訊安全專家及組織所組成的非營利聯盟，該聯盟持續發布與 Web 應用程序的安全性相關之技術資訊、安全指引，以及其他有用的文件以供企業、教育機構、政府部門及應用程序開發人員等使用。

其提出 WASC Threat Classification v2.0，將網站的安全威脅進行分類(以字典順序排序)，詳見表 2。

本文件之智慧財產權屬行政院資通安全辦公室所有。

表2　WASC Threat Classification v2.0: Attack list

編號	攻擊
1	功能濫用(Abuse of Functionality)
2	窮舉法攻擊(Brute Force)
3	緩衝區溢位(Buffer Overflow)
4	內容偽冒(Content Spoofing)
5	認證與會話辨識碼的預測(Credential/Session Prediction)
6	跨站腳本攻擊(Cross-Site Scripting, XSS)
7	跨站冒名請求(Cross-Site Request Forgery, CSRF)
8	阻絕服務(Denial of Service)
9	指紋探索與辨識(Fingerprinting)
10	格式化字串攻擊(Format String)
11	HTTP 回應偷渡(HTTP Response Smuggling)
12	HTTP 回應分割攻擊(HTTP Response Splitting)
13	HTTP 請求偷渡(HTTP Request Smuggling)
14	HTTP 請求分割攻擊(HTTP Request Splitting)
15	整數溢位(Integer Overflows)
16	LDAP 注入(LDAP Injection)
17	郵件命令注入(Mail Command Injection)
18	空字元注入(Null Byte Injection)
19	未經授權執行作業系統命令(OS Commanding)
20	路徑尋訪(Path Traversal)
21	可預測的資源位置(Predictable Resource Location)
22	遠端檔案含入(Remote File Inclusion, RFI)

本文件之智慧財產權屬行政院資通安全辦公室所有。

編號	攻擊
23	路由迂迴攻擊(Routing Detour)
24	會話固定攻擊(Session Fixation)
25	SOAP 陣列濫用(SOAP Array Abuse)
26	SSI 注入(Server-Side Include Injection)
27	SQL 注入(SQL Injection)
28	URL 重定導向濫用(URL Redirector Abuse)
29	XPath 注入(XPath Injection)
30	XML 屬性爆毀(XML Attribute Blowup)
31	XML 外部實體(XML External Entities)
32	XML 實體擴張(XML Entity Expansion)
33	XML 注入 (XMLInjection)
34	XQuery 注入(XQuery Injection)

資料來源：[21]

WASC Threat Classification v2.0 亦將常見的弱點進行以下分類(以字典順序)，詳見表 3。

表3 WASC Threat Classification v2.0: Weaknesses list

編號	攻擊
1	應用程式設定不正確(Application Misconfiguration)
2	目錄索引(Directory Indexing)
3	不當的檔案系統使用權限(Improper Filesystem Permissions)
4	不當的輸入處理(Improper Output Handling)
5	資訊外洩(Information Leakage)

本文件之智慧財產權屬行政院資通安全辦公室所有。

編號	攻擊
6	不安全的索引(Insecure Indexing)
7	應用程式不足以因應自動化攻擊(Insufficient Anti-automation)
8	驗證機制安全性不足(Insufficient Authentication)
9	授權機制安全性不足(Insufficient Authorization)
10	密碼還原機制安全性不足(Insufficient Password Recovery)
11	應用程式流程驗證與控制不足(Insufficient Process Validation)
12	Session 過期機制安全性不足(Insufficient Session Expiration)
13	傳輸層保護機制安全性不足(Insufficient Transport Layer Protection)
14	伺服器設定不當(Server Misconfiguration)

資料來源：[21]

2.3.3. CWE / SANS Top 25

CWE (Common Weakness Enumeration)是由 US-CERT 所資助的 MITRE 組織所發布有關網頁應用程式安全弱點的類別。SANS (SysAdmin, Audit, Networking, and Security)則是個專門進行資訊安全培訓、認證與研究的機構，該機構對於資安領域有著深入地研究及分析，網站上提供最新資訊安全相關研究。CWE / SANS Top 25 就是由 MITRE 和 SANS 合作，共同發布網頁應用程式前 25 大的威脅[28]，詳見表 4。

表4　CWE / SANS Top 25

等級	威脅
1	SQL 注入(SQL Injection)
2	作業系統命令注入(OS Command Injection)

本文件之智慧財產權屬行政院資通安全辦公室所有。

等級	威脅
3	緩衝區溢位(Buffer Overflow)
4	跨站腳本攻擊(Cross-site Scripting, XSS)
5	重要功能缺乏驗證(Missing Authentication for Critical Function)
6	缺乏授權機制(Missing Authorization)
7	將驗證的機密性資料直接寫入(Use of Hard-coded Credentials)
8	敏感性資料缺乏加密機制(Missing Encryption of Sensitive Data)
9	沒有限制危險類型的檔案上傳(Unrestricted Upload of File with Dangerous Type)
10	安全決策依靠不可信任的輸入(Reliance on Untrusted Inputs in a Security Decision)
11	非必要的執行權限(Execution with Unnecessary Privileges)
12	跨站冒名請求(Cross-Site Request Forgery, CSRF)
13	路徑尋訪(Path Traversal)
14	下載源碼時未做完整性檢查(Download of Code Without Integrity Check)
15	不正確的授權(Incorrect Authorization)
16	包含了非信任及非控制範圍的功能(Inclusion of Functionality from Untrusted Control Sphere)
17	對重要資源不正確的權限指派(Incorrect Permission Assignment for Critical Resource)
18	使用有潛在危險性的函式(Use of Potentially Dangerous Function)
19	使用具瑕疵或具風險的加密演算法(Use of a Broken or Risky Cryptographic Algorithm)
20	緩衝區大小計算不正確(Incorrect Calculation of Buffer Size)

本文件之智慧財產權屬行政院資通安全辦公室所有。

等級	威脅
21	沒有適當的限制不斷的驗證企圖(Improper Restriction of Excessive Authentication Attempts)
22	URL 重新導向至非信賴的網站(URL Redirection to Untrusted Site ('Open Redirect'))
23	未控制的字串格式(Uncontrolled Format String)
24	整數溢位或越界繞回(Integer Overflow or Wraparound)
25	雜湊加密時未加上 salt 搗亂(Use of a One-Way Hash without a Salt)

資料來源：[28]

2.4.結語

本章節針對軟體之安全測試進行介紹，包含測試注意事項以及相關之測試標準，包含 ISO/IEC 27001:2013、IEEE 829:2008、ISO/IEC/IEEE 29119:2013，以及 OWASP 測試指引。

本文件之智慧財產權屬行政院資通安全辦公室所有。

3. 安全需求驗證

本節說明如何透過安全需求定義軟體應符合的安全基準。

3.1.安全軟體需求

一般開發團隊以軟體支援組織業務為重，於定義需求時很容易著重在功能需求而忽略安全考量，故將軟體安全需求作為發展階段明確的部分極為重要。安全軟體發展流程即具備事前安全性的思維，亦即從流程初期需求階段就考慮到軟體安全性的相關需求。安全需求最主要的差異在於針對保護資訊資產、功能以及避免軟體被誤用的方向進行考量，相關考量的因素包含：

- 對於軟體可能被誤用及攻擊方式的了解。
- 軟體會存取或提供的資訊資產，其相對於組織面臨的風險、法規及對商譽的可能影響，是否具有適當程度的保護措施。
- 軟體架構及可能的攻擊者。
- 可行的控制措施及其花費與效用。

對安全軟體的開發而言,軟體安全需求以及安全測試是最重要的兩個部分。若要完成一個成功的安全測試專案,首先要清楚了解其測試的目的是什麼，而這些目的可由安全需求來指定。

以下各小節將依序說明軟體安全需求類型的種類,如何進行安全需求萃取，以及用於後續追蹤需求的方法：「需求追溯矩陣」。

3.1.1. 安全需求原則

在分析與描述安全的軟體需求時，OWASP 建議應掌握 SMART+原則具體描述[16]：

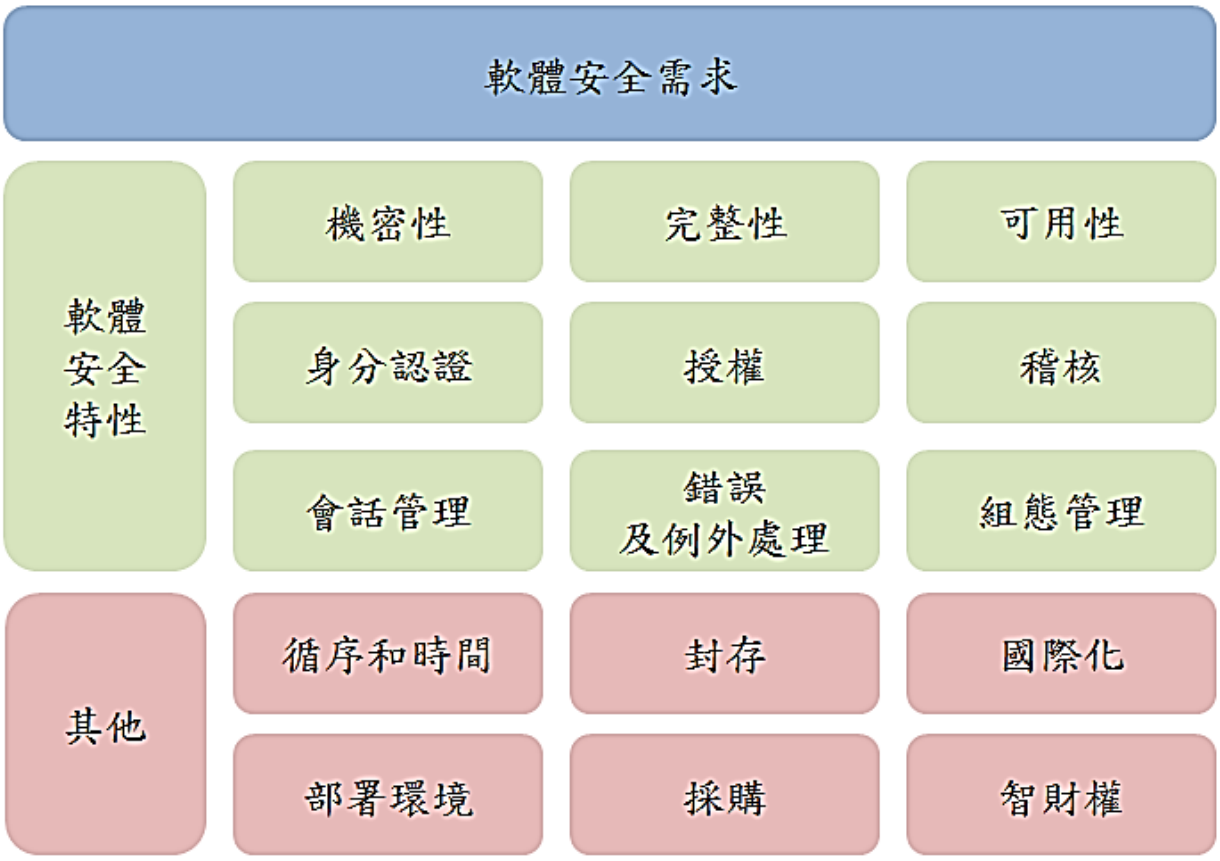
本文件之智慧財產權屬行政院資通安全辦公室所有。

- ●Specific：明確的，不模糊的。需求必須提供詳細的說明，同時包含一致的專業用語。
- ●Measurable：可量化、可量測的。需求必須可以被分析與測試。
- ●Appropriate：適當、符合所需的。需求必須被驗證以確保符合真正需要。
- ●Reasonable：有依據、具合理性。需求執行前最好參照類似專案。
- ●Traceable：可追蹤、有建檔及有紀錄可循。需求必須融入開發生命週期以容易追蹤或驗証。

3.1.2. 軟體安全需求類型

可大致區分為來自「軟體安全特性」及「其他」兩大類[17]，詳見圖 14 所示，分別說明如下。

本文件之智慧財產權屬行政院資通安全辦公室所有。



資料來源：[17]

圖14 軟體安全需求類型

●機密性

防範資料的洩露。機密性需求是私有或機密的資訊不洩露給非授權人員[18]。機密性需求可分為資料 1.儲存時 2.傳送時 3.封存時。

●完整性

保護資訊或程序不被故意或意外的非授權變更。

●可用性

可用性是確保授權使用者需要時可以存取系統或服務。

本文件之智慧財產權屬行政院資通安全辦公室所有。

●身分認證

確認實體(Entity)要求身分之真實性的程序。實體可能是人、程序或硬體裝置等。身分認證要求實體提供憑據以證明其確實為其聲稱的主體。

●授權

確認已認證的實體是否被允許對特定資源進行特定動作的程序。

●稽核

協助建構出使用者行為的歷史紀錄。軟體須可以提供誰、做了什麼、在什麼地方與何時等資訊。

●會話管理

會話(Session)是用來維持軟體中的狀態，但是也對軟體安全性造成影響。成功的身分認證後，會話的識別碼(Session Identifier，Session ID)被發給使用者，Session ID 被用來追蹤使用者，維持其已進行身分認證的狀態，直到其身分認證失效或會話過期。無狀態的協定(例如，HTTP)沒有會話則使用者每一次進行動作，就必須重新進行身分認證，這樣的作法通常無法被使用者接受。然而，使用會話的作法，又面臨 Session ID 如果被竊取，則有被身分偽冒的風險，因此必須規劃安全的會話管理方式。

●錯誤及例外處理

錯誤(Error)及例外(Exception)是資訊洩漏的潛在來源之一。詳細的錯誤訊息和未處理的例外報告，可能會導致洩漏內部應用程序架構、設計和設定的資訊，因此必須在安全需求中明確說明處理錯誤及例外的要求，以避免資訊洩漏威脅。

●組態管理

本文件之智慧財產權屬行政院資通安全辦公室所有。

組態管理是用來建立或維持軟體功能、效能及相關特性的一致性和穩定。軟體所依賴的底層元件，須注意其安全組態設定以確保較高的安全性。對於軟體的設定參數，應加以保護並管理其變更。

●循序(Sequencing)與時間

軟體中關於循序與時間的設計失誤，將導致競賽情況(Race Condition)與檢查時間/使用時間(Time of Check/Time of Use)這類問題，因此相關需求應被注意並加入需求規格中。

●歸檔封存(Archiving)

組織若因為營運持續、內部政策或法規要求，需要將資料做歸檔封存動作，須注意保護資料的相關安全需求。

●國際化

國際化是設計一個軟體或系統，使其未來不需要重大軟體工程變更，便可以適用於各種語言和地區的過程。軟體因應國際化需求，將會有語系及時區的問題需要克服，在這個修改過程當中可能會產生安全問題，需要特別注意。

●部署環境

現今軟體大多依賴其他底層元件或中間軟體才得以執行，因此其部署環境的安全將影響整體安全性。須注意部署環境的安全設定、定期更新及減少受攻擊面等安全議題，並產出相對應的安全需求。

●採購

組織評估效益及專業能力等問題，可能採取以採購的方式取得所需的軟體。除了功能需求外，應考量安全需求並將其包含在法律保護機制，如合約和服務層級協議(Service Level Agreement, SLA)中。

本文件之智慧財產權屬行政院資通安全辦公室所有。

●智財權

若組織開發的軟體將作為商用軟體銷售，則需要特別注意智慧財產權保護方面的安全需求，例如源碼混淆(Code Obfuscation)、源碼簽章(Code Signing)、防止竄改(Anti-tampering)等機制。

3.1.3. 正面需求和負面需求

安全需求可分為正面需求和負面需求[11]，正面需求闡述預期的功能性可以通過安全測試驗證，包含系統應具備的功能以及實現方式。負面需求則是以風險控制的角度進行描述，也就是他們需要驗證非預期的行為，描述系統不應該做的行為。

3.1.3.1. 正面需求

正面需求從功能安全需求角度透視，適當的標準、政策和管理條例推動了安全控制的類型的需求和控制功能性的發展。正面需求的驗證是由斷言的預期功能組成，它能在重建的測試條件中進行測試；並根據預定義的輸入運行測試斷言預期的輸出情況是“失敗/正常”條件。為了安全需求能通過安全測試驗證，安全需求必須是功能驅動的，並且突出預期的功能(做什麼)以及隱含的實現(如何做)。以下為幾個正面需求的範例：

●保護使用者認證資訊。

●保護傳輸和儲存時的機密性。

●在一定數量的登錄失敗後，鎖定使用者帳號。

●使用者登錄失敗後不顯示具體失敗資訊。

●只允許輸入由字母數位元並且包含特殊字元組成的至少六個字元的密碼。

本文件之智慧財產權屬行政院資通安全辦公室所有。

- 使用者要修改密碼，必須通過舊密碼、新密碼、對密碼問題的回答的驗證，以防止通過密碼修改功能對密碼進行窮舉(Brute Force)搜索攻擊。
- 採用密碼重置功能時，系統將臨時密碼發送到使用者指定的信箱之前，需驗證使用者註冊時的使用者名稱和郵箱地址。該臨時密碼只能使用一次。一個密碼重置的網頁連結將發送給使用者。
- 密碼重置網頁應該驗證使用者的臨時密碼，新密碼，以及使用者對密碼問題的回答。

3.1.3.2. 負面需求

安全測試也是需要風險控制的，也就是他們需要驗證非預期的行為。負面需求即是描述應用系統什麼不應該做的行為。負面需求往往更難以測試，因為找不到預期的行為。負向需求如何萃取詳見第 3.1.4.4 小節。

以下為幾個負面需求的例子：

- 應用系統不允許未經授權的使用者登入。
- 應用系統不允許未經授權的資料庫存取。
- 應用系統不允許修改或毀壞資料。
- 應用系統不允許被破壞或者被惡意使用者用於未認證的金融交易事務。

3.1.4. 需求萃取方式(Protection Needs Elicitation, PNE)

安全需求的來源可能有很多，以下小節介紹導出安全需求的方式。

3.1.4.1. 來自軟體安全政策的需求

軟體安全需求的其中一個來源是組織的政策、章程或規範，通常包含高階的、商業上的需求，因此需要被分解或細化為詳細的安全需求。

本文件之智慧財產權屬行政院資通安全辦公室所有。

然而此程序的高階需求來源並不只限於內部政策，亦可能來自於包含：

- 外部法規，例如我國「個人資料保護法」。
- 國際標準，例如 ISO 27001、BS10012。
- 相關業界標準或規範，例如我國「政府組態基準(Government Configuration Baseline,簡稱 GCB)、應用程式安全驗證標準。

若組織已有較明確的軟體安全政策，為遵循法規或客戶需求而將高階需求對應至特定一項或多項控制措施，則發展詳細的軟體安全需求將會簡單許多。

3.1.4.2. 資料分級

在安全領域中，機敏資料或資訊通常被認為是重要的資訊資產，因而需要特別的保護措施。但是並非所有的資料都需要相同程度的保護，可以公開的資料甚至可能不需要任何保護措施。因此，資料分級就是考量資料被洩漏、竄改、破壞後的影響而賦予資料敏感程度的行為。

組織應建立資料分級的準則，使軟體若針對機敏資料進行蒐集、處理與利用時能據以判別資料的風險等級。資料分級後的結果可以被用來決定相對應的軟體安全需求。資料分級的主要目的是降低資料保護的成本，最大化投資報酬率。意即，根據資料的等級實施相對應程度的安全控制措施。組織可以規範資料分級為特定等級時，於不同的安全特性類別，例如機密性、完整性、可用性等，規範適用的共通或特定控制措施。

3.1.4.3. 軟體安全問卷

問卷對於蒐集軟體安全需求是相當有效率的方法。問卷應該同時包含明確特定的問題與開放式的問題。開放式的問題可獲得更多此軟體安全相關的資訊，但其結果不容易有一致性的評判。問卷問題應該依循軟體安全特性

本文件之智慧財產權屬行政院資通安全辦公室所有。

與安全設計法則的分類發展，其結果可以直接產生出軟體安全需求。

軟體安全問卷蒐集軟體的基本資訊以及各面向的狀況，可用來進行風險等級分類。對分類結果為高風險的軟體，後續應有更深度的架構檢視分析。

3.1.4.4. 誤用案例模型(Misuse Case Modeling)

在軟體工程中常以圖解形式描述系統使用案例，以展示操作者與系統之間的互動關係，說明識別應用系統中的執行者身分、關係、每個方案行為的設想結果、可選擇的行為等。

誤用案例的建模方式相似於使用案例，兩者的繪製方式相同，但是誤用案例記錄的是刻意或意外的狀況下，誤用者如何以非原本預期的方式與軟體進行互動。誤用案例可描述應用系統中的未計畫與惡意使用方案，透過發展負面的使用情境可以用來幫助識別安全需求。

從使用案例可發展出誤用案例，藉由採用攻擊者的角度，誤用案例反轉了原先使用案例中正常使用者的視角，審閱使用案例中的各個步驟並思考是如何被惡意利用的，就能發現未被妥善定義的潛在漏洞，並進而擬定對抗措施以緩和這些漏洞所造成的風險，其關鍵是描述所有可能性，或至少描述最重要的使用和誤用情境。

從使用案例中要獲得的安全需求時，重要的是定義功能情景和負面情景，並建成圖解形式，判斷哪些是有資安風險的部分，並將其寫入安全需求。以下利用範例逐步進行說明：

●描述功能情景

使用者透過帳號和密碼進行認證。應用系統允許通過身分認證的使用者存取系統，而當認證失敗時要求使用者重新輸入帳號及密碼，並且為使用者提供具體的錯誤資訊。

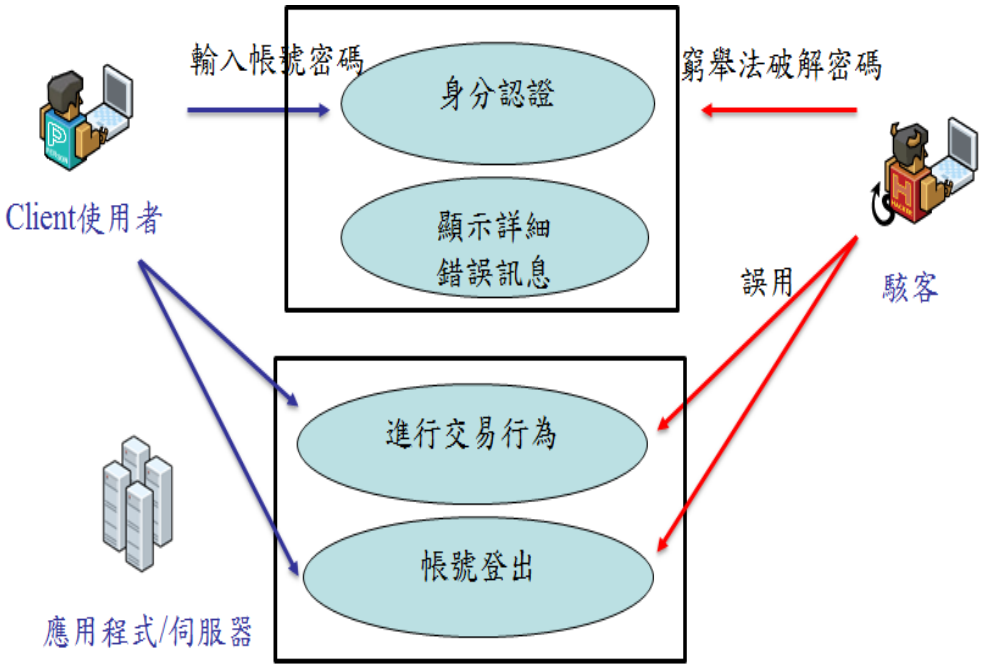
本文件之智慧財產權屬行政院資通安全辦公室所有。

●描述負面情景

驗證失敗時提供的具體資訊使攻擊者能猜測到哪些帳號實際上是合法的註冊帳號。攻擊者試著利用窮舉法破解一個合法帳號的密碼。

●建立使用案例及負面案例

功能情景包括使用者行為(輸入使用者名稱和密碼)和應用行為(認證使用者或提供認證失敗資訊)。誤用案例為攻擊者的行為，即從提示的錯誤資訊中猜測合法的使用者名稱並設法通過窮舉破解密碼。誤用案例的例子詳見圖 15 所示，透過加入攻擊者之角色存取一般使用者登入後之正常使用行為，制定出該使用案例的誤用案例。通過圖解可以看到使用者操作可能造成的威脅(誤用)，就可通過對抗措施緩和應用行動可能帶來的威脅。



資料來源：[11]

圖15 誤用案例範例

●安全需求總結

在這種情況下，可以導出用於認證的安全需求：

本文件之智慧財產權屬行政院資通安全辦公室所有。

– 5 次登錄嘗試失敗以後，帳號鎖定。

– 登錄失敗資訊不具體顯示。

3.1.4.5. 主體-物件矩陣(Subject-Object Matrix)

當軟體中具有多個主體(或角色)需要存取功能或物件，釐清每個主體(角色)被允許存取的功能與物件是非常重要的。物件是主體在軟體中可以互動的項目。高階的物件若可以被細化成細部的物件，將可以更明確的表示出主體對物件關係的正確性。例如，資料庫物件可以被細分為資料表格、資料表格視角(View)、預儲程序等，每種物件又會有多個實體。主體-物件矩陣中也應該正確地表示第三方元件與主體間的關係。主體-物件矩陣用來識別出允許的行為，可以幫助誤用案例模型的發展。

3.1.4.6. 腦力激盪

腦力激盪是最快也最非結構化產生軟體安全需求的方式。於此過程中，成員不需要闡述提出該軟體安全需求背後的原因，而是持續的記錄所有成員想到的軟體安全需求。雖然此快速產出軟體安全需求的方式相當符合現今軟體需要快速發展的環境，卻不建議僅採取此種作法。

腦力激盪有以下幾個缺點：

- 有很高的可能性，提出的軟體安全需求並不直接與組織的商業決策需求或是軟體的技術背景、安全背景吻合。
- 很容易有所遺漏，忽略某些重要的安全考量。
- 因個人想法是相當主觀的，通常結果不全面或是不一致。

腦力激盪可以作為先期軟體安全需求產出的方式，但是需要進一步採用其他更加結構化的方式來確保最終結果的全面性。

本文件之智慧財產權屬行政院資通安全辦公室所有。

3.1.4.7. 來自設計階段活動的回饋

安全軟體設計階段的活動，其結果也應回饋並修正安全需求：

●威脅建模

威脅建模採用系統化的方法，以攻擊者的角度，識別可能影響軟體系統的威脅並進行評估。基於對架構與實作的了解，識別與評估威脅後，以風險高低的順序對威脅發展適當的控制措施。

威脅和對策分類的焦點是根據威脅和弱點的起因定義安全需求。威脅可以使用 STRIDE 分類，例如，偽冒(Spoofing)，竄改(Tempering)，否認行為(Repudiation)，資訊洩漏(Information Disclosure)、拒絕服務(Denial

of Service)和權限提升(Elevation of Privilege)。

●架構風險分析

架構風險分析可以分成三個主要的子流程：

– 抗攻擊能力分析(Attack Resistance Analysis)。

– 模糊分析(Ambiguity Analysis)。

– 底層框架弱點分析(Underlying Framework Weakness Analysis)。

架構風險分析找出的安全問題與缺陷，應發展對應的控制措施。

●攻擊面分析

攻擊面分析的結果，若有可以減少權限或存取性的地方，應調整相對應的設計與需求。

●安全設計審查

分析設計中不安全的部分與缺乏的安全設計後，相對應的安全需求列表應

本文件之智慧財產權屬行政院資通安全辦公室所有。

該被建立並被回饋到「需求追溯矩陣」中。

3.1.4.8. 需求追溯矩陣(Requirements Traceability Matrix, RTM)

各種需求萃取方式(PNE)產出的軟體安全需求，可以放入需求追溯矩陣(Requirements Traceability Matrix, RTM)進行記錄與後續追蹤。

需求追溯矩陣是一個記錄並對應商業需求、功能需求、測試案例編號等需求相關資訊的表格。它可以被用來記錄軟體安全需求，讓測試人員清楚地知道每個安全需求應該以那些測試案例進行驗證，並讓專案管理者可追蹤每個安全需求的後續處理狀態。

對軟體開發流程來說，需求追溯矩陣提供以下好處：

●確保沒有範圍認知上的落差。

●確保後續的設計會滿足特定的安全需求。

●確保實作不會偏離安全設計。

●提供定義測試案例的基礎。

需求追溯矩陣是一種對應需求及後續活動，以方便軟體專案追蹤後續需求是否真正被落實的概念，並非固定採取紙本表格方式呈現，亦可採用自動化系統或軟體進行記錄。

3.2.安全需求驗證實務活動

進行安全測試的主要目的是驗證安全需求是否已被滿足，從安全評估角度看，在安全軟體發展生命週期(SSDLC)的不同階段(例如：研究規劃、需求分析、系統設計、系統開發、系統測試、系統部署、系統維護)，可以使用不同的測試工具和測試方法能驗證安全需求。例如，使用威脅模型能在系統設計階段識別安全性漏洞，安全源碼分析和審查能在系統開發階段於源

本文件之智慧財產權屬行政院資通安全辦公室所有。

碼中發現安全問題，滲透測試能在系統測試階段發現漏洞。

安全需求在架構設計審查階段獲得的認可威脅，並允許記錄對策要求(即：互相認證)。而這些對策能在後來的安全測試中得到確認。

在 SSDLC 較早階段發現的安全問題需要在測試計畫中記錄下來，以便能使用安全測試證實這些安全問題。通過結合各種測試技術的檢測結果，就能得到更好的安全測試案例，同時增加安全需求的可信度。例如，結合滲透測試和源碼分析的結果能區分真正的漏洞以及未被利用的漏洞，並能透過分析研判是否為誤判。

3.2.1. OWASP 應用程式安全驗證標準

進行應用程式的安全驗證現存有各式各樣的檢驗方法、檢驗項目或涵蓋範圍，安全檢測人員往往依照組織的政策或自己的判斷選擇檢測項目及範圍，可能造成安全檢測的嚴謹度不一致。故 OWASP 組織提出的應用程式安全驗證標準(Application Security Verification Standard, ASVS)[19]，其目的是將應用程式安全驗證所涵蓋的層級與範圍予以正規化。

OWASP ASVS 依驗證的涵蓋範圍及嚴謹程度，將驗證項目區分為 4 個等級，包含粗略(cursory)、投機(opportunistic)、標準(standard)，以及進階(advanced)，愈高階的等級包含更多的驗證項目及更高的嚴謹度。其列舉的軟體安全需求分類包含下列 13 項：

- 身分驗證(Authentication)。
- 會話管理(Session Management)。
- 存取控制(Access Control)。
- 資料輸入驗證 (Input Validation)。
- 儲存階段密碼學應用(Cryptography at Rest)。

本文件之智慧財產權屬行政院資通安全辦公室所有。

●錯誤處理與日誌(Error Handling and Logging)。

●資料保護(Data Protection)。

●通訊安全(Communication Security)。

●HTTP 協定安全(HTTP Security)。

●惡意程式控制措施(Malicious Controls)。

●商業邏輯(Business Logic)。

●檔案與資源(Files and Resources)。

●行動裝置(Mobile)。

3.2.2. 需求階段的安全需求驗證

軟體安全需求必須定義清楚才能作為後續軟體設計與開發的依據，進行安全測試時也才能夠驗證是否有滿足事先定義好的安全需求，定義安全需求時應依循安全需求原則，以符合安全的軟體需求，並應包含以下類別：

●輸入資料驗證需求

輸入資料必須符合系統要求的範圍與格式，避免系統無法正常處理或遭受攻擊。在確認來自表單、資料庫及設定檔等的輸入值安全無虞之前，要假設所有的輸入資料都有風險，系統必須檢查所有的輸入值。

●輸出資料確認需求

在資料輸出到網頁、寫到紀錄檔、存到檔案系統及儲存到資料庫之前，必須先編碼跳脫特殊字元，以避免儲存或輸出非預期的結果。

●資料保護需求

所有機敏資料都必須加密保護、存放在安全的位置及具備權限控管機制，

本文件之智慧財產權屬行政院資通安全辦公室所有。

避免作業系統、Web Server 及 Web Services 的弱點造成機敏資料遭到惡意存取。

●驗證檢查需求

在允許限定存取的系統資源被使用前，必須驗證使用者的權限，避免系統資源被誤用或濫用；此外，身分驗證與存取控制無論成功與否都必須被記錄，以利日後的稽核。

●錯誤訊息處理需求

錯誤訊息必須在系統上線前轉換成與系統或資料庫無關，而只與使用者相關的資訊，例如當系統發生錯誤時，不顯示哪個檔案的哪一行出錯、不提供 Web Server 或程式語言的版本及不條列詳細的除錯訊息。此外，當資料庫發生錯誤或找不到資料，只提供一般的訊息，不提供詳細的欄位名稱或與資料庫相關的訊息，例如當使用者查詢不到某一筆資料時，只提供查無此筆資料的訊息，不提供在哪個資料庫的哪一個欄位沒有該筆資料的訊息。

●安全架構文件需求

完整的系統架構說明文件，可提升系統在開發、測試及維護階段的效率。參考安全架構文件可了解經常使用與不再使用函式，刪除不再使用函式有助於系統管理與降低被攻擊的風險；此外，強化與修改經常使用函式，可以避免入侵成功的機率與提升系統的效能。

●源碼完整性需求

確認源碼沒被置入惡意代碼源碼變數內容、函式庫及檔案路徑未被竄改，避免資源被置換所帶來的風險。

●法律與標準的符合性需求

本文件之智慧財產權屬行政院資通安全辦公室所有。

確認安全需求應符合相關的國際標準、法律規範或組織的安全政策等。

3.2.3. 設計階段的安全需求驗證

依照安全需求進行設計，應針對設計結果建立 UML 模型以說明系統運作的方式，並應分析設計架構以進行威脅建模工作，分析軟體設計上是否仍具有安全威脅，確保這些威脅已經減少至所能接受的程度。如何進行威脅建模可詳見 3.1.4.7 小節。

3.2.4. 開發階段的安全需求驗證

在安全軟體發展生命週期中，開發者應依照安全源碼撰寫標準要求進行開發，並注意避免軟體常見漏洞及實作必要控制措施，以提升軟體的安全性。由開發者對源碼進行初步的安全檢查是有效發現安全缺點與未正確進行安全實作的方式。

於開發階段所做的安全需求驗證活動主要可包含兩種，一種為針對源碼進行靜態分析及審查(請詳見 4.1 節)，另一種則是由開發人員進行單元測試。

3.2.4.1. 單元測試

軟體元件(Component)開發完成後應對其進行單元測試(Unit Test)，其對象可能為一個函式(Function)或方法(Method)、一個類別或者是可獨立測試的子系統，而這應是屬於開發者的責任(通常是實際開發該元件的人員)而非由測試人員進行。進行單元測試目的在於確認單元功能的運作符合預期，包含功能性及安全性的需求。單元測試可以發現程式或實作的邏輯錯誤，使問題及早暴露，便於問題的定位與解決，進而減少未來系統測試的複雜度。

開發者可以善用開發框架(framework)所提供的單元測試工具以檢驗開發的源碼的安全性，目前亦有許多軟體開發 IDE 也都內建單元測試的功能，如 Borland 的 JBuilder 便將 JUnit 整合至其開發環境中，甚至提供精靈(wizard)

本文件之智慧財產權屬行政院資通安全辦公室所有。

以幫助建立單元測試。NetBeans 或 Eclipse IDE 亦支援建立 JUnit 的測試專案。常見的程式語言單元測試工具詳見表 5。

表5　單元測試工具

類別	工具
Java framework	JUnit
PHP framework	PHPUnit
C frameworks	CUnit
C++ frameworks	UnitTest++ 與 Google C++
.NET framework	NUnit
Python framework	PyUnit、py.test
Delphi framework	DUnit

資料來源：本指引整理

開發者進行單元測試時除了進行功能性的驗證，亦應針對常見的安全議題設計相關測試案例，以驗證是否具備有效的安全對抗措施。若能利用現成的使用案例來設計單元測試案例亦是一種很好的實踐方式，其可幫助開發者從早先已定義的使用和誤用案例中獲得安全測試功能、方法和分類。

安全單元測試同時驗證正面判斷和負面判斷，例如驗證下列項目：

●輸入參數的型別驗證

例如檢查資料型別、型別轉換(casting)等。

●輸入參數值的範圍驗證

本文件之智慧財產權屬行政院資通安全辦公室所有。

例如檢查邊界值(Boundary Value)、非法範圍的數值與空值(Null Value)等。

●溢位(Overflow)

例如算術溢位、緩衝區溢位及向下溢位(Underflow)。

●浮點數精確度運算誤差

例如使用 float 或 double。

●物件的存取權限

例如檢查是否使用了錯誤的存取控制(private、protected、public)。

●錯誤和例外處理

例如檢查除數為 0 的算術例外。

●日誌(Logging)

例如檢查日誌功能是否正常，以及是否包含了過於敏感的資訊。

3.2.5. 測試階段的安全需求驗證

在安全軟體發展生命週期的測試階段進行安全測試之目的，主要在於驗證軟體中安全控制措施的實施，已為各個層面提供安全保護。安全測試者必須充分了解安全需求，並具備較深入的安全測試技術，包含了解 Web 應用程式弱點以及滲透測試手法。故此階段通常會由品質保證(Quality Assurance, QA)測試者負責軟體功能性測試，而由專門的資訊安全人員或依靠組織自己的專業道德入侵團隊進行安全測試，常見的進行方式是由安全測試者模擬即時攻擊情景對系統進行滲透，並決定是否對漏洞進行利用而讓應用程式面對真實的風險，這些包括一般的 Web 應用程式弱點、早期在安全軟體開發階段就已經識別的安全事件以及被安全威脅模型等。即時攻擊情景可以

本文件之智慧財產權屬行政院資通安全辦公室所有。

用人工試驗技術或利用自動化工具進行測試，以驗證系統的安全性。此方式主要在驗證安全需求是否已充分被滿足，並非是惡意攻擊系統例如竊取系統上之機密資料，故這個類型的安全測試通常被稱為「道德攻擊測試」或是「滲透測試」，其詳細的進行手法詳見 4.2 小節。

3.2.6. 驗收階段的安全需求驗證

驗收測試是向未來的使用者表明系統能夠像預定要求那樣工作，包含驗證功能性及非功能性的需求。典型的驗收測試方法是由開發人員或供應商將軟體安裝於驗收方的模擬測試環境或真實環境,並對驗收方實際操作演示。驗收方則確實依據測試計畫與測試案例進行驗收測試，並記錄測試結果。測試過程所產生的問題依據測試計畫進行管理與追蹤，最後產出測試評估報告，以評估是否可以進行系統移交或部署階段。如未通過驗收測試則不予驗收或安排下一回合的驗收測試直到滿足驗收條件為止。

進行使用者驗收測試(User Acceptance Test , UAT)時，針對安全驗證的部分重要項目之一為系統組態配置的安全性，可透過實際的系統操作介面及測試用資料，驗證系統之組態配置是否已滿足安全需求。例如參考我國政府組態基準(Government Configuration Baseline,簡稱 GCB)對電腦系統環境的安全性設定之要求，包含密碼長度、更新期限等。

3.2.7. 軟體部署與維運階段的安全需求驗證

當 Web 應用程式上線進行運作時，仍應注意其安全性。可善用 Syslog 或軟體本身所具的記錄機制，並針對程式異動、檔案異動、安全修補以及入侵事件等議題進行安全稽核。

3.3.網頁自動化測試

一般習慣上會以人工的方式進行網站測試，以驗證網站之功能符合系統需求及安全需求，但系統即使通過完整的測試，日後仍可能會因為需求異動

本文件之智慧財產權屬行政院資通安全辦公室所有。

或是功能修改而產生系統需要重新測試及回歸測試等議題，若完全以人工進行測試往往需要花費大量的時間，效率較差且可能造成測試涵蓋率的不足，如此則可以考慮使用自動化網頁測試工具輔助進行測試。

自動化網頁測試工具可以透過模擬人為的動作進行網頁的操作，並且進行驗證結果是否正確，常見的操作方式是使用工具所提供的「測試腳本錄製功能」，可以將測試者透過瀏覽器所進行的網頁操作步驟錄製成數個「測試腳本」，並允許測試者針對測試腳本編輯操作步驟及相關參數等，或是允許測試者直接編寫測試腳本以提供更大的測試彈性，自動化測試工具即可依照測試腳本進行網頁操作的模擬，而省去人工操作的時間，如此可以快速地進行重新測試。

自動化網頁測試工具產品有許多的選擇，例如 Selenium[21]、Geb[22]、Sikuli[24]、IEUnit[25]或 Badboy[26]等。

以下針對 Selenium、Geb 以及 Sikuli 進行簡單介紹。

3.3.1. Selenium

Selenium 是為瀏覽器自動化(Browser Automation)需求所設計的一套工具集合，讓程式可以直接驅動瀏覽器進行各種網站操作，例如操作網頁表單資料、點選按鈕或連結、取得網頁內容並進行檢驗等，一般常用來模擬真實使用者操作瀏覽器的行為以進行網站自動化測試。其運作原理是呼叫瀏覽器，透過瀏覽器開啟網頁，接著對網頁中特定元件觸發事件，原則上只要是網頁中可顯示的元件都可以嘗試觸發事件，觸發的事件可為點擊滑鼠或是鍵盤輸入。它能夠直接獲取即時的內容，包括被 JavaScript 修改過的 DOM 內容，讓程式可以直接與網頁元素即時互動、執行 JavaScript 程式，因此也適用於前端採用 AJAX 技術的網站[23]。目前 Selenium 有四個主要的專案發行：

本文件之智慧財產權屬行政院資通安全辦公室所有。

●Selenium IDE

●Selenium Remote Control

●Selenium WebDriver

●Selenium Grid

其中 Selenium IDE 因具有圖形化介面,能用滑鼠簡單地完成一些測試案例，是較容易上手的使用方式。Selenium IDE 是 Firefox 附加元件(extension)，需要搭配 Firefox 瀏覽器才能使用。Firefox 安裝 Sellenium IDE Plugins 後，即可操作 Selenium IDE 以進行錄製網站的動作步驟並產生測試案例(Test Case)的內容。錄製完成後，可以用「播放」重新把網站操作過程重播一次。

Selenium 的使用範例詳見圖 16。此範例以 Google 搜尋為例，建立一組測試案例包含：

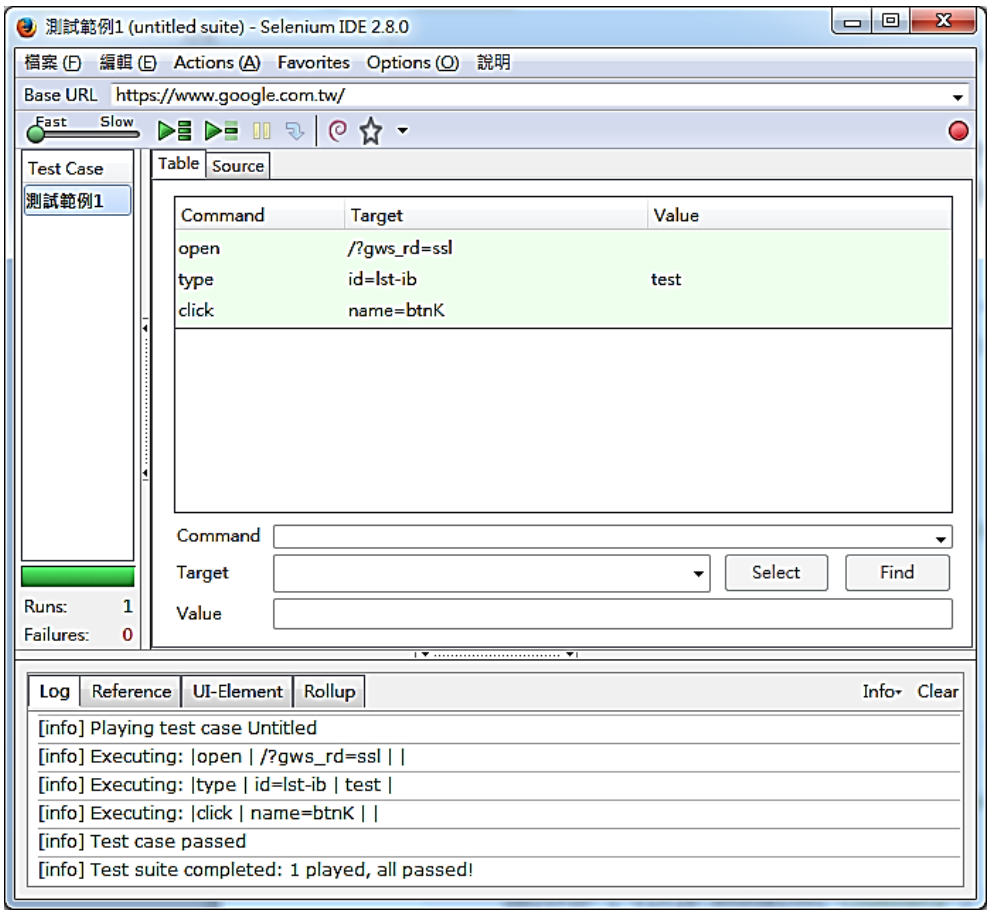
1.打開網址 URL：「https://google.com.tw」。

2.找到填寫關鍵字的表單文字欄位 <input />，填入字串(test)。

3.找到「搜尋」按鈕，按下(Click)。

4.取得搜尋結果，檢查結果是否包含預期的內容。

本文件之智慧財產權屬行政院資通安全辦公室所有。



資料來源：本指引整理

圖16 Selenium 範例

3.3.2. Geb

Geb 是一種網頁測試的自動化測試框架，它是 Groovy 用於 Web Test 自動化測試的特定領域語言(Domain-Specific Language, DSL)，它讓開發者使用 Groovy 撰寫簡單的 Script 程式來操作瀏覽器，完成網站各項畫面或表單的模擬操作。Geb 內部使用了 Selenium WebDriver 瀏覽器自動化測試引擎，提供類似 jQuery Selector 的 DOM 操作方法，可取得畫面的內容加以驗證，還能夠將網頁測試結果及畫面截圖保存。

Geb 是從 Browser（Client）端的角度進行測試，所以跟 Server-side 的平台完全無關。不管網站用什麼平台開發，Server-side 是 Java EE、.NET 或

本文件之智慧財產權屬行政院資通安全辦公室所有。

PHP 等，只要網站可以在瀏覽器操作，就可以用 Geb 的程式進行自動化測試。由於 Geb 需要 Java / Groovy 的執行環境，故要執行 Geb 程式前必須先裝好 JDK 7 以上與 Groovy 2.3 以上的版本，並建議使用 Firefox 33 以上版本之瀏覽器。

Geb 程式範例詳見圖 17，此範例以 Geb 的 DSL 語法撰寫，是自動化操作 Google 搜尋的完整範例。這段程式碼包含幾個常見的 Geb 語法使用範例：

●value( ) 將資料填入表單

●text( ) 取得 Element 包含的文字內容

●click( ) 模擬滑鼠點擊 Element 的事件

●waitFor{ } 等待一個條件被滿足

```
@Grab('org.gebish:geb-core:0.10.0')
@Grab('org.seleniumhq.selenium:selenium-firefox-driver:2.44.0')
import geb.Browser

Browser.drive {
    go 'https://www.google.com.tw/'

    waitFor { title.startsWith('Google') }

    $('input', name: 'q').value('CodeData')
    $('input', name: 'btnI').click()

    waitFor { title.endsWith('CodeData') }

    println $('div.article h3 a').text()
}
```

資料來源：[22]

圖17 Geb 程式範例

3.3.3. Sikuli

本文件之智慧財產權屬行政院資通安全辦公室所有。

Sikuli 是一種透過圖片的辨識來進行自動化的介面測試,它可以用來自動化的在圖形使用者界面（Graphical User Interface, GUI）上進行操作和測試，而 Sikuli 腳本就是實現這一過程的一種腳本語言。Sikuli 是通過截圖來識別需要操作的圖形界面組件，再通過 Jython 來完成操作動作從而實現自動化的操作，只要先將要做動作的圖象, 用 screen capture 給剪下來, 在執行腳本時進行比對, 若有一定程度的相似性,就作 script 中指定的動作，一般的瀏覽器錄製工具是單純地將使用者透過瀏覽器對網頁的操作步驟錄製下來,但 Sikuli 卻是可以將使用者所見的作業系統所呈現之圖形介面的畫面都記錄下來，例如點選電腦桌面上的某個圖標、輸入一段文數字，或是按一下按鈕等，所以不限於針對網頁的操作，亦可以用於執行某個應用程式、開啟或關閉檔案、測試資料的輸入等。而 Sikuli 安裝的方式也相當容易，只要安裝 Java 後即可安裝 Sikuli 的 IDE 進行操作步驟的錄製。Sikuli 的腳本範例詳見圖 18。此範例為使用者登入 gmail 的操作步驟，包含點選 Windows 的開始程式集按鈕，並啟動 Firefox 瀏覽器，輸入帳號及密碼後登入 gmail。



資料來源：[24]

本文件之智慧財產權屬行政院資通安全辦公室所有。

圖18 Sikuli 腳本範例

3.4.結語

本章節針對安全需求驗證活動實務進行介紹，包含說明制定安全需求實應該注意的原則，以及介紹安全需求的類型與萃取方式，並介紹 OWASP 應用程式安全驗證標準(OWASP ASVS)以供決定驗證範圍及項目時的參考，最後說明於軟體開展生命週期內進行安全需求驗證的方式,包含需求階段、設計階段、開發階段、測試階段、驗收階段，以及軟體部署與維運階段等。最後針對常見的網頁自動化測試進行介紹，以提高測試效率。

本文件之智慧財產權屬行政院資通安全辦公室所有。

4. 安全性測試技術

軟體「安全性測試」是有關驗證應用程式識別潛在的安全性瑕疵，一般可分為「源碼靜態分析」以及「動態測試」。「源碼靜態分析」可以在不執行軟體的狀況下，針對軟體的源碼內容進行分析，比對已知弱點模式，找出可能的缺陷或錯誤。「動態測試」則是將軟體在實際的環境中執行起來並進行分析的活動。實務上，靜態分析通常採用「源碼檢測」，動態分析則通常採用外部的「滲透測試」或「弱點掃描」進行，而弱點掃描即為滲透測試的階段步驟之一。

源碼靜態分析由於可以直接針對源碼進行檢視，故可被視為是「白箱測試」的一種，「動態測試」則是對執行的應用程式進行測試，故被視為「黑箱測試」。另外則有「灰箱安全性測試」的概念被提出，其針對黑箱及白箱的檢測結果進行交叉分析，目的在找出真正高風險的問題。

綜觀不同的檢測技術，有其成本考量與應用之適當時機。使用各種工具與技術並無排他性，而是相輔相成，例如：利用自動源碼檢測能在開發階段及早發現源碼的弱點，可有效避免該弱點造成後續災害，在開發後期的測試建置階段則可再進行弱點掃描或滲透測試，皆可針對常見 Web 應用程式伺服器之設定缺失與已知弱點來進行掃描，這方面解決方案，尤其對於發現非程式碼撰寫缺失所造成之弱點非常有幫助。

以下各節討論源碼靜態分析、滲透測試以及灰箱安全性測試的手法。

4.1.源碼靜態分析

所有的軟體開發組織，無論是使用哪一種程式語言、開發方法或產品類別如何，都有一個共通點：他們都有源碼。而且大部分的安全漏洞發生的根本原因都是因為不安全的源碼設計而產生的。大多數應用程式安全性的弱點並不是在安全性相關源碼中(例如加密函式)，反而是在與安全性無關且撰

本文件之智慧財產權屬行政院資通安全辦公室所有。

寫時也未注意到安全性的源碼中，這就是開發者為什麼必須深切了解應用程式安全性的原因。

源碼靜態分析即是在軟體開發階段進行，以期能及早發現安全性漏洞並進行修補，當然，源碼靜態分析的益處並不是僅被動地找出源碼內的問題，還可以在分析過程中讓程式設計人員了解為何是不安全的程式寫法，避免相同弱點再度發生在其他源碼中。

源碼靜態分析一般可分為以人工進行源碼審查(Code Review)以及採用自動化工具進行檢測。以下小節分別針對源碼審查以及自動化工具檢測進行討論。

4.1.1. 源碼安全審查

許多安全問題發生的原因是由於撰寫源碼時簡單的錯誤造成的，這種錯誤雖然有機會於動態測試時被發現，但使用源碼審查更有可能於短時間內就發現這種問題。

源碼安全審查是對源碼進行稽核的流程，透過安全審查以判斷安全需求是否已經符合而可進行發行或者是還需要更多的更改或測試。審查的目的在於找出源碼內的安全問題，並討論正確的程式設計方法。透過審查，可驗證正確的安全控制措施是否已被成功實作，並檢視軟體開發者是否有遵循組織所訂定的源碼撰寫規範，檢查是否採用安全的實作方法以及安全的函式，以避免常見的安全弱點。另外亦可以找出軟體中除錯用途之源碼、維護用後門與測試帳號等安全危害。

源碼審查須具備深入的專業知識,檢測的成效取決於審查者當時的專注力、情緒及功力，有經驗的審查者可以快速的抓出程式內不良的設計方式並給予正確的修改建議，而為提高效率往往參與審查的人數不需太多，所以源碼審查常見的進行方式是由一位較有程式設計經驗的設計師搭配程式原始

本文件之智慧財產權屬行政院資通安全辦公室所有。

開發者一起進行審查。為提高審查效率，進行時應界定審查範圍，並準備相關的源碼與軟體設計規格文件或組織內部訂立的源碼撰寫標準。

審查除了是針對安全進行稽核外，亦應發揮積極的作用，讓參與者在審查過程中學習到正確安全的程式設計方式而有所進步，如此才能提高審查的價值。假若審查時只是一味進行批評或是審查效率太差，容易讓程式設計師覺得沮喪而士氣低落，最終審查將淪為形式。此外，透過審查的機制也可讓開發者使用更安全謹慎的設計方式進行開發，提高源碼的品質。

有些開發平台也內建了自動化的源碼審查機制，版本控制系統管理者可制定的管理政策，例如源碼簽入原則(Check-in Policy)，強制要求開發者在提交源碼時必須先通過審查程序後才能讓源碼進到版本控制系統的檔案庫內，例如微軟所推出的 Visual Studio 系列即可搭配的 Team Foundation Server(TFS)進行源碼管理。使用此方式可以確保源碼品質是經過管理的，以此方式進行安全審查可避免不安全的源碼進入版本控制系統內。

4.1.1.1. 關鍵字搜尋

以人工進行源碼審查時常用的技術之一為利用關鍵字來進行搜尋，以針對特殊有興趣的議題進行審查,例如可以使用 TODO、FIX、FIXME、ASSUME、BUG、TESTING ONLY 或自訂的關鍵字，這些關鍵字由開發人員寫於註解中以作為備忘或警示之用途，在源碼檢視時即可針對這些具有審查價值的源碼進行審查。

使用關鍵字搜尋法可搭配字串搜尋工具使用，例如 Windows 作業系統可於命令提示字元使用內建的 findstr 指令，其功能為搜尋檔案內的字串，常用的選項詳見表 6：

本文件之智慧財產權屬行政院資通安全辦公室所有。

表6　findstr 指令說明

選項	用途
/S	搜尋時也一併搜尋子目錄
/I	搜尋時忽略大小寫
/N	印出符合字串的行號
/R	使用正規表示法進行搜尋
/C:string	以整個字串為一單位進行搜尋
/G:file	從一個檔案內取得搜尋字串
/D:dir	以分號區隔搜尋目錄清單

資料來源：本指引整理

使用範例如下：

findstr　/S　/I　/N　/C:”TESTING　ONLY”　/D:. *

則將會搜尋當前目錄內的所有檔案(/D: . *選項)，並遞迴搜尋所有子目錄(/S 選項)，並忽略大小寫的差異(/I 選項)，印出符合字串的行號(/N 選項)，且會以完整的字串”TESTING　ONLY”進行搜尋而非分成兩個字串進行搜尋(/C 選項)。

Unix 或 Linux 作業系統則可使用 grep 命令，詳細的操作選項可以於終端機輸入 man grep 檢視說明文件。

grep 常用的選項詳見表 7：

本文件之智慧財產權屬行政院資通安全辦公室所有。

表7　grep 指令說明

選項	用途
-r	搜尋時也一併搜尋子目錄
-i	搜尋時忽略大小寫
-n	印出符合字串的行號
-e PATTERN	使用正規表示法進行搜尋
-f FILE	從一個檔案內取得搜尋字串

資料來源：本指引整理

使用範例如下：

grep -r -i -n “TESTING ONLY” src/

將會搜尋 src 目錄內的所有檔案，並遞迴搜尋所有子目錄(-r 選項)，並忽略大小寫的差異(-i 選項)，印出符合字串的行號(-n 選項)，且會以完整的字串”TESTING　ONLY”進行搜尋而非分成兩個字串進行搜尋。

然而關鍵字搜尋只是一種陽春的方法，其缺點為若關鍵字使用不適當就很容易造成搜尋結果數量過多而發散，檢視時較無效率。舉例而言，使用”bug”當作搜尋關鍵字除了可能搜尋到”//This may be a bug.”這種註解外，也可能搜尋到真實源碼例如”log.debug”，對於數量龐大的搜尋結果要每項都進行檢視則相當耗時費力。若要減輕這種現象則必須依賴組織建立良好的源碼撰寫規範，例如可約定於註解時使用特定的格式(例如全為大寫)或特殊字串(例如”FIXME”)，並要求程式設計師必須確實遵守，有錯誤風險或將來可能

本文件之智慧財產權屬行政院資通安全辦公室所有。

需要修正的部分應做必要的標示，如此才能真實顯示出源碼內待處理的議題。

4.1.1.2. 查檢表

發展並採用查檢表，可以確保源碼審查的涵蓋面以及具有一致性的標準。針對安全性議題建議應包含的範圍如下：

- 資料驗證(Data Validation)。
- 身分認證(Authentication)。
- 會話管理(Session Management)。
- 授權(Authorization)。
- 密碼學(Cryptography)。
- 錯誤處理(Error Handling)。
- 稽核日誌(Logging)。
- 安全組態(Security Configuration)。
- 網路架構(Network Architecture)。

4.1.2. 自動化工具檢測

源碼審查較需耗費人力與時間，而且不容易做到全面的檢查，因此自動化工具應運而生，可於短時間內對軟體專案進行全面性的檢測，快速地協助開發人員找出源碼中可能的弱點，並針對弱點進行分級與提供修補建議。源碼安全檢測工具是基於已知規則(pattern)分析源碼，以尋找安全缺陷，最後列出與軟體漏洞模式相符的源碼以供開發人員進行審核。

檢測工具雖然不是萬能，但仍在安全測試中發揮重要的作用，目前已經有

本文件之智慧財產權屬行政院資通安全辦公室所有。

一系列免費或開放源碼的源碼安全檢測工具可以使用，許多軟體廠商也開始重視源碼安全檢測而推出不少產品，可以協助安全檢測工作的進行。工具的總類繁多，要如何挑選出最符合組織需求的檢測工具也變成是一個難題，目前一些資安組織也制定了可參考的標準與規範作為選擇時的參考，例如 WASC (Web Application Security Consortium) 於 2009 年提出了網頁應用程式安全掃描評估準則 (Web Application Security Scanner Evaluation Criteria, WASSEC)[27]，其列出選擇工具所該考量的 8 個面向(categories)，分別為協定支援程度、驗證、會話管理、網站爬行、網頁剖析、測試、命令與控制，以及報告，目的就在讓使用者自行依照重要性的不同進行評估。而 NIST 組織所執行之 NIST 軟體確保指標與工具評估計畫(Software Assurance Metrics and Tool Evaluation，SAMATE)亦制定出 NIST SP 500-269 網頁應用程式安全掃描功能規範[29]，提供組織在採購網頁應用程式安全檢測工具時有一個參考依據，其定義出工具至少應具備的基本檢測能力，例如工具是否能檢測出 OWASP Top 10 或 WASC 所列出的安全威脅，此評比主要是依據功能性的考量，故軟體的易用性或採購成本則不在評比考量內。以下小節針對選擇工具時應考量的要點以及一些免費或商業化的工具進行介紹。

4.1.2.1. 工具選擇要點

不同的工具有其不同的強項與不足之處，選擇源碼安全檢測工具應考量以下議題：

4.1.2.1.1. 工具檢測的能力

此部分討論工具能否給予準確的檢測結果，包含：

- 準確的識別屬於哪一種弱點

這些弱點與被分析的程式的類型有關。舉例來說，應用程式常見的資安弱

本文件之智慧財產權屬行政院資通安全辦公室所有。

點是 SQL 注入、跨站腳本攻擊(XSS)，以及訪問權限控制問題等。

●可將具資安弱點的源碼片段明確標示出來，以供開發人員進行源碼檢視。

●針對不同的弱點類型依嚴重性給予風險等級評估。

●針對檢測出的弱點類型說明其原因。

●針對檢測出的弱點類型給予改善建議。

●檢測結果必須精準，而非盲目地產生大量的誤判(false positives)。

●檢測標準應一致。

對源碼進行重新檢測時，應對同樣的問題給出同樣的名稱，如此才能在問題出現和消除的時候有能力進行跟蹤。

4.1.2.1.2. 部署支援性

●部署模式

採用套裝軟體安裝於個人電腦環境，或是採用軟體即服務(Software as a Service，SaaS)方式遠端提供服務，例如中華電信 SCOD[30]服務。

●安裝支援

檢測工具應提供安裝手冊與操作手冊。而若為 SaaS 方式則應提供服務使用說明、預估的檢測時間，以及保護受測物安全及報告機密性的措施。

●部署架構

應提供下列資訊：

– 部署架構採用伺服器端或客戶端模式。將影響是否需要特殊的權限或需要採買其他硬體設備。

本文件之智慧財產權屬行政院資通安全辦公室所有。

– 同時間可平行檢測的最大能力及是否具備擴充能力。

●執行環境

採用何種環境進行分析，使用編譯後的二進位檔或編譯前的源碼作為檢測基礎。

若使用編譯後二進位檔的方式，需要使用軟體依賴的外部函式庫，採用編譯前源碼方式則不需要。

4.1.2.1.3. 技術支援性

●程式語言

檢測工具所支援的程式語言種類及版本，組織應確保能夠符合本身使用的所有程式語言。

●開發環境

若軟體的開發採用軟體框架或外部函式庫，則當軟體框架或外部函式庫產生安全問題，該軟體便具有該問題。因此，具備識別軟體框架、外部函式庫的能力相當重要，包含以下項目：

– 了解軟體框架如何運作，以及軟體如何與其互動、資料傳輸的能力。

– 識別軟體以不安全的方式使用框架的能力。

– 識別軟體框架中已知安全問題的能力。

●設定檔的支援

部分安全弱點來自於不安全的設定。因此，應了解工具對於設定檔的支援程度，以及找出不安全設定的能力。

4.1.2.1.4. 可檢測的弱點類型

本文件之智慧財產權屬行政院資通安全辦公室所有。

工具需能夠了解、檢測並回報源碼中的常見弱點，以下列出相關類型(依字典順序排序)，詳見表 8：

表8　常用的檢測弱點

編號	威脅項目
1	誤用 API(API Abuse)
2	應用程式設定不正確(Application Misconfiguration)
3	未關閉密碼自動完成(Auto-complete Not Disabled on Password Parameters)
4	緩衝區溢位(Buffer Overflow)
5	命令注入(Command Injection)
6	憑據/會話 ID 預測(Credential/Session Prediction)
7	跨站腳本攻擊(Cross-site Scripting)
8	拒絕存取服務(Denial of Service)
9	權限提升(Escalation of Privileges)
10	不安全的密碼學應用(Insecure Cryptography)
11	字串格式(Format String)
12	憑據以明文寫於源碼中(Hardcoded Credentials)
13	HTTP 回應分割(HTTP Response Splitting)
14	不正確的輸入處理(Improper Input Handling)
15	不正確的輸出編碼(Improper Output Encoding)
16	資訊洩漏(Information Leakage)
17	不安全的資料快取(Insecure Data Caching)
18	不安全的檔案上傳(Insecure File Upload)

本文件之智慧財產權屬行政院資通安全辦公室所有。

編號	威脅項目
19	帳號鎖定機制不足(Insufficient Account Lockout)
20	認證機制不足(Insufficient Authentication)
21	授權機制不足(Insufficient Authorization)
22	日誌機制不足(Insufficient/Insecure Logging)
23	密碼複雜度不足(Insufficient Password Complexity Requirements)
24	密碼週期要求不足(Insufficient Password History Requirements)
25	會話過期機制不足(Insufficient Session Expiration)
26	整數溢位(Integer Overflows)
27	目錄服務命令注入(LDAP Injection)
28	電子郵件命令注入(Mail Command Injection)
29	Null Byte 注入(Null Byte Injection)
30	開放重導攻擊(Open Redirect Attacks)
31	作業系統命令注入(OS Command Injection)
32	造訪路徑攻擊(Path Traversal)
33	競賽情況(Race Conditions)
34	遠端檔案攻擊(Remote File Inclusion)
35	置換會話(Session Fixation)
36	SQL 命令注入(SQL Injection)
37	誤用 URL 導向(URL Redirection Abuse)
38	XML 命令注入(XML Injection Attacks)
39	XML 路徑注入(XPATH Injection)

資料來源：本指引整理

本文件之智慧財產權屬行政院資通安全辦公室所有。

4.1.2.1.5. 命令與控制的支援性

此部分的支援性將影響使用者設定、客製化、將工具整合至組織軟體開發流程內的能力，另外也影響找出問題及修復問題的效率。

●檢測設定

包含以下項目：

– 檢測排程能力。

– 檢視執行排程即時狀態能力。

– 將設定儲存為模板並重複使用的能力。

– 同時間執行多排程的能力。

– 支援多使用者的能力。

– 支援漸進式掃描的能力。

●客製化能力

– 增加/刪除/修改檢測樣式。靜態源碼分析工具常有誤判的問題，因此修改檢測樣式的功能可以幫助消除誤判，或將該筆誤判標示為安全。

– 套裝規則。提供套裝規則並可以選擇其中部分來執行檢測。

– 可選擇的檢測。具備針對特定問題重新掃描檢測的能力。

– 單元測試。具備單元測試能力以驗證新的規則。

– 教育訓練。撰寫新的規則是否需要額外的教育訓練。

●命令列支援

使用者是否可採用命令列方式啟動並執行檢測作業。

本文件之智慧財產權屬行政院資通安全辦公室所有。

●整合式開發環境

工具是否支援組織使用的整合式開發環境(Integrated Development Environment, IDE)，可以從整合式開發環境中對軟體專案進行檢測，此方式將有助於開發人員於實作階段便進行檢測排除問題。

4.1.2.1.6. 規則

檢測的規則(Pattern)是得以識別安全弱點的原因，故為求檢測結果確實有效必須考量以下幾點：

●規則更新的頻率。採用週期性、根據客戶要求或者其他方式。

●規則的有效性。說明如何更新規則以應付不斷演變的最新威脅。

●是否允許使用者自訂規則。

4.1.2.1.7. 檢測報告

檢測工具的報告是對於利害關係人最視覺化的功能呈現。報告呈現方式也應因應不同的報告檢視者的需求而提供選擇。例如，給開發者的報告需要較多的詳細技術資訊以協助修補弱點，給高階主管的報告則須著重於高階的摘要及相對應的風險。可對以下項目進行考量：

●報告呈現方式的支援，應包含提供：

– 執行摘要。

– 弱點所屬類型。

– 弱點發現位置。

– 根據單一問題提出的「解決方案」。

– 範例源碼。

本文件之智慧財產權屬行政院資通安全辦公室所有。

●報告格式支援。

列舉支援的報告產生格式。

●報告客製化能力。

將稽核者發現註記在報告中的能力、將誤判註記並從報告中移除的能力，以及修改報告樣板增加組織標誌、標頭文字等資訊的能力。

4.1.2.1.8. 組織整合支援性

檢測工具與組織現有系統進行整合的支援度

●整合持續整合(Continuous integration, CI)環境

有不少開發組織會建置持續整合環境例如 Jenkins、Hudson 或自行開發的持續整合平台，檢測工具是否能與現有環境密切整合。

●整合問題管理系統(Issues Tracking System)

檢測工具整合問題管理系統將有助於軟體專案後續控管安全問題的追蹤處理。

●整合風險管理系統(Risk Management System)

考量檢測結果如何影響及如何整合至組織的風險管理系統。

●整合軟體專案的能力

將組織內採用相同程式語言或相同技術、軟體框架的專案，透過整合的方式呈現更多資訊，例如同類的軟體專案是否發現比其他類型更多問題?或者問題集中於某些弱點類型。

●軟體授權方式

不同授權方式對於組織總體成本的利弊。

本文件之智慧財產權屬行政院資通安全辦公室所有。

4.1.2.2. 免費工具

目前已有許多免費或開放源碼的可以讓開發者利用，可針對不同的程式語言進行靜態分析，減少程式內的錯誤。以下只挑選幾個工具進行簡單介紹，包含 FindBugs [31]、CAT.NET[32]、Cppcheck[33]以及 PMD[34]等免費工具。

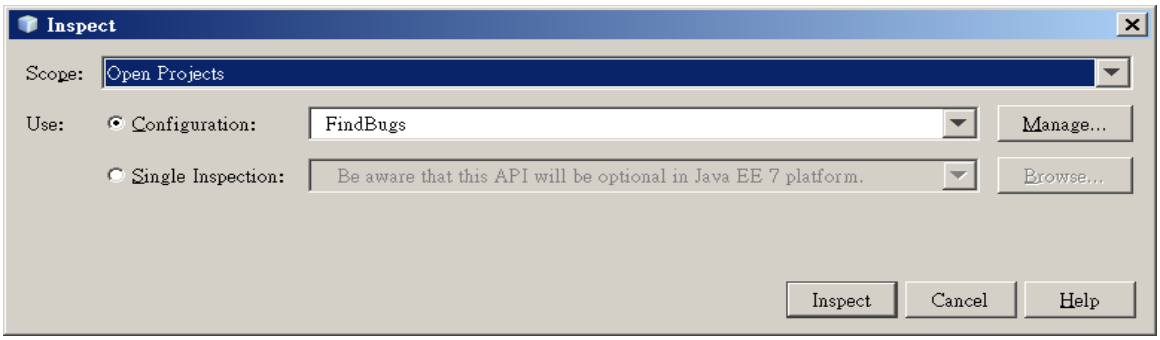
4.1.2.2.1. FindBugs

FindBugs 是一個頗受好評的輕量級檢查工具，可找出 Java 源碼的缺陷，然而並非只專注於安全議題。FindBugs 只需要在系統上安裝 JRE (or JDK) 1.7.0 以上版本即可執行，其支援的執行方式包含下列幾種：

- 使用 FindBugs 執行檔，可用於命令列模式或圖形化介面。
- 以外掛形式使用，支援：Eclipse[35]、Maven[36]、Netbeans[37]、Jenkins [38]、Hudson[39]，以及 IntelliJ [40]。

以下利用 Netbeans 外掛方式使用 FindBugs 進行介紹，NetBeans 及 FindBugs 的安裝方式請參考 Netbeans 說明文件，在此不多做描述。

建立好 Netbeans 的 Java 程式專案以後，於 Netbeans 工具列點選 Source > Inspect，並於 Configuration 選項選擇 FindBugs，詳見圖 19。



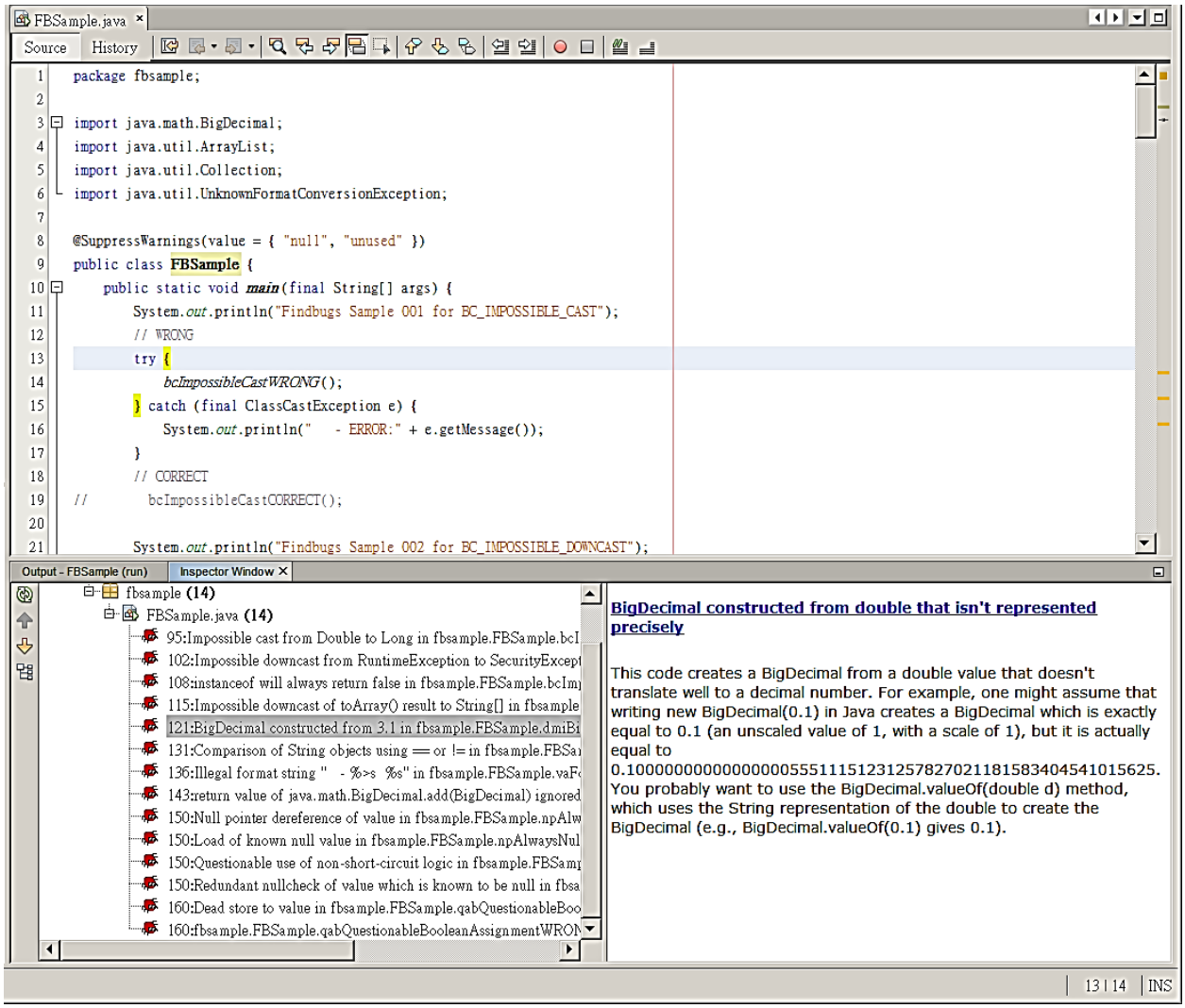
資料來源：本指引整理

圖19 Netbeans Inspect

本文件之智慧財產權屬行政院資通安全辦公室所有。

選擇 Inspect 按鈕，及開始進行 FindBugs 分析。

分析之結果會顯示問題之類型以及問題所在之源碼，並且針對問題的類型進行說明。FindBugs 檢測畫面詳見圖 20。



資料來源：本指引整理

圖20 FindBugs 檢測畫面

以下利用有問題的源碼為範例，介紹 FindBugs 所檢測出的安全問題以及說明，詳見表 9。

本文件之智慧財產權屬行政院資通安全辦公室所有。

表9　FindBugs 檢測範例

Package 名稱	fbsample
Class 名稱	FBSample

範例 Java 源碼	private static void dmiBigDecimalConstructedFromDoubleWRONG() {
	final BigDecimal bigDecimal = new BigDecimal(3.1);
	System.out.println(" - " + bigDecimal.toString());
	}

Findbugs 訊息	121:BigDicimal constructed from 3.1 in fbsample.FBSample.dmiBigDecimalConstructedFromDoubleWRONG( )
問題類型	BigDecimal constructed from double that isn't represented precisely
問題說明及建議	This code creates a BigDecimal from a double value that doesn't translate well to a decimal number. For example, one might assume that writing new BigDecimal(0.1) in Java creates a BigDecimal which is exactly equal to 0.1 (an unscaled value of 1, with a scale of 1), but it is actually equal to 0.1000000000000000055511151231257827021181583404541015625. You probably want to use the BigDecimal.valueOf(double d) method, which uses the String representation of the double to create the BigDecimal (e.g., BigDecimal.valueOf(0.1) gives 0.1).

資料來源：本指引整理

4.1.2.2.2. Microsoft Code Analysis Tool .NET (CAT.NET)

Microsoft Code Analysis Tool .NET (CAT.NET) 是一套靜態源碼安全檢測工具，支援.NET、C#、Visual Basic .NET、J#等程式語言，可以直接分析二位

本文件之智慧財產權屬行政院資通安全辦公室所有。

元的 .NET 組件，並協助找出在組件中潛在的安全性漏洞，例如：

●跨站腳本攻擊。

●SQL 注入。

●重新導向至使用者控制的站台。

●檔案規範化(File Canonicalization)。

●透過例外造成資訊暴露。

●程序命令執行。

●XPATH 注入。

●LDAP 注入。

其檢測規則都是使用 XML 定義，故很容易擴充。CAT.NET 可於 Windows 作業系統環境上使用，其執行方式包含下列幾種：

●搭配 Visual Studio IDE 使用。

●命令列。

●作為 FxCop 規則 [41]。

●整合進 Visual Studio Team Fundation TeamBuild。

4.1.2.2.3. Cppcheck

Cppcheck 是針對 C/C++所開發的靜態分析工具，主要能發現程式內常被編譯器所遺漏的潛在錯誤，例如記憶體洩漏(Memory leaks)、Standard Template Library(STL)的錯誤使用等。

Cppcheck 可於 Windows 及 Linux 作業系統環境上使用，其執行方式包含下

本文件之智慧財產權屬行政院資通安全辦公室所有。

列幾種：

●使用 Cppcheck 執行檔，可用於命令列模式或圖形化介面。

●以外掛形式使用，支援 Eclipse[42]、gedit[43]、Hudson[44]、Jenkins[45]、Mercurial (Linux) [46]、Tortoise SVN[47]以及 Visual Studio[48]。

4.1.2.2.4. PMD

PMD 是一套源碼靜態分析工具，可找出常見的程式缺陷，例如沒有使用到的變數、空白的 try-catch 程式區塊，以及非必要的物件建立等。其支援 Java、JavaScript、XML，以及 XSL。

PMD 其執行方式包含下列幾種：

●使用 PMD 執行檔，可用於命令列模式或圖形化介面。

●以外掛形式使用，支援 Maven[49]、Netbeans[50]、Ecplise[51]、JBuilder[52]、JDeveloper[53]，以及 IntelliJ[54]。

4.1.2.3. 商業化工具

有許多靜態分析工具廠商注重在發現源碼內的安全問題，例如以下列出數個相關產品(以字母順序顯示)：Armorize [55]、Checkmarx [56]、Coverity [57]、HP Fortify SCA [58]、Klocwork [59]、IBM AppScan Source [60]、Veracode [61]，以及 WhiteHat [62]等。由於靜態分析工具商業化產品總類繁多，以下僅針對 HP Fortify SCA、IBM AppScan Source 以及 Checkmarx 當作範例進行簡要介紹。

4.1.2.3.1. HP Fortify Static Code Analyzer (SCA)

Fortify Static Code Analyzer 是惠普(Hewlett-Packard)企業安全產品解決方案的一部分，功用為幫助開發人員和安全團隊在整個開發生命週期中找到並

本文件之智慧財產權屬行政院資通安全辦公室所有。

修復安全弱點。它採用多種安全源碼撰寫規則對源碼進行分析，能偵測超過 500 種類型的弱點，及識別安全性弱點的根本原因與源碼位置，對如何修改源碼提供了指導及建議，並依照安全嚴重性判定其優先等級，讓開發者能利用其所提供的資訊對源碼進行修補。

HP Fortify SCA 可支援平台包含:Windows、Linux、Mac OS X、Oracle Solaris、HP-UX，以及 AIX。

HP Fortify SCA 支援多種程式語言，包含：

●傳統語言：C/C++、COBOL。

●Java 系列：Java、JSP。

●Microsoft 系列：ASP、VBScript、VB6、ASP.Net、VB.Net、C#.Net。

●Apple Mac 系列：Objective-C、Action Script。

●Web：HTML、JavaScript/AJAX、PHP、XML、ColdFusion 5.0、Python。

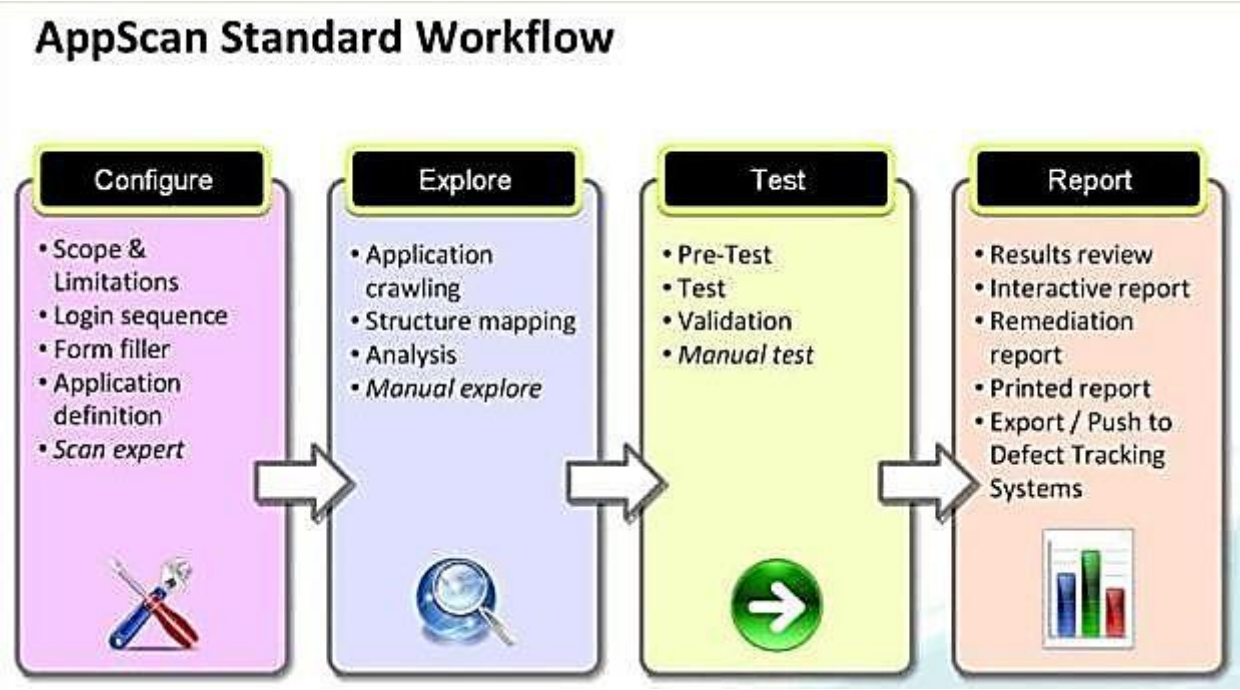
●資料庫：PL/SQL(Oracle 資料庫)、T-SQL(MS-SQL 資料庫)。

●SAP 系列：ABAP/BSP。

4.1.2.3.2. IBM Security AppScan

IBM Security AppScan 產品系列是 IBM 安全框架解決方案中應用安全的一個重要部分，可以自動化應用程式安全性測試，包括實現對 Web 應用安全性漏洞的動態掃描、源碼靜態分析，以及針對已上線的系統做 Web 安全攻擊測試，其能掃描及測試多種應用程式安全性漏洞，並產生報告提供優先補救建議及修正建議，以達成補救工作，減輕相關的安全性風險。IBM Security AppScan 的運作流程詳見圖 21。

本文件之智慧財產權屬行政院資通安全辦公室所有。



資料來源：[60]

圖21 IBM Security AppScan Workflow

4.1.2.3.3. Checkmarx 源碼安全檢測工具

Checkmarx 是一家成立於 2006 年的以色列公司，其推出的源碼安全檢測工具，專門為識別、追蹤和修復軟體源碼技術上和邏輯方面的安全漏洞而設計。以簡單操作介面為其主要特色，使用虛擬編譯器(Virtual Compiler)的專利技術可掃描未編譯的程式碼(uncompiled code)，並提供中文報表及中文介面，提供問題說明及修復建議，方便測試人員進行工具操作及分析結果。

Checkmarx 支援多數主要語言包含 Java, C# / .NET, PHP, C, C++, Visual Basic 6.0, VB.NET, APEX, Ruby, Javascript, ASP, Perl, Mobile(Android, Objective C, Windows Phone)等。

4.2.滲透測試

滲透測試是評估資訊系統安全性的一種方法，其從一個惡意攻擊者的角度

本文件之智慧財產權屬行政院資通安全辦公室所有。

來評估目標系統，同時利用技術和非技術的漏洞，以確定對業務可能造成的影響。常見的進行方式是由一群具有專業與豐富的「網路及系統資訊安全」相關知識與經驗的團隊人員負責執行測試，在具備高道德標準與完善的作業準備下，進行檢測作業。在取得受測者合法授權後，以攻擊者的思維角度針對已經上線服務的系統發動攻擊，以自動化掃描工具、攻擊程式及技術等輔助，對目標進行安全探測，其目的並不是要破壞系統，而是要找出資安防護機制存在的弱點或安全問題。使接受滲透測試的單位，能了解所面臨的資安問題，並於檢測作業完畢後提供完整的評估報告及修補建議。在測試的過程中也會盡可能遵循一致的公認方法、開發工具及技術的系統測試，驗證技術和非技術漏洞，以達到可重複、可量測評估的最終目標。

滲透測試執行的重點如下：

●依賴人員的技術能力與經驗。

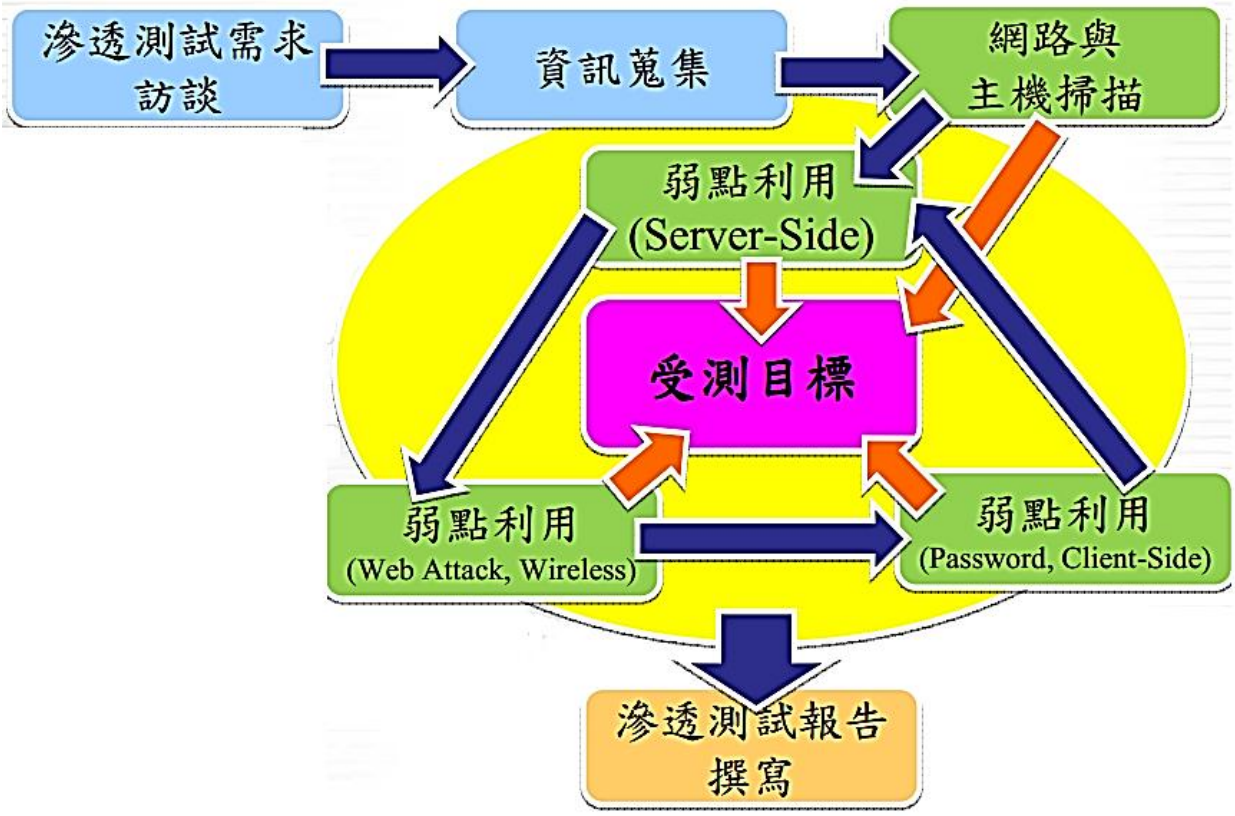
●取決於檢測人員的思維與創意。

●需透過人工的方式與工具的輔助。

●發覺軟體或系統上的已知或未知的漏洞。

滲透測試的基本流程，詳見圖 22。

本文件之智慧財產權屬行政院資通安全辦公室所有。



資料來源：本指引整理

圖22 滲透測試的基本流程

滲透測試的主要階段為：「需求訪談」、「資訊蒐集」、「網路與主機掃描(弱點掃描)」、「弱點利用」、「滲透測試報告撰寫」。以下分別就各階段內部活動進行說明。

值得注意的是弱點掃描(Vulnerability Assessment, VA)與滲透測試不能畫上等號，弱點掃描僅使用自動化工具針對目標進行掃描，無法進行弱點利用，也無法使用 Dos 或 Buffer Overflow 等方式進行檢測，其僅為滲透測試的檢測步驟之一，滲透測試則可依需求採用更多樣的駭客攻擊手法。

4.2.1. 需求訪談

滲透測試需求訪談的主要目的如下

本文件之智慧財產權屬行政院資通安全辦公室所有。

●討論並確認測試範圍

在執行滲透測試前為避免測試範圍變動，需確認滲透測試主要目標為何，明確定義測試範圍，什麼可以被測試，什麼不能被測試，可能測試目標包含特定網域、IP 區段、單一主機、特定應用程式等。透過限制執行範圍目的在於保護系統內有價值性或敏感性的資料。

政府機關滲透測試服務委外服務案建議書徵求文件[63]列出了滲透測試的參考項目，詳見表 10。

表10 測試項目範例

測試類型	測試類別	測試項目
作業系統	遠端服務	至少包含遠端服務套件點測試
	本機服務	在已取得系統控制權限的條件下，可執行至少包含本機服務套件點測試
網站服務	設定管理	至少包含： ▪應用程式設定測試 ▪檔案類型處理測試 ▪網站爬行測試 ▪後端管理介面測試 ▪HTTP 協定測試
	使用者驗證	至少包含： ▪使用者帳號列舉測試 ▪測試機敏資料是否透過加密通道進行傳輸
	連線管理	至少包含： ▪Session 管理測試

本文件之智慧財產權屬行政院資通安全辦公室所有。

測試類型	測試類別	測試項目
		▪ Cookie 屬性測試 ▪ Session 資料更新測試 ▪ Session 變數傳遞測試 ▪ CSRF 測試
	使用者授權	至少包含： ▪ 目錄跨越測試 ▪ 網站授權機制測試 ▪ 權限控管機制測試
	邏輯漏洞	至少包含： ▪ 網站功能測試 ▪ 網站功能設計缺失測試 ▪ 附件上傳測試
	輸入驗證(1)	至少包含： ▪ XSS 漏洞測試 ▪ SQL Injection 測試 ▪ LDAP Injection 測試 ▪ XML Injection 測試 ▪ SSI Injection 測試 ▪ XPath Injection 測試 ▪ Code Injection 測試
	輸入驗證(2)	至少包含： ▪ XSS 漏洞測試 ▪ SQL Injection 測試 ▪ OS Commanding 測試 ▪ 偽造 HTTP 協定測試
	Web Service	至少包含：

本文件之智慧財產權屬行政院資通安全辦公室所有。

測試類型	測試類別	測試項目
		▪WSDL 測試 ▪XML 架構測試 ▪XML 內容測試 ▪XML 參數傳遞測試
	Ajax	至少包含 Ajax 點測試等項目： ▪輸入驗證缺失測試 ▪權限控管測試 ▪套件弱點測試
應用程式	電子郵件服務套件	至少包含 SMTP、POP3 及 IMAP 等常見對外郵件服務之弱點測試，如設定缺失、權限控管及套件點等測試項目
	網站服務套件	包含常見 WEB 套件的弱點測試，例如設定缺失、權限控管及套件弱點等測試項目
	檔案傳檔服務套件	至少包含下列常見檔案傳輸服務的弱點測試，例如設定缺失、權限控管及套件弱點等測試項目： ▪FTP ▪NETBIOS ▪NFS
	遠端連線服務套件	至少包含下列常見遠端連線服務之弱點，例如設定缺失、權限控管及套件弱點等測試項目： ▪SSH

本文件之智慧財產權屬行政院資通安全辦公室所有。

測試類型	測試類別	測試項目
		▪ Telnet ▪ VNC ▪ RDP
	網路服務套件	至少包含下列常見網路服務之弱點，如設定缺失、權限控管及套件弱點等測試項目： ▪ DNS ▪ PROXY ▪ SNMP
	其他	至少包含下列常見應用程式或網路套件之弱點測試項目： ▪ Firewall ▪ IDS/IPS ▪ Database ▪ LDAP ▪ SMB
密碼破解	密碼強度測試	至少包含下列常見對外服務之密碼字典檔測試： ▪ WEB ▪ FTP ▪ SSH ▪ Telnet ▪ SMTP ▪ POP3 ▪ IMAP ▪ SNMP ▪ NetBIOS

本文件之智慧財產權屬行政院資通安全辦公室所有。

測試類型	測試類別	測試項目
		▪ RDP ▪ VNC ▪ Database

資料來源：本指引整理

●確定是正式營運環境還是測試環境

若檢測目標是正式營運環境，則需考量是否會因檢測作業而影響到營運服務。若是測試環境則必須考量其檢測結果是否會與真實的環境有所差異。

●確定可利用的攻擊手法

滲透測試執行期間若需要執行具侵入性質的檢測作業應先經過確認，並於雙方議定之適當時間且具備適當應變措施與風險評估後，才進行相關檢測作業，例如能否使用阻斷式服務或是社交工程等攻擊手法，簡單介紹如下：

– 社交工程手法是建立在人與人的互信基礎之上，也是最常見的攻擊手法之一。有時滲透測試團隊可透過此一手法，例如使用釣魚(Phishing)信件誘使內部員工開啟，進而透過攻擊程式騙取帳號密碼或取得系統的控制權。

– 阻斷式服務(Denial-of-Service，DoS)也是一種常見的攻擊手法，目標在於使連接上網路的目標電腦的網路資源及系統資源耗盡，造成服務主機暫時中斷或停止服務，使它無法對正常使用者提供服務。

●確認是否允許資料下載或刪除

由於測試系統上可能存放機密性或高價值性的資料，故必須確認是否允許

本文件之智慧財產權屬行政院資通安全辦公室所有。

對資料進行下載或刪除。

另外，提供服務的系統一般會具備稽核功能，讓系統管理者可以了解系統營運狀況及是否有異常狀況發生。而在執行滲透測試後是否要保留或刪除稽核檔案資料，事先需要經過確認。

●確定執行日期與時間

明確規範檢測作業執行日期與時間，包含確認每日可以執行的時間，是否允許在營運時段執行?是否為 24 小時執行或是離峰時段才可以執行?執行的期間有多久?

●確認內部相關部門是否知悉

採用告知相關部門的方式其好處是在檢測過程中若可取得其配合，則有助於測試順利進行，例如要求提供所需資訊以減少檢測作業時間，但也必須考量若內部系統網路人員知曉滲透測試進程後，為了避免被檢測出安全弱點而暫時性的加強系統及網路的防護性，這種狀況下所得到的檢測結果未必與平時的運作情況相符。若選擇不告知相關部門則可以測試其對於發現網路攻擊與防護的相關能力，但檢測人員須考量是否會造成正常業務的危害。

●確認是透過網際網路或是組織內部網路進行測試

網際網路滲透測試係滲透測試小組直接由遠端進行，內部網路則於組織內部進行。

●制定溝通規則

雙方需具備暢通的聯絡管道，當測試主機在過程中發生問題或發現遭到入侵的足跡等事件，需要能夠即時聯絡到相關人員。

另外，需制定安全的資訊交換管道，例如測試過程的相關資訊提供，測試

本文件之智慧財產權屬行政院資通安全辦公室所有。

結果報告的提供等，皆需透過安全管道交換以確保機密性。

●定期進行溝通

設定時間定期討論執行進度，以確保雙方都能掌握目前的執行情況進度，並說明是否有發生重大的問題。

●撰寫執行計畫書

開始執行之前，將訪談階段的所有結論都寫進計畫書。受測單位主管與測試團隊主管需要簽署這份文件，必要時包含測試參與人員也需簽署。

4.2.2. 資訊蒐集

此階段已正式開始執行滲透測試。資訊蒐集是滲透測試的必要步驟。

4.2.2.1. 滲透測試分類

滲透測試進行方式依照受測方所提供的資訊多寡可分為以下三類：

●黑箱測試

黑箱測試表示受測方完全不提供任何資訊，或是僅告知受測目標的網址，其餘一切有賴測試團隊發揮其能力及創意來進行資訊蒐集。

●白箱測試

白箱測試表示由受測方提供完整系統網路的相關資訊，目的在加速檢測作業的進行，以求盡可能利用一切資訊找出系統的弱點，同時也比較小的機會讓系統遭到破壞。而由於這些資訊有其敏感性及機密性，故通常是由組織內部人員執行檢測作業時才會使用的方式。

●灰箱測試

灰箱測試是指介於黑箱與白箱之間的測試方法，由於有時受測方並不是完

本文件之智慧財產權屬行政院資通安全辦公室所有。

全清楚受測系統的資訊(例如委外開發的系統)，無法主動提供完整的受測目標的訊息，受測方僅能提供少部分資訊供測試團隊，其餘資訊仍需要測試團隊進行蒐集。

4.2.2.2. 資訊收集的方式

測試團隊收集任何可以從網際網路上找到的受測單位公開資訊，收集有關如何，何時，何地、做甚麼事的相關資訊，透過資訊蒐集的手段，識別「人」或「系統」的行為模式，以後續可以發現和利用漏洞。基本上這些公開資訊的蒐集來源可能包含：

●客戶的公開網站

網站資訊可能就提供了組織的架構圖、公司部門與負責人、組織技術人員的名稱與 Email、公司 Email 格式等。

●受測單位公開網站可下載的資訊文件

除檢視文件內容外亦可從檔案附加資訊(Metadata)取的作者帳號等訊息。

●Archive.org

網際網路檔案館，定期收錄保存全球網站訊息，以對網站的發展與歷史資料進行研究。

●公開的企業合作資訊、新聞群組

●搜尋引擎結果

使用 google、bing 等搜尋搜索直接取得公開資訊，或是透過搜尋引擎蜘蛛(Spiders)、機器人(Robots)、網路爬蟲(crawler)等工具進行自動資訊抓取。或是利用 Google Hacking 的技巧，使用 google 等搜尋引擎的進階搜尋字串功能試圖找尋網站的漏洞或資訊，例如網站設定檔，或利用鍵入內部網址、

本文件之智慧財產權屬行政院資通安全辦公室所有。

搜尋網站目錄、檔案類型等方式找到包含網站伺服器檔案、帳號、密碼、管理介面等資料。

●社群網站

利用例如臉書(Facebook)、推特(Twitter)等社群網站，尤其是以誘騙使用者加入好友、群組等方式以蒐集使用者私密的資訊。

●透過電子郵件帳號取得內部使用者帳號

利用公開的電子郵件帳號猜測該使用者在組織內部系統所使用的帳號。

●DNS

例如使用 uslookup、dig 或 host 進行查詢的結果。

●whois 查詢結果

whois 是用來查詢網域名稱是否已被註冊以及註冊域名的資料庫，例如網域名稱所有人、網域名稱註冊商、網域名稱註冊日期和過期日期等。

4.2.3. 網路與主機掃描

此階段的目標主要在於使用網路足跡追蹤的結果來探索正在進行活動的主機與服務，進行弱點識別，找出可能的弱點主機與應用程式，以增加其後漏洞利用成功的機率。透過與受測目標互動，包含採用連接埠掃描(Port Scanning)、作業系統與服務偵測、網路流量分析等活動，試圖取得受測目標的相關資訊，例如開啟的連接埠列表、使用的作業系統、開放的服務、網路應用程式名稱與版本，並利用針對這些訊息查詢是否存在已知的弱點可進一步加以利用。

其中通訊埠掃描及作業系統與服務偵測最著名的工具為 Nmap，其是一套功能強大的網路探測工具，包含了許多種不同形式的掃描以用來判斷主機的

本文件之智慧財產權屬行政院資通安全辦公室所有。

對外服務與開啟的通訊埠，例如 TCP 連線掃描(TCP Connect)、TCP 半開放掃描(TCP Half-Open 或稱 SYN Scan)、ACK 掃描、FIN 掃描及其他不同 TCP 封包標籤組合的掃描類型。

網路流量分析常用的工具有 Wireshark、Tcpdump 及 Windump 等，此目的為從單一主機或整個網段所截獲的網路封包中，分析網路攻擊或攻擊網路系統的流量。通常包含攻擊流量分析、防守流量分析、蒐集攻擊目標進一步的網路流量及探索隱蔽服務列舉等。

4.2.4. 弱點利用

弱點利用階段是滲透測試過程中實際驗證的步驟，此階段為實際利用前面階段所發現的弱點，嘗試取得軟體或系統的權限。利用弱點的目標為證明弱點可以實際造成威脅及取得權限後進行更深入的滲透測試。但由於實際利用弱點時，可能情況將造成應用程式無法回應或系統當機，故應根據這些漏洞發現規劃相對應的弱點利用情境,確認經過受測方的同意後再執行。如果有需要的話，甚至可以截取測試過程中的封包內容以供後續追蹤使用。

一般弱點利用可概略分類如下：

- 伺服器端：針對開啟網路連線的服務。
- 使用者端：針對瀏覽器、微軟 Office、PDF 檔案、Java 等。
- 權限提升：當利用弱點程式取得權限後，便需要考慮目前所獲得的權限為何，並想辦法將有限的權限或一般使用者的權限提升為管理者或最高權限以便可獲取最大的使用權來進行下一步的活動。同時可以利用已取得權限的主機來尋找下一台目標進行滲透，利用相同模式不斷擴充目標，以取得最大量的網域主機管理。

本文件之智慧財產權屬行政院資通安全辦公室所有。

4.2.5. 滲透測試報告撰寫

滲透測試最後一個階段是針對已經完成的滲透測試活動進行總整理，並總結為一個報告以受測方進行報告，測試人員依據測試過程中所顯示的安全事件進行統整，並分析其潛在風險的嚴重性，並以清晰易懂的方式呈現出來。同時在必要的情況下也可能會需要提供原始的評估資料(例如封包流量等)，作為受測方後續的分析或證據使用。

4.2.5.1. 報告要求

測試報告所呈現的資訊可能包含以下內容：

●依照每一個弱點的類型進行分類

弱點可以根據不同的標準分類以助於對弱點進行管理。這可以是統計範疇，例如 OWASP Top10 或 WASC 威脅分類。

●安全問題的嚴重程度(例如：高、適中、低、警告)

列出嚴重程度以方便制定修補計畫，擬定修復的優先權。

●找到此安全問題的測試方法

描述測試方法是提供指引給開發者如何再測試以驗證修補是否成功，並有助於分析是否為誤判亦或使用了不正確的測試方法。

●安全問題的起因(例如：源碼 bugs、不安全的加密機制等)

報告問題的起因有助於精確定位需要被修補的源碼或是組態配置。

●處理對策或建議

針對檢測的安全問題提供修補或處理的建議，資訊中可能包含安全源碼設計範例、組態設定範例，並提供充分的參考文件。

本文件之智慧財產權屬行政院資通安全辦公室所有。

4.2.5.2. 測試報告格式

測試報告的格式可參考國際標準 ISO/IEC/IEEE -29119:2013 第 3 部分所介紹的測試報告文件範本進行撰寫，建議應具備以下章節內容：

●概述

描述此份報告的緣起與歷史。

●唯一識別碼

唯一識別碼建議包含報告標題、發行日期、版本或發行狀態(如草案、已修訂、已審核、最終版)。

●發行組織

負責準備與發行該文件的組織，可包含報告撰寫者。

●核准授權

指定有責任檢視與(電子)簽核文件之人員，包括審核者與管理者。

●簡介

提供此報告內容與章節結構的說明資訊。

●範圍

定義涵蓋的範疇，並描述相關包含內容、排除內容、假設與限制。

●摘要

針對測試內容進行摘要說明，包含下列幾點：

– 說明整個專案內容，包含執行日期、目標、人員。

– 說明整個過程中是否有發現重大弱點及對組織可能的影響。

本文件之智慧財產權屬行政院資通安全辦公室所有。

– 列出較嚴重的弱點。

●簡介

針對測試內容進行簡介，包含下列幾點：

– 執行日期與時間區間。

– 執行範圍。

– 參與人員。

●執行過程

詳細說明此次測試的執行過程，包含下列幾點：

– 列出每個階段的執行活動。

– 詳細說明執行過程。

– 以表格方式呈現所發現的資訊。

●執行結果

– 呈現方式以主機、弱點風險等級區分。

– 提供詳細的技術資訊。

– 弱點修補建議。

●結論

對整個測試專案進行總結，包含：

– 說明執行範圍與時間等相關資訊。

– 受測單位的整體資安現況。

本文件之智慧財產權屬行政院資通安全辦公室所有。

– 說明執行結果，尤其高風險的弱點。

– 大方向的弱點修補方向。

●參考資料

此部分應條列參考文件，可能包含弱點分類依據例如 OWASP Top 10。

●詞彙表

提供報告內所使用的詞庫，針對術語、縮寫、首字母。

4.2.6. 滲透測試工具介紹

以下小節針對 Metasploit、Burp suite、Acunetix Web Vulnerability Scanner 以及 HP WebInspect 等滲透測試工具進行介紹。

4.2.6.1. Metasploit

Metasploit 可分為免費版本及收費版本[64]，可用於 Windows 環境及 Linux 環境，詳見表 11：

表11 Metasploit 版本

版本名稱	費用	功能
Metasploit Framework Edition	免費	包括命令行界面，擁有第三方引入, 手動攻擊和手動暴力破解等功能
Metasploit Community Edition	免費	基於收費版本，但並不包括所有付費版的功能。它的功能包括：網路發現，模塊瀏覽，和手動攻擊
Metasploit	開放核心的	使用網頁界面，擁有 Metasploit Community 的所有功能和智能攻擊、密碼審計，以及基

本文件之智慧財產權屬行政院資通安全辦公室所有。

版本名稱	費用	功能
Express	付費版本	準滲透測試報告等免費版所沒有的功能
Metasploit Pro	開放核心的付費版本	使用網頁界面，擁有 Metasploit Express 的所有功能和閉環弱點驗證，模擬釣魚式攻擊等社會工程學功能，網路應用程序測試，攻擊自動化，和其他 Express 版所沒有的功能

資料來源：本指引整理

以下針對 Metasploit Framework 架構進行說明，主要包含 6 個部分：

●使用者介面(User Interface)

Metasploit 提供 Armitage、msfconsole、 msfcli、msfpayload、msfencode、xmlrpc 等多種使用者介面，可以提供使用者輸入指令以進行 Metasploit 檢測任務。雖然有圖形化介面但因整合性仍沒有十分完善，故講師推薦最好仍使用文字介面才能進行較完整的操作。

●工具(Tools)

Metasploit 是以 Ruby 程式開發出來的，也有內建許多的輔助工具，用來支援 Metasploit 各項作業，例如密碼破解工具 lm2ntcrack.rb 即是以 ruby 寫成的小工具。pdf2xdp.rb 則可以用來將 pdf 轉成 XML Data Package 的格式。

●外掛(Plug-ins)

協助 Metasploit Framework 整合的其他輔助工具，例如與資料庫 MySQL 或 PostgreSQL 整合、監控網路連線狀態等工具都屬於外掛程式。

●模組(Modules)

此部分為 Metasploit 最主要的核心區塊，存放用來滲透與存取系統的重要

本文件之智慧財產權屬行政院資通安全辦公室所有。

指令。Metasploit 也提供了良好的擴充性，使用者可以在該平台上自行開發新的模組，主要使用的基本模組分別為：

– Exploits ：弱點的攻擊程式。

– Payloads：攻擊的內容，例如決定是否要新增使用者帳號。

– Encoders：將 payload 內容進行編碼以躲過防毒軟體的偵測。

– Auxiliary：提供多種的攻擊元件，例如阻斷式服務攻擊、模糊攻擊、掃描器、SQL Injection 攻擊及網路電話(VoIP)攻擊等。

– Post：目標攻擊成功後所提供的功能，例如，鍵盤側錄、權限提升等。

– NOPs：也就是 No Operation，作用是讓目標系統在一個時間週期內，不執行任何的處理程序以提升攻擊者執行 exploit 成功的機率。

●文件(Documentation)

Metasploit 工具的相關說明文件，包含使用者指引與開發者指引。

●資料(Data)

存放 Metasploit 會使用到的文字、圖片以及字典檔等相關資料。

4.2.6.2. Burp suite

Burp suite 是一套可以用來檢視、請求及觀看 HTTP 的傳輸內容的檢測工具[65]，其包含了一系列 burp 工具以方便進行滲透測試時使用，讓攻擊者可以結合手工和自動技術去列舉、分析、攻擊 Web 應用程序。

包含以下工具：

●Proxy(代理)：攔截並修改從使用者端到 Web 應用程式的資料。

●Spider(蜘蛛)：爬取 Web 應用程式的內容和所有聯結。

本文件之智慧財產權屬行政院資通安全辦公室所有。

●Scanner(掃描器)：掃描 Web 應用程式的漏洞。

●Intruder(入侵)：可用於多種用途，例如弱點利用、窮舉法破解等。

●Repeater(重複器)：根據不同情況修改和發送相同的請求次數並分析。

●Sequencer(定序器)：檢查 Web 應用程式提供的會話 Token 的隨機性並執行各種測試。

●Decoder(解碼器)：解碼成原來的形式，或者進行編碼和加密。

●Comparer(比較器)：執行任意的兩個請求、回應或任何其他形式的資料之間的比較。

4.2.6.3. Acunetix Web Vulnerability Scanner

Acunetix Web Vulnerability Scanner 簡稱 AWVS 或 Acunetix WVS，是一款網路漏洞自動化掃描工具[66]，目的為簡化發現與修復網頁應用程式安全性問題的工作，無需取得網站源碼即可直接針對 Web 應用程式可能遭受的攻擊例如參數注入、跨網站指令碼(XSS)、目錄遊走(Directory traversal)等進行廣泛的安全性檢測和評估。 除了有商業版本，亦有 14 天功能限制的試用版可於 Acunetix 的官方網頁下載試用。

4.2.6.4. HP WebInspect

WebInspect 是惠普(Hewlett-Packard)所推出的一套自動化弱點掃描工具[67]，針對以網頁形式提供服務的應用程式進行安全性檢測，模擬駭客攻擊的手法，無需取得網站源碼即可直接針對 Web 應用程式可能遭受的攻擊例如參數注入、跨網站指令碼、目錄遊走等進行廣泛的安全性檢測和評估，以判斷系統是否存在各種安全性問題，按照問題的輕重緩急順序，提供可立即處理問題的建議做法，最後以報表列出掃描弱點，將弱點依照嚴重程度進行分類，以利作為修復優先順序的參考，並針對每一項弱點提供詳細說明、

本文件之智慧財產權屬行政院資通安全辦公室所有。

成因及修復方式，有助於開發者提升安全知識並進行安全性漏洞修補。

WebInspect 具有以下主要特點：

- 引導式掃描功能，透過內建的引導模式設定檢測專案以簡化設定步驟。
- 證明符合法規和安全政策，預設配置多種安全性檢測設定，包括 ISO 17799、ISO 27001、OWASP Top 10 等，可檢測 web 應用程式如何滿足政府管理法規和行業標準。並允許使用者自訂策略。
- 互動式弱點審查和重新測試功能，增進使用者發現問題的能力，並應用於安全軟體發展的生命週期，讓開發者針對檢測結果進行修補與重新測試。
- 管理、查看和共用檢測報告，除內建許多標準報告範本外並允許使用者編輯或引入外部參考資料。

4.3.灰箱安全性測試

灰箱安全性測試的概念是針對黑白箱交叉分析，目的在比對白箱及黑箱的檢測結果，找出真正高風險的問題，以制定修正問題的優先順序。由於白箱測試難免會有誤判，利用檢測工具所掃描的結果不一定代表軟體就具有該項弱點，但只進行黑箱測試的困難點則是發現弱點後卻無法立即知道造成弱點的源碼所在位置，須由程式開發者進一步分析原因確認問題所在。灰箱安全性測試的軟體工具並不多，商業化的軟體工具則有 HP 所推出的 Security Scope (目前更名為 HP WebInspect Agent)[68]。

4.4.結語

本小節介紹安全性測試技術，分別針對源碼靜態分析、滲透測試，以及灰箱安全性測試進行探討。源碼靜態分析包含以人工進行之源碼審查以及使用自動化工具檢測等方式。滲透測試則包含「需求訪談」、「資訊蒐集」、「網路與主機掃描(弱點掃描)」、「弱點利用」，以及「滲透測試報告撰寫」

本文件之智慧財產權屬行政院資通安全辦公室所有。

等階段，本小節對於各階段的施行要點進行說明。最後介紹灰箱安全性測試的概念。

本文件之智慧財產權屬行政院資通安全辦公室所有。

5. 案例分析

本小節以實際開發案例說明安全需求驗證以及安全性測試的進行過程。安全需求驗證主要會以人工操作系統的方式進行，以驗證是否滿足安全性的要求。另外為了針對 WEB 常見的資安弱點進行檢測，故使用了 HP Fortify SCA 進行源碼安全性掃描，以及使用 Acunetix Web Vulnerability Scanner (AWVS)進行弱點掃描。

5.1.案例簡介

技服中心雲端網站 (以下稱本系統)係使用.NET 程式語言所開發，使用「平台即服務」(Platform as a Service，簡稱 PaaS)之雲端環境，目的為提供技服中心網站內容,並具備管理維護子系統以進行使用者權限管理與日誌稽核。技服中心雲端網站使用者身分分為一般使用者、貼文者、審核者以及系統管理員，需透過 HTTPS 協定連線至本系統，以瀏覽網頁資訊或對系統進行操作，只有系統管理員可以登入管理維護系統。

5.2.安全需求

本系統之管理維護網站具有人員資料管理功能，且保有人員相關個人資料(例如：E-mail)，應依個人資料保護法及資訊安全相關規範辦理安全性及隱私性保護，包括：

- 帳號/密碼驗證、帳號等級/群組權限管控、日誌稽核(Auditing)、來源 IP 限制等安全性控管。
- 於開發過程須針對源碼進行安全性檢測。
- 於測試過程進行弱點掃描。

5.3.安全測試計畫

此專案所進行之安全測試包含安全需求驗證、源碼安全性檢測，以及弱點

本文件之智慧財產權屬行政院資通安全辦公室所有。

掃描等 3 個活動，安全需求驗證會以人工方式進行，對網頁進行操作以驗證安全需求，而針對 Web 應用程式常見的弱點例如 OWASP Top 10 所規範的項目則計畫以源碼檢測工具以及弱點掃描工具進行自動化檢測，並針對檢測結果進行人工分析。

5.3.1. 安全需求驗證

5.3.1.1. 測試目標

針對系統的安全需求進行驗證，包括系統機密性、完整性，可用性、以及日誌管理等類別。

5.3.1.2. 測試方式

針對各個測試項目設計一至多個測試案例,並以人工操作網頁的方式進行，其中包含以本機登入以及透過遠端連線登入系統，並列出使用者的身分類型，以及他們所具有關於功能/資料的權限。然後修改使用者的身分類型，重新進行測試，試驗其權限是否符合規範。除使用不同權限的使用者帳號進行驗證外，並會使用非法的使用者帳號嘗試登入系統以及試圖對網頁進行不當的操作，以了解系統抵抗攻擊的能力。

5.3.1.3. 測試完成的條件

所有的測試項目皆通過。

5.3.1.4. 測試結果

測試結果應留下關於安全需求之檢核表，詳見表 12 所示，另除標記是否通過之外，亦建議留意實際之測試結果畫面，以作為佐證。

本文件之智慧財產權屬行政院資通安全辦公室所有。

表12 測試項目範例

驗證項目	測試結果
除了公開的頁面外，所有頁面及資源需要經過身分驗證	通過
身分驗證失敗無法登入系統	通過
所有密碼欄位當使用者輸入時不會顯示輸入值	通過
密碼必須為 8 位以上英文字母大小寫、數字或符號組合，至少須包含 3 種類型	通過
密碼不可與帳號相同	通過
所有身分驗證控制機制實做於伺服器端	通過
超過了 3 次嘗試次數後，系統會鎖定帳戶	通過
身分驗證失敗不會顯示帳號錯誤或是密碼錯誤等詳細失敗原因	通過
系統管理員登入的來源 IP 必須寫於白名單內	通過
應用程式使用了框架預設的會話((Session))管理控制實做	通過
會話在指定時間後會過期	通過
使用者登出後會話即失效	通過
針對使用者輸入欄位有安全控制可阻止緩衝區溢位	通過
創建密碼時有進行雜湊(Hash 運算)並且使用 Salt	通過
每一個使用者類型，都能夠正確的取用其所被授權的頁面	通過
每一個使用者類型無法越權使用其未經授權的頁面、功能與資料檔案	通過
無法越權使用未經授權的功能與資料	通過
禁止目錄瀏覽	通過
記錄所有的存取控制的成功與失敗的結果	通過

本文件之智慧財產權屬行政院資通安全辦公室所有。

驗證項目	測試結果
在伺服器端實現所有的日誌控制	通過
系統日誌拒絕未經授權的存取及修改	通過
日誌包含時間郵戳(timestamp)、事件分級、帳號、事件來源 IP 以及事件描述	通過
日誌沒有包含敏感資訊，例如個人資料	通過
應用系統輸出的錯誤訊息或堆疊追蹤(stack traces)沒有包含敏感資訊，例如會話 ID 及個人資料	通過
URL 參數不可用來傳送敏感資料	通過

資料來源：本指引整理

5.3.2. 源碼安全性檢測

5.3.2.1. 測試目標

以源碼安全性檢測工具，明確地指出有潛在安全問題的源碼位置及建議修補方式。

5.3.2.2. 測試方式

使用 HP Fortify SCA 3.8 源碼檢測工具進行自動化源碼安全性檢測。

進行方式係使用 HP Fortify SCA 3.8 中的 HP Fortify Scan Wizard，在命令列提示模式下進行掃描。

5.3.2.3. 測試完成的條件

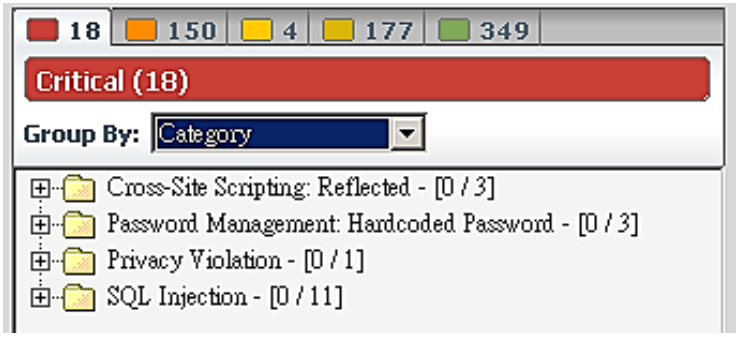
HP Fortify SCA 掃描完成，檢測結果經測試人員分析後判定是否存在誤判或是需要進一步處理之安全問題。

5.3.2.4. 測試結果

本文件之智慧財產權屬行政院資通安全辦公室所有。

「雲端版技服中心網站」程式碼共計 682 個檔案，總計 103,914 行。

管理維護網站掃描結果畫面詳見圖 23 所示。



資料來源：本指引整理

圖23 HP Fortify SCA 3.8 對管理維護網站掃描果

由檢測結果顯示 Critical 等級危險的項目有 18 個，High 等級危險的項目則有 150 個。Critical 等級的結果包含以下 4 個類型：

●跨網站腳本(Cross-Site Scripting:Reflected)

●密碼寫死(Hardcoded Password)

●隱私權違反(Privacy Violation)

●SQL 注入(SQL Injection)

但 HP Fortify SCA 3.8 會為求儘可能安全，故只要可疑的都會列入結果，故難免會有誤判的情況。因此需由開發人員以及具有資安知識的人員一起針對結果進行分析，判斷是否為真正的安全問題或是誤判。

在人工判斷階段，亦可以手動把人工判別為無安全疑慮的項目逐項關閉，讓掃描結果都變成 0，但這樣做費時且除為了於報告呈現外並沒有太大意義，故一般只是檢查分析掃描結果，認為沒問題就忽略不管。

針對誤判的項目建議可以記錄下來，將來若進行重新測試時可以作為是否

本文件之智慧財產權屬行政院資通安全辦公室所有。

誤判的參考。

對於維護管理網站的掃描結果，雖有 Critical 等級危險，但經分析為誤判，或不屬於該方案的檔案。

5.3.3. 弱點掃描

5.3.3.1. 測試目標

以黑箱方式檢測系統之弱點。

5.3.3.2. 測試方式

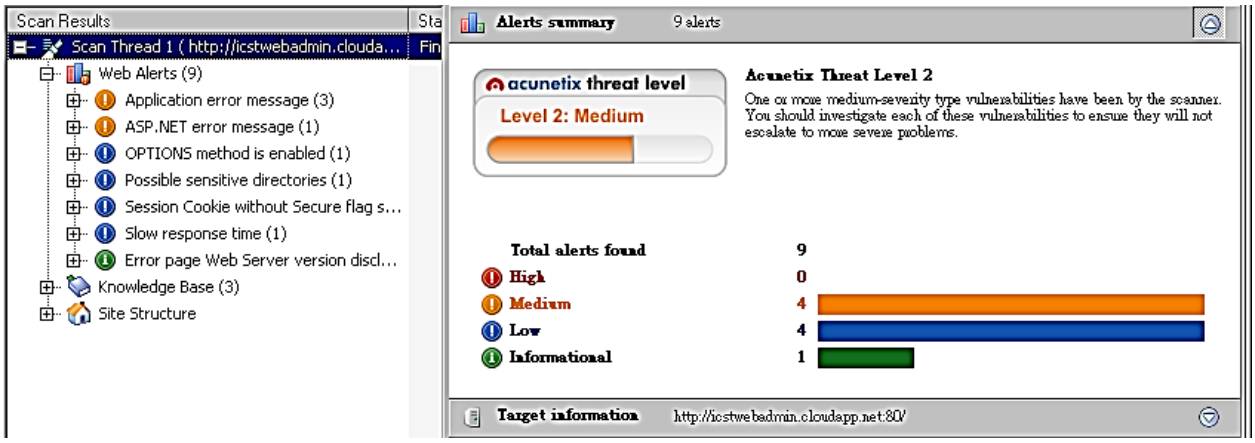
使用 Acunetix 公司的 Web Vulnerability Scanner v8.0 弱點掃描自動化工具進行檢測，輸入維護管理網站之網址後即可進行測試。

5.3.3.3. 測試完成的條件

弱點掃描完成，檢測結果經測試人員分析後判定無安全問題。

5.3.3.4. 測試結果

AWVS 測試結果畫面詳見圖 24 所示。



資料來源：本指引整理

圖24 WVS 對管理維護網站掃描結果

本文件之智慧財產權屬行政院資通安全辦公室所有。

結果顯示並沒有 High 等級的安全性問題，Medium 等級的問題有 4 項，包含 3 項應用程式錯誤訊息，以及 1 項 ASP.NET 錯誤訊息，經人工確認後判斷其只是網頁上顯示的錯誤訊息，並沒有洩漏重要資訊以及對系統造成影響。

5.4.結語

本小節以資安管考系統為案例介紹如何進行安全測試，包含進行安全需求驗證以及分別使用源碼檢測工具 HP Fortify Static Code Analyzer 與滲透測試工具 Acunetix Web Vulnerability Scanner 進行弱點掃描。

本文件之智慧財產權屬行政院資通安全辦公室所有。

6. 結論

本指引乃延續安全軟體發展流程指引之內容，針對系統安全測試流程作法進行探討，歸納出進行系統安全測試之實行重點，協助政府機關之資訊與資安人員做好軟體安全的防護工作。並整理與測試相關之國際標準，包含如資訊安全管理標準 ISO/IEC 27001:2013、測試文件標準 IEEE 829:2008、軟體測試標準 ISO/IEC/IEEE 29119:2013，以及 OWASP 測試指引(OWASP Testing Guide)等，其中 SO/IEC/IEEE 29119:2013 提供的軟體測試管理方法可適用在任何類型的軟體測試任務，故安全測試亦可參考使用。OWASP 測試指引則提供了安全測試的框架，可套用在安全軟體發展生命週期內的各個階段內，以驗證安全需求。

安全測試的目標可從安全需求所獲得，報告中針對安全需求的類型與其萃取方式，說明驗證安全需求實務活動之要項，包含需求、設計、開發、測試、驗收以及軟體部署與維運等階段的安全需求驗證。

本指引說明常用的安全性測試技術，包含源碼靜態分析以及滲透測試，以協助相關人員了解進行軟體安全測試時應該注意的事項及進行方式。最後並以實際案例介紹安全需求驗證、源碼檢測，以及弱點掃描的進行方式。

本指引主要為提供機關於進行安全功能驗證之基礎，可作為自行開發或委外管理系統的參考依據，期能落實安全軟體生命週期之實踐，透過測試以驗證整體資訊系統之安全性，強化政府整體安全軟體發展能量。

本文件之智慧財產權屬行政院資通安全辦公室所有。

7. 參考文獻

[1] NIST,「The Economic Impacts of Inadequate Infrastructure for Software Testing」 http://www.nist.gov/director/prog-ofc/report02-3.pdf

[2] MITRE, Being Explicit About Weaknesses, Slide 30, Coverage of CWE http://cwe.mitre.org/documents/beingexplicit/BlackHatDC_BeingExplicit_Slides.ppt

[3] ISO/IEC 27001, Information security management http://www.iso.org/iso/home/store/catalogue_tc/catalogue_detail.htm?csnumber=54534,ISO

[4] IEEE 829-2008, IEEE Standard for Software and System Test Documentation http://ieeexplore.ieee.org/xpl/login.jsp?tp=&arnumber=4578383&url=http%3A%2F%2Fieeexplore.ieee.org%2Fiel5%2F4578271%2F4578382%2F04578383.pdf%3Farnumber%3D4578383

[5] ISO/IEC/IEEE 29119, Software Testing http://softwaretestingstandard.org/

[6] ISO/IEC/IEEE 29119 Part2, Test Processes http://softwaretestingstandard.org/part2.php

[7] ISO/IEC/IEEE 29119 Part3, Test Documentation http://softwaretestingstandard.org/part3.php

[8] SO/IEC/IEEE 29119-4, Test Techniques http://softwaretestingstandard.org/part4.php

[9] ISO/IEC/IEEE 29119-5, Keyword-Driven Testing http://softwaretestingstandard.org/part5.php

本文件之智慧財產權屬行政院資通安全辦公室所有。

[10]OWASP, The Open Web Application Security Project
https://www.owasp.org/index.php/Main_Page

[11]OWASP, OWASP Testing Guide v4
https://www.owasp.org/index.php/OWASP_Testing_Project

[12]UML, Unified Modeling Language™ (UML®) Resource Page
http://www.uml.org/

[13]ISO/IEC 17799:2005, Code of practice for information security management
http://www.iso.org/iso/catalogue_detail?csnumber=39612

[14]SOX-404, http://sarbanes-oxley-101.com/SOX-404.htm

[15]Visa, Merchant 指引 http://usa.visa.com/merchants/index.jsp

[16]OWASP, Document security relevant requirements
https://www.owasp.org/index.php/Document_security-relevant_requirements

[17]Official ISC2 Guide to the CSSLP, ISC2

[18]An Introduction to Computer Security: NIST SP800-12

[19]OWASP, Application Security Verification Standard Project
https://www.owasp.org/index.php/Category:OWASP_Application_Security_Verification_Standard_Project

[20]OWASP, OWASP Top 10 2013
https://www.owasp.org/index.php/Top_10_2013-Top_10

[21]SeleniumHQ, Browser Automation http://www.seleniumhq.org/

[22]Geb, Very Groovy Browser Automation http://www.gebish.org/

[23]啟動 Geb－網站自動化測試之美 http://learngeb.readbook.tw/

本文件之智慧財產權屬行政院資通安全辦公室所有。

[24]Sikuli Scrupt http://www.sikuli.org/

[25]IeUnit https://code.google.com/p/ieunit/

[26]Badboy Software http://www.badboy.com.au/

[27]Web Application Security Consortium, Web Application Security Scanner Evaluation Criteria Version 1.0 http://projects.webappsec.org/w/page/13246986/Web%20Application%20Security%20Scanner%20Evaluation%20Criteria

[28]CWE/SANS, TOP 25 Most Dangerous Software Errors http://www.sans.org/top25-software-errors/

[29]NIST, SP500-269 http://samate.nist.gov/docs/webapp_scanner_spec_sp500-269.pdf

[30]中華電信程式碼線上安全掃描 https://scod.hinet.net/scodmain/index.html

[31]Findbugs, Find Bugs in Java Programs http://findbugs.sourceforge.net/

[32]Microsoft Code Analysis Tool .NET (CAT.NET) v1 CTP - 32 bit http://www.microsoft.com/en-us/download/details.aspx?id=19968

[33]Cppcheck http://cppcheck.sourceforge.net/

[34]PMD http://pmd.sourceforge.net/

[35]Findbugs, Eclipse plugin http://findbugs.cs.umd.edu/eclipse

[36]Findbugs, maven plugin http://mojo.codehaus.org/findbugs-maven-plugin/

[37]Software Quality Environment https://kenai.com/projects/sqe/pages/Home

本文件之智慧財產權屬行政院資通安全辦公室所有。

[38]Findbugs, Jenkins plugin
https://wiki.jenkins-ci.org/display/JENKINS/FindBugs+Plugin

[39]Findbugs, Hudson plugin
http://wiki.hudson-ci.org/display/HUDSON/FindBugs+Plugin

[40]Findbugs, IntellijFindBugsPlugins
http://code.google.com/p/findbugs/wiki/IntellijFindBugsPlugins

[41]FxCop http://msdn.microsoft.com/en-us/library/bb429476(v=vs.80).aspx

[42]Cppcheck, Cppcheclipse
https://code.google.com/a/eclipselabs.org/p/cppcheclipse/?redir=1

[43]Cppcheck, gedit plugin https://www.openhub.net/p/gedit-cppcheck

[44]Cppcheck, Hudson plugin
http://wiki.hudson-ci.org/display/HUDSON/Cppcheck+Plugin

[45]Cppcheck, Jenkins plugin
https://wiki.jenkins-ci.org/display/JENKINS/Cppcheck+Plugin

[46]Cppcheck, Static source code analysis tool for C and C++ code
http://sourceforge.net/apps/mediawiki/cppcheck/index.php?title=Mercurialhook

[47]Omerez http://omerez.com/automatic-static-code-analysis/

[48]Cppcheck plugin
https://github.com/VioletGiraffe/cppcheck-vs-addin/releases/latest

[49]Maven, pmd plugin http://maven.apache.org/plugins/maven-pmd-plugin/

[50]Eclipse, pmd plugin http://pmd.sourceforge.net/integrations.html#eclipse

本文件之智慧財產權屬行政院資通安全辦公室所有。

[51]Software Quality Environment, http://kenai.com/projects/sqe/

[52]Jbuilder, pmd plugin http://pmd.sourceforge.net/integrations.html#jbuilder

[53]Jdeveloper, pmd plugin
http://pmd.sourceforge.net/integrations.html#jdeveloper

[54]IDEA, pmd plugin http://pmd.sourceforge.net/integrations.html#idea

[55]阿碼科技 http://armorize.com

[56]Checkmarx http://www.checkmarx.com

[57]Coverity http://www.coverity.com

[58]HP, Application security http://fortify.com

[59]Klocwork http://www.klocwork.com

[60]IBM Appscan
http://www-01.ibm.com/software/rational/products/appscan/source/

[61]Veracode http://www.veracode.com

[62]White hat security https://www.whitehatsec.com

[63]「技術服務中心」政府機關滲透測試服務委外服務案建議書徵求文件 V1.0

[64]維基百科, Metasploit http://zh.wikipedia.org/zh-hant/Metasploit

[65]Burp suite http://portswigger.net/burp/

[66]Acunetix, AWVS https://www.acunetix.com/vulnerability-scanner/

本文件之智慧財產權屬行政院資通安全辦公室所有。

[67]HP WebInspect

http://www8.hp.com/us/en/software-solutions/webinspect-dynamic-analysis-dast/

[68]HP WebInspect Agent

http://h30499.www3.hp.com/t5/Fortify-Application-Security/WebInspect-10-20-Release/ba-p/6463432#.VGWjUDSUcmM

本文件之智慧財產權屬行政院資通安全辦公室所有。